

算法基础与在线编程实验教程 (C++版)



李胜睿、卢伟清、张琦、林颖、高岩、蔡炳跃等编著

二〇二〇年九月十五日

厦门大学本科教材资助项目

目录

目录	I
前言	V
第1章 南强百题语法篇	1
1.1 求连续整数和NQ001	1
1.2 字符比大小NQ002	2
1.3 计算线段长度NQ003	3
1.4 求圆球的体积NQ004	4
1.5 实数绝对值NQ005	5
1.6 成绩分区NQ006	6
1.7 日期计算NQ007	8
1.8 求数列中奇数之积NQ008	9
1.9 循环求解NQ009	10
1.10 小鲁记账NQ010	11
1.11 构造数列并求和NQ011	12
1.12 水仙花数NQ012	14
1.13 倒数数列求和NQ013	15
1.14 凤凰花开吃杨梅NQ014	17
1.15 辩论赛评分NQ015	18
1.16 十六进制加法NQ016	19
1.17 计算绩点NQ017	21
1.18 计算折线长度NQ018	23
1.19 最大公约数NQ019	25
1.20 时钟夹角NQ020	26
1.21 废墙重建NQ021	28
1.22 竞选投票NQ022	30
1.23 桌球比赛NQ023	32
1.24 珠穆朗玛峰测距NQ024	33
1.25 胜利会师NQ025	34
1.26 公式变代码NQ026	35

1.27 丑数 ^{NQ027}	37
1.28 统计0-1矩阵中1的数量 ^{NQ028}	38
1.29 回文数 ^{NQ029}	39
1.30 小球进盒子 ^{NQ030}	41
1.31 解密文本初阶 ^{NQ031}	42
1.32 首字母大写 ^{NQ032}	44
1.33 手机短号 ^{NQ033}	45
1.34 求小数点后的数 ^{NQ034}	46
1.35 字符串奇偶位互换 ^{NQ035}	47
1.36 合并字符串初阶 ^{NQ036}	48
1.37 数列的绝对值排序	49
1.38 数字字符的统计 ^{NQ038}	51
1.39 合法标识符 ^{NQ039}	52
1.40 每行诗最大元素 ^{NQ040}	53
1.41 统计字符个数初步 ^{NQ041}	54
1.42 不包含重复字符的最长子串长度 ^{NQ042}	57
1.43 统计班级成绩 ^{NQ043}	58
1.44 密码安全问题 ^{NQ044}	60
1.45 奇偶校验 ^{NQ045}	62
1.46 悬垂问题 (Overhang) ^{NQ046}	63
1.47 数组去重问题 ^{NQ047}	65
1.48 寻找多数元素 ^{NQ048}	66
1.49 数字5的封杀 ^{NQ049}	67
1.50 排序考试 ^{NQ050}	69
第2章 南强百题基础算法篇	71
2.1 数组元素交换 ^{NQ051}	71
2.2 大数排序 ^{NQ052}	72
2.3 找到最长回文字串 ^{NQ053}	74
2.4 ACM纳新赛 ^{NQ054}	76
2.5 最小公倍数 ^{NQ055}	78
2.6 三数判断 ^{NQ056}	79
2.7 灯的开关 ^{NQ057}	81
2.8 两数之和 ^{NQ058}	82

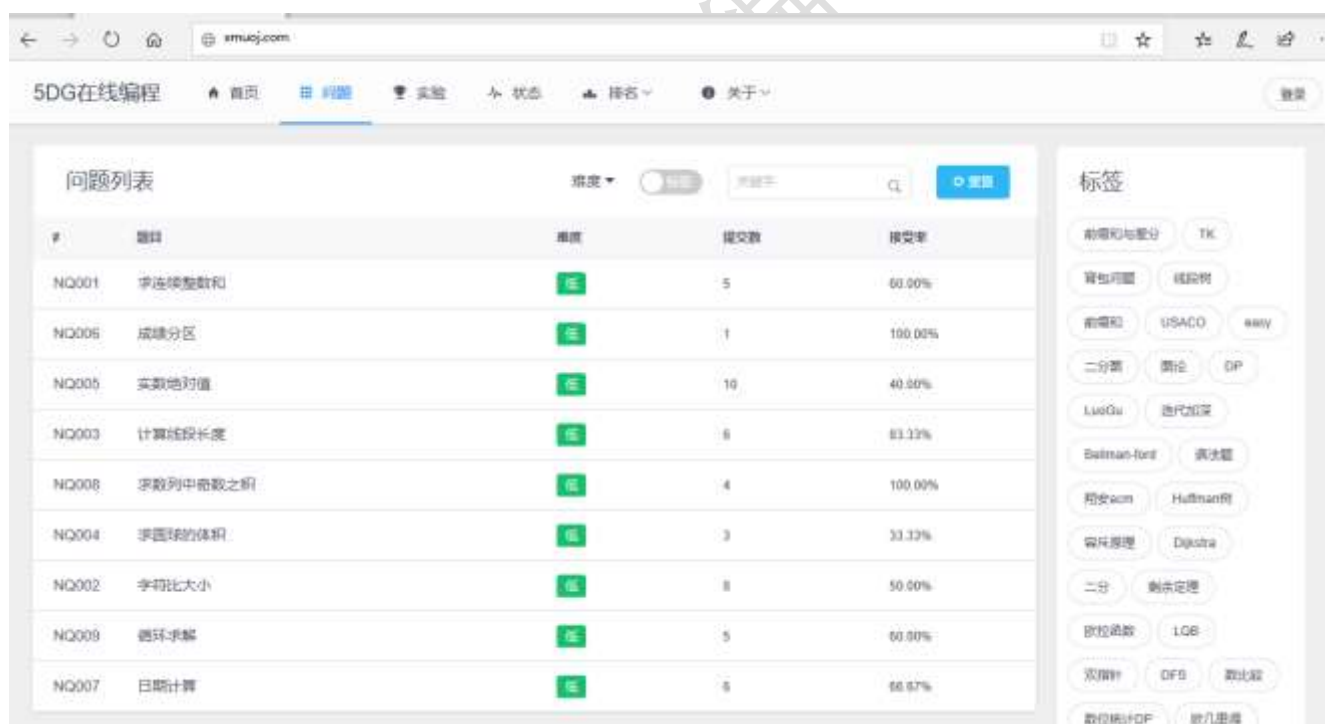
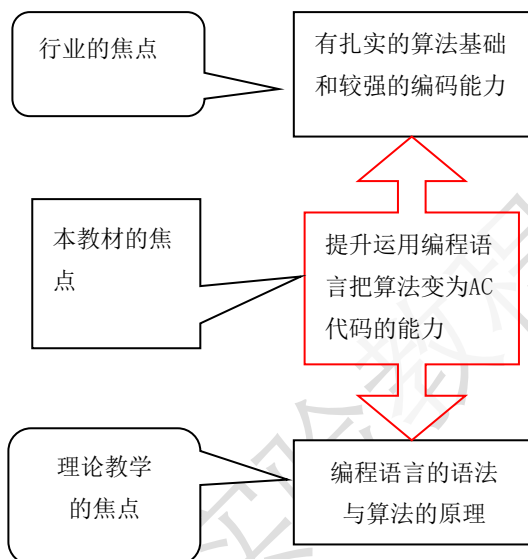
2.9 三数之和 ^{NQ059}	83
2.10 四数之和 ^{NQ060}	85
2.11 求矩形重叠面积 ^{NQ061}	87
2.12 二进制中1的个数 ^{NQ062}	89
2.13 求上台阶方案数 ^{NQ063}	90
2.14 小鲁与猫的羁绊 ^{NQ064}	91
2.15 计算 $N!$ ^{NQ065}	92
2.16 求谦虚数 ^{NQ066}	94
2.17 贴瓷砖 ^{NQ067}	96
2.18 求方程的根 ^{NQ068}	97
2.19 寻找指定和的整数对 ^{NQ069}	98
2.20 寻找最接近的元素 ^{NQ070}	100
2.21 手撕农夫与牛的问题 ^{NQ071}	103
2.22 N皇后编程问题 ^{NQ072}	106
2.23 递归求波兰表达式 ^{NQ073}	107
2.24 求八皇后的第n种解 ^{NQ074}	109
2.25 P进制 ^{NQ075}	111
2.26 二进制取幂法 ^{NQ076}	113
2.27 求第K大的正整数 ^{NQ077}	114
2.28 四则运算表达式求值 ^{NQ078}	116
2.29 算24 ^{NQ079}	118
2.30 质数环 ^{NQ080}	121
2.31 试除法求约数 ^{NQ081}	123
2.32 试除法分解质因数 ^{NQ082}	125
2.33 求最长公共子列长度 ^{NQ083}	127
2.34 同构字符串 ^{NQ084}	128
2.35 数字三角形 ^{NQ085}	130
第3章 南强百题自强进阶篇	132
3.1 林克的01背包 ^{NQ086}	132
3.2 踩盾滑行 ^{NQ087}	135
3.3 拦截雷电箭(1) ^{NQ088}	138
3.4 拦截雷电箭(2) ^{NQ089}	140
3.5 击杀黄金蛋糕人马 ^{NQ090}	141

3.6 骑士林克的怜悯 (1) ^{NQ091}	144
3.7 骑士林克的怜悯 (2) ^{NQ092}	147
3.8 突袭路线 ^{NQ093}	150
3.9 全面反击 ^{NQ094}	153
3.10 真假记忆碎片 ^{NQ095}	155
3.11 寻找林克的回忆 (1) ^{NQ096}	159
3.12 寻找林克的回忆 (2) ^{NQ097}	163
3.13 寻找林克的回忆 (3) ^{NQ098}	166
3.14 最省赛程 ^{NQ099}	170
3.15 滚石柱 ^{NQ100}	174
后记	180

前言

高校计算机的理论教学的焦点是搭建学生的编程和算法的理论基础，并且讲授1-2门编程语言的语法；而这个时代需要大量的既有扎实的算法基础又有较强编码能力的人才。本书希望能够成为计算机理论教学的有益补充，本书采用线上线下相结合的方式，基于5DG在线编程平台(xmuoj.com)提升学生运用编程语言把算法变成正确程序的能力。

本书的设计是采用线上线下结合的方式，来达到提升学生编码能力的目标。在线上，我们在5DG在线编程平台(xmuoj.com)上搭建了南强系列题库——NQ100（包含100道在线编程题），并且提供在线的题解链接或者视频讲解；在线下，我们提供这本包含NQ100题题解的教材，帮助学生透过代码，直接学习如何运用编程语言，如何使用程序解题，如何优化代码等重要的技能。



5DG在线编程(<http://xmuoj.com>)

NQ100涵盖ACM入门的多种类型题目，100道题目从易到难编排。前50题为基础题，帮助零基础的初学者逐步提高自己的编程水平；后50题为提高题，供达到一定水平学生进阶训练用。本书提供了NQ100的全部AC代码，学生可以采用直接学习AC代码的方式获取如何编写代码的实际经验。我们希望鼓励学生养成透过阅读大量的优质代码来学习如何写代码的习惯，并且触类旁通开始操练自己动手写出高质量的代码。

我们希望透过提供NQ100的全部AC代码的这种方式，建立学生到xmuoj.com上做题的信心，尝到自己

能够解题并且透过努力能够AC难题的成就感，并由此尝到编程的甜蜜从而愿意投资更多的时间学习编程。与此同时，如果学生能在本书的帮助下，成功AC所有的NQ100题，那么一定可以收获好几十个小时甚至上百个小时的编程经验，而5DG在线编程平台的自动判题系统(OJ)，能够24小时不歇息的提供判题服务，让学生可以得到立即而精准的代码运行结果反馈。

我们期待透过这种线上和线下互动结合的教学和学习方式，锻炼学生分析问题的能力和编程解决问题的能力，并且可以为算法课程及计算机其他专业课程的编程学习打好基础。我们相信，一旦学生养成了在线编程的学习习惯，并且持之以恒的操练在线编程，经过多年的积累，当学生面临就业或者保研的时候，一定会拥有更强的竞争力。

本书可以作为低年级本科生计算机编程课的实验教材，例如《C语言程序设计》、《算法基础与在线编程》、《学科实训等》、也可以作为本科二、三年级专业课《数据结构》、《算法设计与分析》的实验教材，与此同时本书也可以作为零基础想要入门ACM等编程竞赛的同学们的培训资料。

本书由信息学院实验教学中心的5DG在线编程团队的老师们编著，团队成员有卢伟清老师、李胜睿老师、张琦老师、林颖老师、高岩老师、蔡炳跃老师等等。在过去一年的时间内，5DG在线团队的老师们一起配合，既在线上创作题库，又在线下编写教材。NQ100题库和本书的出版涉及到大量的文字和编码工作，这其中的酸甜苦辣只有一起参与创作、一起编码、一起写书的团队成员们才能够感同身受。非常感恩在这样一件艰难的挑战面前，我们一直不是一个人，乃是有整个团队一起携手努力。

本书得以成稿，与5DG在线编程团队每个成员的默默的付出是分不开的。看到每个老师为了下一代的益处，都愿意付代价，投资心力，投资时间，发挥各自的恩赐，为要造就新一代人才，笔者深感荣幸。

在本书的撰写过程中特别感谢2019级信息学院的高逸飞、王河山、黄盟涵和方明俊同学，他们参与5DG在线编程平台的建设，并且参与NQ100的题库建设（题面梳理、题面撰写、AC代码编写、造测试数据等工作）。而目前5DG在线编程平台XMUOJ作为《厦门大学ACM竞赛的集训与竞技OJ系统设计》这个国家级大创项目的初步成果，已经上线供ACM兴趣班使用。未来还会继续建设XMUOJ使之成为支撑ACM竞赛集训与竞技需要的OJ系统，与此同时又满足信息学院专业课程的编程训练的需要。

此外，还要特别感谢2019级信息学院的同学们，尤其是ACM提高班和ACM竞赛队的同学们，他们在本书的早期题库建设阶段参与题库建设，参与造测试数据，并且贡献了不少高质量的AC代码，NQ100题库的成型与2019级同学们的贡献是分不开的。

最后，要特别感谢信息学院计算机系的黄绍辉老师，若没有他的鼎力相助，就没有这个5DG在线编程平台XMUOJ的上线，万分感谢。同时也要特别感谢嘉庚学院计算机系的蔡明老师，没有他在在线编程领域的开拓性工作 (<http://xujcoj.org/home>) 和在嘉庚学院计算机教学创新的实战经验分享和以及取得丰硕成果的见证，也不可能有今天的xmuoj，非常感谢蔡明老师的鼎力支持。

最后的最后，要感谢厦门大学这孕育莘莘学子的摇篮，这南强NQ100的题目和故事取材于我们所爱的这所南方之强。愿使用本书在线编程的同学们，能够像本书的主人公小鲁一样，谦卑受教，坚持不懈，努力操练，自强不息，从入门到成才！

仅以此书献给我们所爱的厦门大学和厦门大学信息学院。

5DG在线编程团队

2020年9月15日

厦门大学海韵园

第1章 南强百题语法篇

1.1 求连续整数和^{NQ001}

描述

小华立志想要成为未来的数学家，迅速的从数字中找到需要的数字，并且迅速的完成计算是他的必备能力。小华给自己设计了一个算数测试，想要超越高斯一年级的水平。一年级的高斯计算的是从1开始连续100个数字的和，小华要计算的是从a开始连续n个数字的和。输入数据分为两行，第一行是一个整数a，第二行包含若干整数，其中第一个大于0的数是n。

请计算从a开始n个连续整数的和。

数据范围:

$1 \leq a \leq 10000$

$1 \leq n \leq 10000$

输入

输入数据分为两行，第一行是一个整数a，第二行包含若干整数，其中第一个大于0的数是n。

输出

输出一个整数，即从a开始n个连续整数的和。

输入样例

```
1
-3 -4 -13 0 -7 100 208 12 -9
```

输出样例

```
5050
```

参考代码(C++版)

```
1. // NQ001 求连续整数和
2. #include <iostream>
3. using namespace std;
4. int main() {
5.     int a, n;
6.     cin >> a; //读入 a
7.     //过滤负数和 0 的空循环
8.     while (cin >> n, n <= 0) ; // cin 每次读写都会越过空格和换行符
```

```

9.   int res = 0;                                // res 存储结果值
10.  for (int i = 0; i < n; i++) res += a++;      //从 a 开始连续 n 个数累计
11.  cout << res << endl;                        //输出结果
12.  return 0;
13. }

```

1.2 字符比大小^{NQ002}

描述

小鲁刚接触编程不久，但他一直不明白计算机中存储的字符跟数值一样是有大小关系的，但是谁大谁小，他一直搞不清楚。你能帮他解除这个疑惑吗？

请输入N组数据，每组数据包含三个字符，然后按从小到大的顺序输出这三个字符。

输入

第一行输入数值N

输入N行，每行3个字符。

输出

输出有N行，每行是排序后的3个字符，用空格隔开。

输入样例

```

3
awe
bsd
aZt

```

输出样例

```

a e w
b d s
Z a t

```

思路：

利用ASCII码值比较大小。

参考代码(C++版)

```

1.  //NQ002 字符比大小
2.  #include <iostream>
3.  #include <cstring>
4.  #include <algorithm>
5.  using namespace std;
6.  int main()

```

```

7. {
8.     int n;
9.     cin >> n;
10.    string line;
11.    for (int i = 0; i < n; i++)
12.    {
13.        cin >> line;
14.        sort(line.begin(), line.end()); //使用算法 sort 排序
15.        cout << line[0] << ' ' << line[1] << ' ' << line[2] << endl;
16.    }
17.    return 0;
18. }

```

1.3 计算线段长度^{NQ003}

描述

文学青年小鲁想学习编程，他去请教小华自己该如何入门。

小华说：想学编程，得从基础开始！这样吧，先给你出道简单题：

计算一下直角坐标系中以两点 $M(mx, my)$ ， $N(nx, ny)$ 为端点的线段长度。

输入

第一行输入正整数 n ，代表有 n 个测试实例。

第二行开始输入 n 行，每行代表一个测试实例。包含由空格分开的4个实数，分别表示 mx, my, nx, ny 。

输出

对于每组输入数据，输出一行，结果保留两位小数。

输入样例

```

2
0 0 0 1
0 3 6 4

```

输出样例

```

1.00
6.08

```

参考代码(C++版)

```

1. //NQ003 计算线段长度
2. #include <iostream>

```

```

3. #include <cmath>
4. #include <iomanip>
5. using namespace std;
6. int main()
7. {
8.     int n;
9.     cin >> n; //n 组数
10.    double a, b, c, d;
11.    while (n--)
12.    {
13.        //读入 4 个数
14.        cin >> a >> b >> c >> d;
15.        //固定小数位数
16.        cout << setiosflags(ios::fixed);
17.        //结果保留 2 位小数
18.        cout<<setprecision(2)<<sqrt((a-c)*(a-c)+(b-d)*(b-d))<<endl;
19.    }
20.    return 0;
21. }

```

1.4 求圆球的体积^{NQ004}

描述

上一道简单题轻松搞定，小华给小鲁出了第二道简单题，继续帮助小鲁建立信心。

第二道题题目如下：

给出一组圆球的半径，请计算出不同半径的圆球体积。

输入

首先输入整数N，表示有N组半径。

接着每行输入一个实数，表示半径，一共输入N行。

输出

每行输出一个实数，表示圆球的体积，实数保留三位小数。

输入样例

```

3
2.5
8.4
6.9

```

输出样例

```
65.450
2482.713
1376.055
```

参考代码(C++)

```
1. //NQ004 求圆球的体积
2. #include <iostream>
3. #include <cmath>
4. #include <iomanip>
5. using namespace std;
6. #define PI 3.1415926
7. int main()
8. {
9.     int n;
10.    cin >> n; //n 组数
11.    double r; //半径值
12.    while (n--)
13.    {
14.        cin >> r;
15.        //固定小数位数
16.        cout << setiosflags(ios::fixed);
17.        //结果保留 3 位小数
18.        cout << setprecision(3) << PI*r*r*r*4/3 << endl;
19.    }
20.    return 0;
21. }
```

1.5 实数绝对值^{NQ005}

描述

看着小鲁AC后开心的笑，小华也和他一起开心，于是继续出简单题帮助小鲁在微小的长进中建立编程的自信。

简单题第三题：求实数的绝对值。

输入

输入数据有多组，每组占一行，每行包含一个实数。

输出

对于每组输入数据，输出它的绝对值，要求每组数据输出一行，结果保留两位小数。

输入样例

```
135
-247.00
```

输出样例

```
135.00
247.00
```

参考代码(C++)

```
1. //NQ005 实数绝对值
2. #include <iostream>
3. #include <cmath>
4. #include <iomanip>
5. using namespace std;
6. int main()
7. {
8.     int n;
9.     double x; //实数
10.    while (cin >> n)
11.    {
12.        cin >> x;
13.        //固定小数位数
14.        cout << setiosflags(ios::fixed);
15.        //结果保留 2 位小数
16.        cout << setprecision(2) << fabs(x) << endl;
17.    }
18.    return 0;
19. }
```

1.6 成绩分区 NQ006

描述

教育是一个民族要崛起的必备条件。小嘉排除万难，创办学校。

艰苦创校期，小嘉自己身兼数职，既是校长，又是后勤总管，既要为办校募款，又要为学校广罗人才。于此同时，还要亲自在一线教学，评估学生的学习状况。

小嘉每学期最重要的工作之一就是评估每个学生的学习成绩属于哪个档次，并且根据原始数据调整新学期的教学方案和人才引进方案。

给定一个 $[0, 100]$ 之间的浮点数Score，请你判断该数字属于以下哪个区间：

A: [90, 100] B: [80, 90) C: [70, 80) D: [60, 70) E: [0, 60)

如果Score小于0或大于100，则程序输出“Wrong Score!”，表示超出范围。

开区间 (a,b) ：在实数 a 和实数 b 之间的所有实数，但不包含 a 和 b 。

闭区间 $[a,b]$ ：在实数 a 和实数 b 之间的所有实数，包含 a 和 b 。

输入

输入数据有 n 行，每行是一个浮点数 $Score$ ，代表学生的综合成绩。

输出

对于每行输入数据，请判定该数 $Score$ 落在哪个区间，并且输出完整区间。

例如

输入数据是99.9 程序输出 A:[90,100]

输入数据是85，程序输出 B:[80,90)

若输入数据不在 $[0,100]$ 区间内，程序输出：“Wrong Score!”。

输入样例

```
99.9
88.8
77.7
66.6
55.5
-77.7
```

输出样例

```
A:[90,100]
B:[80,90)
C:[70,80)
D:[60,70)
E:[0,60)
Wrong Score!
```

参考代码(C++)

```
1. //NQ006 成绩分区
2. #include <iostream>
3. using namespace std;
4. int main()
5. {
6.     double score;
7.     while (cin >> score) //读入 Score
8.     {
9.         if (score >= 90 && score <= 100)
10.            cout << "A:[90,100]" << endl;
11.         else if (score >= 80 && score < 90)
12.            cout << "B:[80,90)" << endl;
```

```

13.         else if (score >= 70 && score < 80)
14.             cout << "C:[70,80)" << endl;
15.         else if (score >= 60 && score < 70)
16.             cout << "D:[60,70)" << endl;
17.         else if (score >= 0 && score < 60)
18.             cout << "E:[0,60)" << endl;
19.         else
20.             cout << "Wrong Score!" << endl;
21.     }
22.     return 0;
23. }

```

1.7 日期计算^{NQ007}

描述

学会基本的输入输出、函数调用之后，小鲁对于新的挑战跃跃欲试。小华知道现在是帮助小鲁编程过程需要考虑周全的好时候，他给小鲁出了一道新题：

对于某个日期，请计算该日期是这一年的第几天。（需要考虑是否是闰年）

输入

输入多组数据，每组占一行，数据格式为YYYY/MM/DD组成

输出

输出一个整数，表示日期是本年的第几天。

输入样例

```

2008/06/01
2006/09/10
1920/05/31
2020/12/31

```

输出样例

```

153
253
152
366

```

参考代码(C++)

```

1. //NQ007 日期计算
2. #include <iostream>

```



```

3. using namespace std;
4. //判断是否是闰年
5. inline bool isLeap(int y){
6.     return (y%4 ==0 && y%100 !=0 || y%400==0 );
7. }
8. //计算第几天
9. int DayInYear(int y, int m, int d)
10. {
11.     //int _day = 0;
12.     int days[12]={31,28,31,30,31,30,31,31,30,31,30,31};
13.     if(isLeap(y))
14.         days[1] = 29;
15.     for(int i=0; i<m - 1; ++i)
16.     {
17.         d += days[i];
18.     }
19.     return d;
20. }
21. int main(){
22.     int y,m,d;
23.     char p,q;
24.     while(scanf("%d%c%d%c%d",&y,&p,&m,&q,&d)!=EOF)
25.     {
26.         printf("%d\n",DayInYear(y,m,d)); //输出第几天
27.     }
28.     return 0;
29. }

```

1.8 求数列中奇数之积 NQ008

描述

连续AC多道题，小鲁对编程兴趣大增，他渴望能够成长得再快一点。

小华知道如何处理循环是小鲁必须熟练掌握的，他立刻给小鲁出了道循环题：

一个有m个元素（整数）的数列，求其中所有奇数之积。

输入

输入数据有若干行，每行代表一个数列。

每行的格式如下：第一个数字m代表数列长度，接下去连续m个被空格隔开的整数表示数列元素。

输出

输出若干行，每行代表数列中所有奇数之积。

输入样例

5 1 2 3 4 5

输出样例

120

参考代码(C++)

```
1. //NQ008 求数列中奇数之积
2. #include <iostream>
3. using namespace std;
4. int main()
5. {
6.     int m;
7.     while (cin>>m)
8.     {
9.         long long x,res =1;
10.        for (int i=0;i<m;i++) {
11.            cin>>x;
12.            if(x%2) res*=x;//如果是奇数就累乘
13.        }
14.        cout<<res<<endl;//输出结果
15.    }
16.    return 0;
17. }
```

1.9 循环求解^{NQ009}

描述

在循环中不单单可以处理某一需要，还可以同时处理多个需要。

看着刚解决完上一道循环题的小鲁，小华稍微加了一点难度，又出了道循环题：

给定一个区间 $[a,b]$ ，求该区间中所有偶数的平方和以及所有奇数的立方和。

输入

输入有若干行，每行包含两个数 a 和 b ，代表区间的起点和终点 ($a \leq b$)

输出

对于每组输入数据，输出两个整数，分别表示所有偶数的平方和以及所有奇数的立方和。

输入样例

3 8

2 9

输出样例

116 495
120 1224

参考代码(C++)

```
1. //NQ009 循环求解
2. #include <iostream>
3. using namespace std;
4. int main()
5. {
6.     int a, b;
7.     while (cin >> a >> b)
8.     {
9.         if (a > b)
10.            swap(a, b);
11.         long long s1 = 0, s2 = 0;
12.         for (int i = a; i <= b; i++)
13.             if (i % 2)
14.                 s2 += i * i * i;
15.             else
16.                 s1 += i * i;
17.         cout << s1 << ' ' << s2 << endl;
18.     }
19.     return 0;
20. }
```

1.10 小鲁记账^{NQ010}

描述

小鲁的编程技术有不小的进步，舍友家里开一个小卖部，小卖部的客源不稳定，请小鲁设计一个程序每天帮忙记账，把盈利记为正数，亏损记为负数，收支相抵则记为0。现在他想要统计n天账单中亏损，盈利与收支相抵各自的天数。

输入

输入数据有多组，每组占一行，每行的第一个数是整数n (n<100)，表示需要统计的数值的个数，然后是n个实数；如果n=0，则表示输入结束，该行不做处理。

输出

对于每组输入数据，输出一行a,b和c，分别表示给定的数据中负数、零和正数的个数。

输入样例

```
7 0 2 -2 2 3 -1 0
4 4 3 4 0.5
0
```

输出样例

```
2 2 3
0 0 4
```

参考代码(C++)

```
1. //NQ010 小鲁记账
2. #include <stdio>
3. int main()
4. {
5.     int n, a, b, c;
6.     float d;
7.     while(scanf("%d", &n) && n!=0) //如果 n 不等于 0，持续输入
8.     {
9.         a = 0; b = 0; c = 0;
10.        for(int i = 1; i <= n; i++) //输入 n 个实数，判断这个数并且计数
11.        {
12.            scanf("%f", &d);
13.            if(d == 0) b++;
14.            else if(d < 0) a++;
15.            else c++;
16.        }
17.        printf("%d %d %d\n", a, b, c); //每输完一行，输出结果
18.    }
19.    return 0;
20. }
```

1.11 构造数列并求和^{NQ011}

描述

立志成为数学家的小华，又给自己构造了一个新的思维测试来锻炼自己的记忆力以及精准算术的能力。

这次思维挑战分为两个步骤：

1. 根据数字 n 构造实数数列 ($1 < n < 10000$)，方法如下： n 是数列第一项，第二项是 $\sqrt{n}$ ，第三项是 $\sqrt{\sqrt{n}}$ 以此类推。

2. 选择任意数字 m ($1 < m < 1000$)，计算数列前 m 项的和。

每次让人出两个数字 n, m ，小华能在10分钟之内算出答案。

小鲁看了很不服，作为未来文人的代表，他也想和小华一样，但是1分钟之内就算出答案。

你能写个程序帮帮他吗？

输入

若干行数据，每行是两个整数 n m 。

输出

输出以 n 为第一项的开方序列前 m 项的和，结果保留两位小数。

输入样例

```
65004 5750
87184 2112
94597 561
80010 2732
58744 7710
```

输出样例

```
71026.70
89610.44
95485.14
83043.60
66713.64
```

参考代码(C++)

```
1. //NQ011 构造数列并求和
2. #include <iostream>
3. #include <cmath>
4. using namespace std;
5. int main()
6. {
7.     int n, m;
8.     float res, a;
9.     while (cin >> n >> m) //读入 n,m 直到文件末尾
10.    {
11.        res = 0; //累加结果
12.        a = n; //初始化数列第1项
13.        for (int i = 1; i <= m; i++)
14.        {
15.            res += a;
16.            a = sqrt(a); //构造数列下一项
```

```

17.     }
18.     // 输出结果,保留两位小数
19.     printf("%.2f\n", res);
20. }
21. return 0;
22. }

```

1.12 水仙花数^{NQ012}

描述

小华桌面上摆放了一盆水仙花，伫立于一泓清水之上，亭亭玉立，竞相开放，散发出淡淡清香。

小华闻着水仙花的香气想起了小鲁，他喊小鲁过来说：有一种数叫做水仙花数。这种数有3位数，每个位上的数字的3次幂之和等于它本身。

例如：水仙花数153，有 $1^3 + 5^3 + 3^3 = 153$ 。假设正整数区间 $[m, n]$ ，你可以写个程序算出区间 $[m, n]$ 中所有的水仙花数吗？

输入

输入数据有多组，每组占一行，包括两个整数 m 和 n ($100 \leq m \leq n \leq 999$)。

输出

对于每个 $[m, n]$ 区间，请从小到大输出所有的水仙花数，用一个空格隔开。

如果给定的区间内不存在水仙花数，输出"no"。

输入样例

```

100 300
256 326

```

输出样例

```

153
no

```

参考代码(C++)

```

1. //NQ012 水仙花数
2. #include <stdio>
3. int main()
4. {
5.     int m, n;
6.     int count = 0;
7.     while(scanf("%d %d", &m, &n) != EOF)
8.     {
9.         count = 0; // 水仙花数计数清零

```

```

10.     for (int i = m; i <= n; i++)
11.     {
12.         // 计算立方和：从个位、十位到百位分别计算求和
13.         int a,b,c;
14.         a = i/100;
15.         b = i%100/10;
16.         c = i%100%10;
17.         int sum = a*a*a + b*b*b + c*c*c;
18.         // 输出结果
19.         if(sum == i)
20.         {
21.             if(count > 0) printf(" ");
22.             printf("%d", i);
23.             count ++;
24.         }
25.     }
26.     // 输出结果（没有水仙花数）和换行
27.     if(count == 0)
28.         printf("no");
29.     printf("\n");
30. }
31. return 0;
32. }

```

1.13 倒数数列求和^{NQ013}

描述

小华看着在骄傲和自卑中纠结的小鲁充满同情，因为人本来就不应该这样翻来覆去，颠倒反复。唯有谦卑才是最快的成才之路。

他喊小鲁过来，让他算一道“颠倒数”的题目：

有一个倒数数列，数列的第 i 项是 i 的正/负倒数。当 i 为奇数时，是 i 的正倒数，当 i 为偶数时，是 i 的负倒数，其中($i \geq 1$)

数列前几项为 $+1 \quad -1/2 \quad +1/3 \quad -1/4 \quad +1/5 \dots\dots$

请编程求该数列前 n 项之和。

输入

第一行输入正整数 q ，表示第二行有 q 个整数 ($1 < q < 100$)

第二行有 q 个整数，每个整数 n 表示求数列前 n 项之和。 ($1 < n < 10000$)

输出

输出n行，每行对应一个输入实例，结果保留2位小数。

输入样例

```
2
3 4
```

输出样例

```
0.83
0.58
```

参考代码(C)

```
1. //NQ013 倒数数列求和
2. #include <stdio.h>
3. int main()
4. {
5.     double res;
6.     int n, q;
7.     while (scanf("%d", &q) != EOF)
8.     {
9.         while (q--)
10.        {
11.            res = 0.;
12.            scanf("%d", &n);
13.            for (int i = 1; i <= n; i++)
14.                if (i % 2) res += 1.0 / i;
15.                else    res -= 1.0 / i;
16.            printf("%.2lf\n", res);
17.        }
18.    }
19.    return 0;
20. }
```

参考代码(C++)

```
1. ///NQ013 倒数数列求和
2. #include <iostream>
3. #include <iomanip>
4. using namespace std;
5. const int N = 10007; //N 最大为 10000
6. double a[N];
7. int main()
8. {
```



```

9.      //打表法
10.     a[1] = 1;
11.     for (int i = 2; i <= N; i++)
12.         if (i % 2) a[i] = a[i - 1] + 1.0 / i;
13.         else      a[i] = a[i - 1] - 1.0 / i;
14.     //查表
15.     int n, q;
16.     while (cin >> q){
17.         while (q--){
18.             cin >> n;
19.             cout << setiosflags(ios::fixed); //固定小数位数
20.             cout << setprecision(2) << a[n] << endl;
21.         }
22.     }
23.     return 0;
24. }

```

1.14 凤凰花开吃杨梅^{NQ014}

描述

每年厦大凤凰花开的时候，也是杨梅果实成熟的时节，酸酸甜甜的杨梅是小鲁最喜欢吃的水果之一，小华送小鲁一筐的杨梅，小鲁高兴坏了，但是光吃可不行，小华安排小鲁必须要解决一道与杨梅相关的题目喔！假设是这样的：小鲁第一天吃掉半筐的杨梅再多一个，第二天又将剩下的杨梅吃掉一半多一个，以后每天总是吃掉剩下的杨梅的一半多一个，到第 N 天时只剩下一个杨梅，请问这筐杨梅一共多少个呢？

输入

每行一个数，输入多行正整数 N ($1 < N < 30$)， N 表示剩下最后一个杨梅时是第 N 天。

输出

对应每行输入的正整数 N ，输出第一天开始吃的时候杨梅的总数 T ，每个 T 占一行。

输入样例

```

5
8
10

```

输出样例

```

46
382
1534

```

参考代码(C++)

```
1. //NQ014 凤凰花开吃杨梅
2. #include <iostream>
3. using namespace std;
4. int f(int n) //f 函数输入天数 n 返回最初的杨梅数 p
5. {
6.     int res = 1;
7.     //上一天的杨梅数为这一天的加 1 再乘以 2，循环 n-1 次
8.     for (int i = n - 1; i >= 1; i--)
9.         res = (res + 1) * 2;
10.    return res;
11. }
12. int main()
13. {
14.    int x;
15.    while (cin>>x)//输入
16.        cout<<f(x)<<endl;//输出
17.    return 0;
18. }
```

1.15 辩论赛评分^{NQ015}

描述

小鲁参加一年一度的信息学院辩论赛，这样的比赛对他简直是如鱼得水游刃有余，这不，经过3小时激烈的辩论，小鲁很轻松的驳倒众人，让众选手哑口无言。

这次比赛采取网络投票，每个在线观看辩论赛的ID都可以评分，评分规则为：为去掉一个最高分和一个最低分，然后计算平均得分。

比赛结束后，由于大众评委众多，成绩难以手工统计。热心的小鲁找到满头大汗的评委，笑着说，这事容易，我来写个程序搞定这事！

输入

输入数据有多组，每组占一行，每行的第一个数是 n ($2 < n \leq 100$)，表示评委的人数，然后是 n 个评委的打分。

输出

对于每组输入数据，输出选手的得分，结果保留2位小数，每组输出占一行。

输入样例

```
3 96 98 97
4 96 99 98 97
```

输出样例

```
97.00
97.50
```

参考代码(C++)

```
1. //NQ015 辩论赛评分
2. #include <iostream>
3. #include <iomanip>
4. using namespace std;
5. int main()
6. {
7.     int n;
8.     while (cin >> n)
9.     {
10.        int q = n;
11.        float min = 101, max = -1, s = 0, x; //令 min=101, max=-1, s=0
12.        while (q-->0)
13.        {
14.            cin >> x;
15.            if (x < min)
16.                min = x; //找出最小值
17.            if (x > max)
18.                max = x; //找出最大值
19.            s += x;      //分数和
20.        }
21.        cout << setiosflags(ios::fixed); //固定小数位数
22.        cout << setprecision(2) << (s - max - min) / (n - 2) << endl;
23.    }
24.    return 0;
25. }
```

1.16 十六进制加法^{NQ016}

描述

与喜欢争强好胜的小鲁不同，小华从来就不屑与人竞争。

他所求的不是赢，他所求的乃是突破。在数字领域，小华的创意总是层出不穷。

在小华构造的数字世界里，十进制只是 P 进制的一个具体实例，同样的十六进制也是 P 进制的具体实例。

在精通十进制加减法之后，小鲁开始得意洋洋不思进取，为了帮助小鲁进一步成长，小华给小鲁出了一道题：

把十进制加法改为十六进制加法，要求有两点：

1. 用大写字母表示十六进制
2. 如果遇到负数，即便是十六进制也要加上负号"-"。

面对手足无措的小鲁，你能再帮帮他吗？

输入

输入数据有若干行，每行有两个十六进制数。

数据范围：

$-10^9 < \text{输入数据} < 10^9$

输出

输出数据有若干行，每行对应输入数据的两数之和。

输入样例

```
+A -A
+1A -B
1ABCD -2020
-1FEDF777 -FFAEC8
```

输出样例

```
0
F
18BAD
-20EDA63F
```

参考代码(C++)

```
1. //NQ016 辩论赛评分
2. #include <iostream>
3. #include <iomanip>
4. using namespace std;
5. int main()
6. {
7.     long long a,b;
8.     cin>>setbase(16); //输入设置为 16 进制
9.     cout<<setiosflags(ios::uppercase)<<setbase(16); //输出设置为大写 16 进制
10.    while(cin>>a>>b) //十六进制
11.    {
12.        long long res=a+b; //计算加法
13.        if(res<0)
14.            cout<<"-"<<-(res)<<endl; //小于 0 时手动输出负号。
15.        else
16.            cout<<res<<endl; //正数直接输出
```

```

17.     }
18.     return 0;
19. }

```

1.17 计算绩点^{NQ017}

描述

编程学习需要投入大量的时间，与此同时理论学习的成绩又不能拉下，对于数学0基础的小鲁来说，这样的张力尤为明显。

早已驾驭这样的张力的的小华，递给小鲁一个锦囊，对小鲁说：知己知彼，百战不殆。

小鲁将信将疑的打开锦囊，里面写着六个字：“绩点制，须知晓”。

小鲁豁然开朗，下功夫研究了一把：“绩点制”：

原来课程学分绩点即Grade Point Average (缩写GPA)，是对学生各门课程所获学分的加权平均值。在“绩点制”的时代，90与100没有差别，80与81却截然不同，从某种意义上说，89比85更加让人扼腕叹息。

绩点的具体计算方法如下：

单门课程学分绩点 = 该课程的学分 × 成绩绩点

课程总绩点 = ∑单门课程学分绩点

课程总学分 = ∑单门课程学分

课程的平均绩点（GPA）= 课程总绩点 ÷ 课程总学分

百分制 ¹	等级制 ²	GPA ³
95-100 ⁴	A+ ⁵	4.0 ⁶
90-94 ⁴	A ⁵	4.0 ⁶
85-89 ⁴	A- ⁵	3.7 ⁶
81-84 ⁴	B+ ⁵	3.3 ⁶
78-80 ⁴	B ⁵	3.0 ⁶
75-77 ⁴	B- ⁵	2.7 ⁶
72-74 ⁴	C+ ⁵	2.3 ⁶
68-71 ⁴	C ⁵	2.0 ⁶
64-67 ⁴	C- ⁵	1.7 ⁶
60-63 ⁴	D ⁵	1.0 ⁶
60 以下 ⁴	F ⁵	0 ⁶

因此，怎样根据自己的恩赐，研究不同的科目，赚到最多的绩点，才是正道。省出来的时间，就可以投资在编程上了。小鲁很感激小华的点拨，决定写个程序，计算绩点，帮助自己制定计划。

输入

第一行输入课程总数N

第二行输入每门课程的成绩及其对应的学分（均为正数类型），每门课程输入一行

输出

最终的绩点成绩，小数点后保留4位

输入样例

```
5
90 4
78 2
80 2
58 4
85 3
```

输出样例

```
2.6067
```

参考代码(C++)

```
1. //NQ017 计算绩点
2. #include <iostream>
3. #include <iomanip>
4. using namespace std;
5. int main()
6. {
7.     float s, c, gpa, ctot, gpatot;
8.     int q;
9.     cin >> q;
10.    while (q-->0)
11.    {
12.        cin >> s >> c;
13.        if (s >= 90 && s <= 100)
14.            gpa = 4.0;
15.        else if (s >= 85 && s < 90)
16.            gpa = 3.7;
17.        else if (s >= 81 && s < 85)
18.            gpa = 3.3;
19.        else if (s >= 78 && s < 81)
20.            gpa = 3.0;
21.        else if (s >= 75 && s < 78)
22.            gpa = 2.7;
23.        else if (s >= 72 && s < 75)
24.            gpa = 2.3;
25.        else if (s >= 68 && s < 72)
26.            gpa = 2.0;
27.        else if (s >= 64 && s < 68)
```

```

28.         gpa = 1.7;
29.     else if (s >= 60 && s < 64)
30.         gpa = 1.0;
31.     else if (s < 60)
32.         gpa = 0.0;
33.     ctot += c;
34.     gpatot += gpa * c;
35. }
36. cout << setiosflags(ios::fixed);           //固定小数位数
37. cout << setprecision(4) << gpatot / ctot << endl; //输出
38. return 0;
39. }

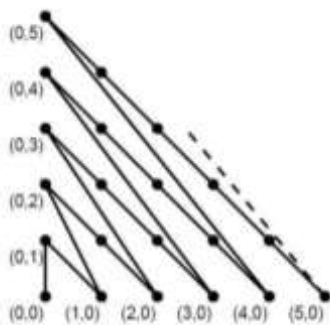
```

1.18 计算折线长度^{NQ018}

描述

文艺青年小鲁兴趣广泛，艺术也是他的兴趣之一，然而编程的输出总是数字让他很烦躁。他觉得如果输出的不够好看，就没啥意思。他突发奇想，要不就不学编程和算法，学习怎么使用电脑画图修图吧？他兴致勃勃地去找小华，想要放弃编程改学画画，因为编程实在无聊。小华知道好奇心有时候并非好事，但是还是迁就小鲁，给他一道与图形相关的模拟题，帮助小鲁回到他生命当下的重点——打好编程基础。

题目如下：给定如下的折线图，请计算任意两点之间折线长度。



输入

第一行输入一个正整数 n ($n \leq 100$)，代表有 n 组测试实例，接下来的 n 行，每行输入四个非负整数 x_1, y_1, x_2, y_2 ，分别表示折线起点和终点坐标， $x, y \leq 100$ 且均为非负整数。

输出

输出 n 行，每行对应输入数据中两坐标之间的折线长度，小数点后保留三位有效位。

输入样例

```

3
0 0 3 3

```

```
3 7 6 5
3 6 9 2
```

输出样例

```
51.511
33.251
63.675
```

参考代码(C++)

```
1. //NQ018 计算折线长度
2. #include <stdio.h>
3. #include <math.h>
4.
5. int main()
6. {
7.     int x1, y1, x2, y2, t, n;
8.     double r;
9.     scanf("%d", &n);
10.    while (n--)
11.    {
12.        scanf("%d%d%d%d", &x1, &y1, &x2, &y2);
13.        if (x1 + y1 > x2 + y2) {
14.            t = x1, x1 = x2, x2 = t, t = y1, y1 = y2, y2 = t;
15.        }
16.        else if (x1 + y1 > x2 + y2 && y2 > y1) {
17.            t = x1, x1 = x2, x2 = t, t = y1, y1 = y2, y2 = t;
18.        }
19.        r = 0;
20.        while (1) {
21.            if (x1 == x2 && y1 == y2)
22.                break;
23.            if (y1 == 0) {
24.                r += sqrt(x1 * x1 + (x1 + 1) * (x1 + 1));
25.                y1 = x1 + 1, x1 = 0;
26.            }
27.            else {
28.                do {
29.                    r += sqrt(2);
30.                    x1++, y1--;
31.                    if (y1 == 0)
32.                        break;
33.                } while (x1 != x2 && y1 != y2);
34.            }
35.        }
```



```
36.     printf("%.3lf\n", r);
37.     }
38.     return 0;
39. }
```

1.19 最大公约数^{NQ019}

描述

看着编程热情逐步高涨，并且可以开始运用各种语法的小鲁，小华意识到这是帮助他进一步进阶的时候了。

他喊小鲁过来，嘱咐他说：带着怎样的心学编程很重要，心对了，效果就好，心不对，可能会事倍功半。而编程最需要带着谦卑的心！这意味着我们不要觉得自己比他们聪明，乃要承认自己距离那些历史上的智者有巨大的差距，并且向前人学习。这样的心可以让人恒久成长。

看着云里雾里的小鲁，小华知道他需要实操才能明白这个道理，他给小鲁布置了道经典题：

欧几里得算法又称辗转相除法，是总所周知的算法，给定 n 对正整数 a, b ，请你求出每对数的最大公约数。但是有一个条件，请用一行代码完成求最大公约数的运算。

数据范围

$1 \leq n \leq 10^5$, $1 \leq a, b \leq 2 \times 10^9$

输入

第一行包含整数 n 。

接下来 n 行，每行包含一个整数对 a, b 。

输出

输出共 n 行，每行输出一个整数对的最大公约数。

输入样例

```
10
111 111
777 777
7777 777
11111 777
16583682 70871575
27870547 47025162
51590293 97432785
4825080 64853463
54282624 58567018
83355874 42258167
```

输出样例

```
111
777
7
1
1
1
1
3
2
7
```

参考代码(C++)

```
1. //NQ019 最大公约数
2. #include <iostream>
3. using namespace std;
4. // 利用性质: gcd(a , b) == gcd(b,a mod b)
5. int gcd(int a, int b){
6.     return b? gcd(b, a%b) :a;
7. }
8. int main()
9. {
10.     int n;
11.     cin>>n;
12.     while (n--){
13.         int a,b;
14.         cin>>a>>b;
15.         cout<<gcd(a,b)<<endl;
16.     }
17.     return 0;
18. }
```

1.20 时钟夹角 NQ020

描述

在图书馆里学习的时间真是难熬，小鲁每晚回宿舍都和妈妈视频，但妈妈交代要学习到点才行，万般无难，他只好盯着时钟，希望时间过得快一点。他突然灵光一闪，要是知道时间求出来现在表盘时针与分针的夹角，说不定可以加快时间的流逝。

注：夹角的范围 $[0, 180]$ ，时针和分针的转动是连续而不是离散的。

输入

输入数据的第一行是一个数据 T ，表示有 T 组数据。每组数据有三个整数 h ($0 \leq h < 24$)， m ($0 \leq m < 60$)， s ($0 \leq s < 60$) 分别表示时、分、秒。

输出

对于每组输入数据，输出夹角的大小的整数部分。

输入样例

```
2
18 13 17
21 25 30
```

输出样例

```
106
129
```

参考代码(C++)

```
1. //NQ020 时钟夹角
2. #include <cstdio>
3. #include <cmath>
4. int main()
5. {
6.     int t;
7.     int h0, m0, s0, x0;
8.     double h, m, s, n, x; //用 double 定义时, 分, 秒
9.     scanf("%d", &t);
10.    while (t--)
11.    {
12.        scanf("%d%d%d", &h0, &m0, &s0);
13.        h = h0, m = m0, s = s0;
14.        double c = s / 60.0; //求出秒针对分针的影响
15.        m += c;
16.        double d = m / 60.0; //分针对时针的影响
17.        if (h >= 12) h = h - 12; //每 12h 时针转一圈
18.        h += d, m = m * 6, h = h * 30; //60 分钟 360°
19.        if (h > m) n = h - m;
20.        else n = m - h;
21.        if (n > 180) x = 360 - n;
22.        else x = n;
23.        x0 = (int)(x); //转换整形
24.        printf("%d\n", x0);
25.    }
```

```
26.     return 0;
27. }
```

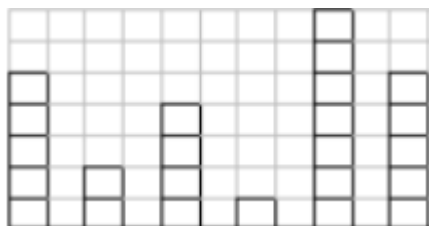
1.21 废墙重建^{NQ021}

描述

小栋自幼爱好工程，小小年纪就显出对土木工程，对机械工程，对建筑设计的浓厚兴趣。

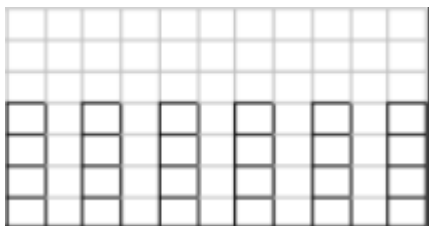
为了在百废待兴的山区兴建校区，小栋必须利用现有建材，重建校园内的墙。

每堵废墙，可以用如下示意图表示：



墙宽度：6

砖块数：5 2 4 1 7 5



墙宽度：6

砖块数：4 4 4 4 4 4

小华的目标是把原来的废墙修缮为宽度不变，高度相同的一堵新墙，如上图。

请估算废墙变新墙，每面墙至少要移动几块砖块。

输入

整个校区有若干面墙，因此输入数据有若干组。

每组数据有两行。

第一行表示墙宽度 x ($0 < x < 100$)。

第二行有 x 个数字，表示每列有多少块砖。

当输入数据中 x 为0表示结束。

输出

每组数据输出最少需要移动的砖块数。

不同组的数据之间由空行隔开。

输入样例

```
16
78 97 22 67 3 30 36 5 87 14 9 83 83 10 52 55
27
61 13 58 78 13 82 95 2 5 24 40 99 2 48 42 24 22 93 7 91 45 61 34 77 44 46 5
21
24 6 13 43 42 48 79 84 6 60 78 25 28 98 49 78 27 84 43 58 22
15
23 58 21 50 3 12 40 43 41 43 53 29 35 24 23
23
21 99 76 76 4 14 17 65 57 50 5 30 60 97 18 75 3 53 51 31 1 29 57
0
```

输出样例

```
231
339
238
96
300
```

参考代码(C++)

```
1. //NQ021 废强重建
2. #include <iostream>
3. using namespace std;
4. const int N=107;//墙最大宽度<100
5. int main()
6. {
7.     int n, a[N];
8.     while (cin >> n, n > 0)
9.     {
10.         int avg = 0; //平均值
11.         for (int i = 0; i < n; i++)
12.         {
13.             cin >> a[i];
14.             avg += a[i];
15.         }
16.         avg = avg / n; //求平均值
17.         int res = 0; //统计小于平均值的列中砖块的空缺数
18.         for (int i = 0; i < n; i++)
```

```

19.     {
20.         if (a[i] < avg)
21.             res += avg - a[i];
22.     }
23.     cout << res << endl << endl; //数据之间有空行
24. }
25. return 0;
26. }

```

1.22 竞选投票^{NQ022}

描述

小鲁准备竞选计算机系新一届的学生会主席，今年的竞选规则相比于之前有所改变，将通过各班的投票结果来确定。

如果得到超过一半班级的支持就可以当选，而每个班的投票结果又是由该班级的所有同学投票产生，如果超过一半的同学支持小鲁，则他将赢得这个班级的支持。

现在给出每个班级的学生人数，小鲁想算算，他至少需要得到多少票才能当选。

输入

多组输入数据。

每组数据的第一行包括一个整数 N ($1 \leq N \leq 101$)，表示计算机系的班级数，接下来一行包括 N 个正整数，分别表示每个班的学生人数，用空格隔开。

输出

输出一个数据，表示至少需要的票数。

输入样例

```

5
20 18 17 16 19

```

输出样例

```

28

```

参考代码(C)

```

1. // NQ022 竞选投票
2. #include <stdio.h>
3. #include <stdlib.h>
4. //给 qsort 预备的比较函数
5. int cmp(const void *a, const void *b) { return *(int *)a - *(int *)b; }
6. int main() {
7.     int n;

```

```

8.  scanf("%d", &n);
9.  int A[n + 1];
10. if (n > 0) {
11.     int ans = 0;
12.     for (int i = 0; i < n; i++) {
13.         scanf("%d", &A[i]);
14.         A[i] = A[i] / 2 + 1;
15.     }
16.     //调用 qsort 排序
17.     qsort(A, n, sizeof(A[0]), cmp);
18.     n = n / 2 + 1;
19.     for (int i = 0; i < n; i++) {
20.         ans += A[i];
21.     }
22.     printf("%d", ans);
23. }
24. return 0;
25. }

```

参考代码(C++)

```

1. //NQ022 竞选投票
2. #include <algorithm>
3. #include <iostream>
4. using namespace std;
5. int main() {
6.     int n;
7.     cin >> n;
8.     int x;
9.     vector<int> students;
10.    for (int i = 0; i < n; i++) {
11.        cin >> x;
12.        students.push_back(x / 2 + 1); //每个班级只需要过半的人
13.    }
14.    sort(students.begin(), students.end()); //排序
15.    n = n / 2 + 1; //取前半班级
16.    int res = 0;
17.    for (int i = 0; i < n; i++) res += students[i];
18.    //输出结果
19.    cout << res << endl;
20.    return 0;
21. }

```

1.23 桌球比赛NQ023

描述

小华和小鲁决定进行一场台球友谊赛，台面上共有红、黄两色球各7个，1个黑球，1个白球。小华选择打红球，小鲁选择打黄球。

比赛规则是：红、黄两名选手轮流用白球击打各自颜色的球，如果将该颜色的7个球全部打进，则这名选手可以打黑球，如果打进则算他赢。如果在打进自己颜色的所有球之前就把黑球打进，则算输。如果选手不慎打进了对手的球，入球依然有效。现在，请你根据进球（白球）的颜色顺序，以及黑球是由谁打进的，判断谁是赢家。

提示：

“Y”表示黄球，“R”表示红球。“B”表示红方打进黑球，“L”表示黄方打进黑球。

假如输入数据为：YYYYRRB。

则判断胜负的思路是：1. R和Y的个数 2. 再根据输入的最后一个字母是B还是L分情况判定。当为B时，根据R的值是否是否等于7判断输赢；当为L时，根据Y的值是否等于7判断输赢。

输入

每一行先输入整数 m ($1 \leq m \leq 15$)，表示进球个数， $m=0$ 表示本次比赛结束。

随后输入 m 个字母，依次表示打进球的颜色。

“Y”表示黄球，“R”表示红球。

“B”表示红方打进黑球，“L”表示黄方打进黑球。

字符之间没有空格。

输入的数据保证有效（所有球打进时比赛正好结束，打进的同颜色球不得超过7个）

输出

输出 n 行，每行与输入的对应。如果小华获胜，输出‘Red’；如果小鲁胜，输出‘Yellow’。

输入样例

```
5
RRYRB
7
YYYYRRB
```

输出样例

```
Yellow
Yellow
```

参考代码(C++)

```
1. //NQ023 桌球比赛
2. #include <iostream>
```



```

3. #include <string>
4. using namespace std;
5. int main() {
6.     int m;
7.     while (cin >> m) {
8.         int cntR = 0, cntY = 0;
9.         char c; //读入 m 个字符
10.        while (m--) {
11.            cin >> c;
12.            if (c == 'R') cntR++;
13.            if (c == 'Y') cntY++;
14.            if (c == 'B') {
15.                if (cntR == 7)
16.                    cout << "Red" << endl;
17.                else
18.                    cout << "Yellow" << endl;
19.            }
20.            if (c == 'L') {
21.                if (cntY == 7)
22.                    cout << "Yellow" << endl;
23.                else
24.                    cout << "Red" << endl;
25.            }
26.        }
27.    }
28.    return 0;
29. }

```

1.24 珠穆朗玛峰测距^{NQ024}

描述

近期国家测量登山队开始实施珠穆朗玛峰实地测距项目。登山的过程十分危险，恶劣的天气，氧气稀薄，加上可能雪崩的威胁，整个登山团队必须步步为营，处处小心。通常到达一定的高度后，就必须就地扎营，同时要计算所在营地的海拔高度来知晓氧气的稀薄程度以及出现恶劣气候的可能。

小华给小鲁出个问题，帮忙计算出相对海拔高度，定义两点的坐标为 $Z(x, y)$ ， $W(m, n)$ ，求 $f(x, y, m, n) = \sqrt{(xx + yy + mm + nn - 2mx - 2ny)}$ 。

输入

第一行输入 n ，表示有 n 行测试数据。

接着每行都输入四个实数 x, y, m, n ，每个实数均小于等于 100。

输出

输出相对海拔高度，小数点后保留一位。

输入样例

```
3
3 5 8 0
6 2 20 8
4 10 36 7
```

输出样例

```
7.1
15.2
32.1
```

参考代码(C++)

```
1. //NQ024 珠穆朗玛峰测距
2. #include <cmath>
3. #include <cstdio>
4. int main() {
5.     int c;
6.     double x, y, m, n;
7.     scanf("%d", &c); // 输入有 c 行测试数据
8.     while (c--) {
9.         scanf("%lf %lf %lf %lf", &x, &y, &m, &n);
10.        printf("%.1lf\n",
11.            sqrt(x * x + y * y + m * m + n * n - 2 * m * x - 2 * n * y));
12.    }
13.    return 0;
14. }
```

1.25 胜利会师^{NQ025}

描述

为了运送修建校区的物资，小嘉多方游说，成功募款到资金修筑一条从山区通向城镇的公路。工程队分别从公路两端相向修路。在小栋的率领下，今天甲工程队和乙工程队就要胜利会师了，已知甲工程队推进速度 U 米/秒，乙工程队推进速度 V 米/秒，两工程队现在相距 L 米。

小鲁受邀报道两只工程队会师情况。假设小鲁以 w 米/秒的速度马不停蹄地往返于两工程队之间，作工程快报。当他抵达一方完成采访后，就要立刻奔向另一方，这样来回往返做报道（在山区没有无线网络，无法连线，人必须往来奔走才能作工程快报）。

假设小鲁的采访时间快到可以忽略不计，报道稿全在路上写成。

请问当两只工程队会师之时，小鲁走的总路程是多少？

输入

第一行输入 t ，表示有 t 组数据。

随后 t 行数据，每行是四个实数 u, v, w, l ，分别表示甲工程队速度，乙工程队速度，小鲁的速度，以及起始的距离。

输出

输出一个实数表示小鲁走的总的路程（精确到小数点后3位）。

输入样例

```
1
15 16 30 500
```

输出样例

```
483.871
```

参考代码(C++)

```
1. // NQ025 胜利会师
2. #include <iomanip>
3. #include <iostream>
4. using namespace std;
5. int main() {
6.     int t;
7.     cin >> t;
8.     while (t--) {
9.         float u, v, w, l;
10.        cin >> u >> v >> w >> l;           //读入 4 个浮点数
11.        cout << setiosflags(ios::fixed);     //固定小数位数
12.        cout << setprecision(3) << w * l / (u + v) << endl; //输出三位小数
13.    }
14.    return 0;
15. }
```

1.26 公式变代码^{NQ026}

描述

经过一段时间的学习，小鲁终于领悟了一个道理，与其跨行竞争，而不跨行合作。

这回他不再急忙想着在数学的专业度上超越小华，乃是虚心请教，题目如下：

$$\arctan(x) = \sum_{n=0}^{\infty} \frac{(-1)^n x^{2n+1}}{2n+1}$$

已知 $\arctan$ 的定义是

变量 s, u, v 满足: $\arctan(1/s) = \arctan(1/u) + \arctan(1/v)$

函数 $f(s, u, v) = v*u - s*u - s*v$

若 s, u 已知, 求函数 $f(s, u, v)$ 的值。

小华笑着对小鲁说, 数学公式的具体含义太复杂, 你不需要理解, 你只要调用系统的库函数计算 $\arctan$ 然后套公式即可。至于如何构造公式, 公式的涵义是什么, 这是数学家的领域, 对于当下的你, 把答案算出来即可。

小鲁豁然开朗, 如释重负, 原来可以不求甚解, 于是马上开写代码!

输入

第一行是一个整数 t , 表示接下来有 t 行数据。

接下来 t 行数据, 每行是两个数, 表示 s, u 。

数据范围:

$s \leq 12^3; u \leq 2^{20}$

$s, u, v > 0$ 且 s, u, v 均为实数

输出

输出 t 行整数, 每行表示函数 $f(s, u, v)$ 的值。(如果函数值为小数, 则直接取整)

输入样例

```
1
1 2
```

输出样例

```
1
```

参考代码(C++)

```
1. //NQ026 公式变代码
2. #include <cmath>
3. #include <iostream>
4. using namespace std;
5. int main() {
6.     int t; // t 组数据
7.     cin >> t;
8.     while (t--) {
9.         double s, u;
10.        scanf("%lf%lf", &s, &u);
11.        double v = 1.0 / tan(atan(1.0 / s) - atan(1.0 / u));
12.        double res = v * u - s * u - v * s;
```

```
13.     printf("%.01f\n", res);
14. }
15. return 0;
16. }
```

1.27 丑数^{NQ027}

描述

看着小鲁逐步掌握了基本的编程语法，小华知道现在是帮助小鲁重建数学基础的时候了，他给小鲁出了一道判断题：

编写一个程序判断给定的数是否为丑数。

丑数就是只包含质因数2, 3, 5的正整数。

6是丑数，因为 $6 = 2 \times 3$ 。

8是丑数，因为 $8 = 2 \times 2 \times 2$ 。

14不是丑数，因为它包含了另外一个质因数 7。

补充说明

1. 1是丑数。

2. 输入不会超过 32 位有符号整数的范围： $[-2^{31}, 2^{31}-1]$ 。

输入

输入数据包含若干行，每行是一个数 n ($-2^{31} \leq n \leq 2^{31}$)。

输出

如果 n 是丑数输出 `true`，否则输出 `false`。

每个数据的输出结果占一行。

输入样例

```
6
8
14
```

输出样例

```
true
true
false
```

参考代码(C++)

```
1. //NQ027 丑数
2. #include <iostream>
3. using namespace std;
```

```

4.
5. bool isUgly(int num) {
6.     if (num <= 0) return false;
7.     //去掉因子 2
8.     while (num % 2 == 0) num /= 2;
9.     //去掉因子 3
10.    while (num % 3 == 0) num /= 3;
11.    //去掉因子 5
12.    while (num % 5 == 0) num /= 5;
13.    //如果余数为 1, 则为丑数
14.    return num == 1;
15. }
16. int main() {
17.     int n;
18.     while (cin >> n)
19.         if (isUgly(n))
20.             cout << "true" << endl;
21.         else
22.             cout << "false" << endl;
23.     return 0;
24. }

```

1.28 统计0-1矩阵中1的数量 NQ028

描述

文艺小青年小鲁希望挑战更难编程题目，小华告诫他：心急吃不了热豆腐，要脚踏实地，一步步来。于是，又给了一道基础题，判断0，1矩阵中1的个数。小鲁暗下决心，一定以闪电般的速度做出来，让小华刮目相看。

输入

第一行输入 n ，代表有 n 个测试样例。

第二行开始输入 n 个矩阵，每个矩阵第一行输入 a, b 表示矩阵大小为 $a \times b$ ，接着输入 a 行，每行包含 b 个元素。

输出

输出 n 行，每行对应输入矩阵实例中1的个数。

输入样例

```

2
2 2
1 0

```

```
0 1
3 3
1 0 1
0 1 1
1 0 0
```

输出样例

```
2
5
```

参考代码(C++)

```
1. // NQ028 统计 0-1 矩阵中 1 的数量
2. #include <iostream>
3. using namespace std;
4.
5. int main() {
6.     int t;
7.     cin >> t;
8.     while (t--) {
9.         int a, b, x, cnt = 0;
10.        cin >> a >> b;
11.        for (int i = 1; i <= a; i++)
12.            for (int j = 1; j <= b; j++) {
13.                cin >> x;
14.                if (x == 1) cnt++;
15.            }
16.        cout << cnt << endl;
17.    };
18. }
19. return 0;
20. }
```

1.29 回文数^{NQ029}

描述

小华叫小鲁过来，继续帮助小鲁重建数学基础，他再给小鲁出了一道判断题：

编写一个程序判断给定的数是否为回文数。

回文数是指正序（从左向右）和倒序（从右向左）读都是一样的整数。

例如：121 是回文数，-121不是回文数，因为从左向右读，为 -121 。 从右向左读，为 121- 。

进阶：* 你能不将整数转为字符串来解决这个问题吗？

输入

输入数据包含若干行，每行是一个数 n ($-2^{31} \leq n \leq 2^{31}$)。

输出

如果 n 是回文数输出true，否则输出false。

每个数据的输出结果占一行。

输入样例

```
121
1221
1234321
1234412341
-121
-11
```

输出样例

```
true
true
true
false
false
false
```

参考代码(C++)

```
1. //NQ029 回文数
2. #include <iostream>
3. using namespace std;
4. bool isPalindrome(int x) {
5.     if (x < 0) return false;    //负数不是回文数
6.     long long res = 0, oldx = x; //把旧的x存下来
7.     while (x) {
8.         res = res * 10 + x % 10;
9.         x /= 10;
10.    }
11.    return res == oldx; //判断逆转后的两个数是否相同
12. }
13. int main() {
14.     int n;
15.     while (cin >> n)
16.         if (isPalindrome(n))
17.             cout << "true" << endl;
18.         else
```



```

19.         cout << "false" << endl;
20.     return 0;
21. }

```

1.30 小球进盒子^{NQ030}

描述

小华最近在研究小球进盒子问题，有R个红色盒子和B个黑色盒子，还有R个红色小球和B个黑色小球。每个盒子只能装一个小球，每个小球都要放在一个盒子里。如果把一个红色小球放在一个红色盒子里，那么得分是C。如果把一个黑色小球放在一个黑色盒子里，那么得分是D。如果把一个红色小球放在一个黑色盒子里，那么得分是E。如果把一个黑色小球放在一个红色盒子里，那么得分也是E。现在给出R, B, C, D, E。应该如何放置这些小球进盒子，才能使得总得分最大？输出最大的总得分。

输入

一行，5个整数，分别是R, B, C, D, E。

数据范围： $1 \leq R \leq 100$, $1 \leq B \leq 100$, $-1000 \leq C, D, E \leq 1000$

输出

一个整数，最大总得分。

输入样例

```
4 3 200 300 250
```

输出样例

```
1700
```

参考代码(C++)

```

1. // NQ030 小球进盒子
2. #include <iostream>
3. using namespace std;
4. int R, B, r, b, A;
5. int main() {
6.     cin >> R >> B >> r >> b >> A;
7.     if (r + b > 2 * A) {
8.         // r+b>2*A 情况，红黑球都放自己盒子
9.         cout << R * r + B * b;
10.        return 0;
11.    } else // r+b<2*A 情况
12.    {
13.        if (R > B) {

```

```

14.      //红比黑多，红黑球交换盒子后，剩余红球放红盒
15.      cout << B * A + B * A + (R - B) * r;
16.      return 0;
17.  }
18.  if (B > R) {
19.      //黑比红多，红黑球交换盒子后，剩余黑球放黑盒
20.      cout << R * A + R * A + (B - R) * b;
21.      return 0;
22.  }
23.  if (B == R) {
24.      // 红和黑相等，红黑球交换盒子
25.      cout << R * 2 * A;
26.      return 0;
27.  }
28.  }
29.  return 0;
30. }

```

1.31 解密文本初阶^{NQ031}

描述

在兴建校园的过程中，为了防止信息被窃取，小栋发明了一种简单的加密方法：

对于纯文本消息中的每个字母，将其向右移动五个位置以创建密文消息（即，如果字母为“ A”，则密文为“ F”）。

加密过程：任何非字母字符都应保持不变，并且所有字母字符均应为大写。

例如，如果密文消息是

```
ABCDEF GHI JKLMNOPQRSTU VWXYZ
```

则解密后的明文是

```
VWXYZ ABCDEF GHI JKLMNOP QRSTU
```

这个加密方法显然很容易被破解，但是对于小栋建校工程各部门工程的通讯协同，已经够用了。你能写一个解密程序，把密文翻译成明文吗？

输入

输入将包含最多100个数据集。

每个数据集包含3个组成部分：

起始行固定不变为“ START”

一行加密后的密文消息

结束行固定不变为“ END

读到数据集结束行代码“ ENDOFINPUT”就退出程序

输出

对于每个数据集，输出一行：解密后的消息。

输入样例

```
START
YMJ UWTAJWGK TK XTQTRTS YMJ XTS TK IFANI, PNSL TK NXWFJQ;YT PSTB BNXITR FSI
NSXYWZHYNTS;
END
START
IFSLJW PSTBX KZQQ BJQQ YMFY HFJXFW NX RTWJ IFSLJWTZX YMFS MJ
END
START
YMJWJKTWJ XMFQQ YMJD JFY TK YMJ KWZNY TK YMJNW TBS BFD, FSI GJ KNQQJI BNMYM YMJNW TBS
IJANHJX.
END
ENDOFINPUT
```

输出样例

```
THE PROVERBS OF SOLOMON THE SON OF DAVID, KING OF ISRAEL;TO KNOW WISDOM AND
INSTRUCTION;
DANGER KNOWS FULL WELL THAT CAESAR IS MORE DANGEROUS THAN HE
THEREFORE SHALL THEY EAT OF THE FRUIT OF THEIR OWN WAY, AND BE FILLED WITH THEIR OWN
DEVICES.
```

参考代码(C++)

```
1. //NQ031 解密文本初阶
2. #include <ctype.h>
3. #include <stdio.h>
4. #include <string.h>
5. char ans[20] = {"ENDOFINPUT"};
6. int main() {
7.     char string[500];
8.     while (gets(string), strcmp(string, ans)) {
9.         gets(string);
10.        int len = strlen(string);
11.        for (int i = 0; i < len; i++)
12.            if (isalpha(string[i])) ///如果是字母
13.                string[i] = (string[i] - 'A' + 5) % 26 + 'A';
14.        printf("START\n");
15.        printf("%s\n", string);
16.        printf("END\n");
17.        gets(string);
18.    }
```

```
18.  }
19.  printf("ENDOFINPUT\n");
20. }
```

1.32 首字母大写^{NQ032}

描述

小鲁这天接到了导师的一个任务，请他整理一份英文文献列表，要求文献标题都要按照英文文章标题的大小写规则，保证每个单词的第一个字母改成大写。看着洋洋洒洒的近千篇文章标题，有的首字母已经是大写，有的所有单词都是大写，小鲁犯难了，这个任务虽然简单，但是手动更改起来也得费不少功夫，于是，他灵机一动，我何不编个程序自动更改呢！要求输入若干个英文句子，将每个单词的第一个字母改成大写字母，其余改成小写字母。

输入

输入数据包含多个测试实例，每个测试实例是一个长度不超过1000的英文句子，占一行。

输出

请输出按照要求改写后的英文句子。

输入样例

```
JoiNt imbalanced classification and Feature selection for hospital readmissions
Rational construction and microwave absorption properties of porous FeOX/Fe/C
composites
```

输出样例

```
Joint Imbalanced Classification And Feature Selection For Hospital Readmissions
Rational Construction And Microwave Absorption Properties Of Porous Feox/fe/c
Composites
```

参考代码(C++)

```
1.  // NQ032 首字母大写
2.  #include <iostream>
3.  using namespace std;
4.  int main() {
5.      string line;
6.      int i, len;
7.      char ch;
8.      while (getline(cin, line)) {
9.          len = line.size();
10.         //读入行首字母如果是小写字母就变为大写字母
11.         if ((line[0] >= 'a') && (line[0] <= 'z')) line[0] -= 32;
```

```

12.
13.     //双指针算法: line[i]与 line[i+1]两个相邻指针
14.     for (i = 0; i < len; i++) {
15.         if (line[i] == ' ') {
16.             if ((line[i + 1] >= 'a') && (line[i + 1] <= 'z')) line[i + 1] -= 32;
17.         } else {
18.             if (line[i + 1] >= 'A' && line[i + 1] <= 'Z') line[i + 1] += 32;
19.         }
20.     }
21.     cout << line << endl;
22. }
23. return 0;
24. }

```

1.33 手机短号^{NQ033}

描述

开学季，校园网号码大促销，如果申请加入校园网，就能拥有一个手机短号，用短号打电话可以便宜非常非常多。11位长的手机号，变为短号的规则是 6+手机号的后5位。

例如号码为13512345678的手机，对应的短号就是645678。

小华喊小鲁过来说，学以致用的时候了，请写个程序批量把新生通讯录中的所有手机号码都变为短号。

输入

第一行输入n (n ≤ 200) 代表要更新为短号的电话号码个数。

接下来输入n行，每行是一个11位的手机号码。

输出

输出n行，每行对应输入的短号。

输入样例

```

2
13159269056
13159234089

```

输出样例

```

669056
634089

```

参考代码(C++)

```
1. // NQ033 手机短号
2. #include <cstring>
3. #include <iostream>
4. using namespace std;
5.
6. int main() {
7.     int n;
8.     string s;
9.     cin >> n;
10.    while (n--) {
11.        cin >> s;
12.        int len = s.length();
13.        //取字符串倒数 5 个字符
14.        cout << "6" + s.substr(len - 5) << endl;
15.    }
16.    return 0;
17. }
```

1.34 求小数点后的数^{NQ034}

描述

在小华的栽培下，小鲁逐渐成为年段零基础同学中的冉冉兴起的编程新星。隔壁宿舍的小傲同学很不服，想和小鲁比划比划，输的人请喝咖啡。比赛的题目是：给定小数 x ，和位数 n ，请快速取出小数点后第 n 位的值。

输入

首先输入整数 n ，表示有 n 组数据。

每行输入一个小数和一个整数 t ($1 \leq t \leq 6$)， t 表示小数点后第 t 位。

输出

输出小数点后第 t 位的数。

输入样例

```
3
0.2369 3
3.14156 4
88.697214 2
```

输出样例

```
6
46
```

```
5
9
```

参考代码(C++)

```
1. // NQ034 求小数点后的数
2. #include <iostream>
3. using namespace std;
4. int main() {
5.     int n;
6.     cin >> n;
7.     while (n--) {
8.         double a;
9.         int t;
10.        cin >> a >> t;
11.        //小数点后几位就乘以几次 10
12.        while (t--) a *= 10.0;
13.        //得到的数字化整后模 10
14.        cout << int(a) % 10 << endl;
15.    }
16.    return 0;
17. }
```

1.35 字符串奇偶位互换^{NQ035}

描述

小鲁觉得字符串的处理很有趣，刚接触到字符串他文人的天性就发作了，想着怎样折腾折腾。于是，他决定给自己布置一道简单的折腾题连连手：对一个长度为偶数位的0,1字符串，实现这个字符串的奇偶位互换。

输入

输入包含多组测试数据；输入的第一行是一个整数C，表示有C测试数据；接下来是C组测试数据，每组数据输入均为0,1字符串，保证串长为偶数位（串长≤50）。

输出

请为每组测试数据输出奇偶位互换后的结果；每组输出占一行。

输入样例

```
3
1110
1001
011110
```

输出样例

```
1101
0110
101101
```

参考代码(C++)

```
1. // NQ035 字符串奇偶位互换
2. #include <cstring>
3. #include <iostream>
4. using namespace std;
5. int main() {
6.     int n;
7.     cin >> n;
8.     while (n--) {
9.         string s;
10.        cin >> s;
11.        for (int i = 0; i < s.length() - 1; i += 2)
12.            swap(s[i], s[i + 1]); //交换 s[i] 与 s[i+1]
13.        cout << s << endl;
14.    }
15.    return 0;
16. }
```

1.36 合并字符串初阶^{NQ036}

描述

操作文字乃是小鲁的热情，告别令他痛不欲生的数学之后，小鲁在编程大陆里找到了他最跃跃一试的领域，字符串。字符串操作难度跨度很大，不过因为热爱文字，小鲁立定心志，重建根基，迎难而上，步步为营，稳扎稳打。这不，他遇到了最基本的字符串操作：合并字符串。

给定两串字符串，第一串的字符个数为偶数。请合并两串字符串，要求把第二串插入第一串正中间。

输入

首行为一个整数 t ，表示测试数据的组数。

然后是 t 组数据，每组数据是2行字符串 ($0 < \text{字符串长度} < 50$)

第一串的字符串长度必为偶数。

输出

输出为 t 行字符串，每行为合并后的字符串。

输入样例

```
2
I LOVE THIS WORLD SO MUCH
BEAUTIFUL
IN THE BEGINNING
HE
```

输出样例

```
I LOVE THIS BEAUTIFULWORLD SO MUCH
IN THE BHEEGINNING
```

参考代码(C++)

```
1. // NQ036 合并字符串初阶
2. #include <cstring>
3. #include <iostream>
4. using namespace std;
5. int main() {
6.     int t;
7.     cin >> t;    //读入t组数据
8.     getchar();  //去掉换行符
9.     while (t--) {
10.         string a, b;
11.         getline(cin, a);    //读入第一行
12.         getline(cin, b);    //读入第二行
13.         int p = a.size() / 2; //取中点
14.         //合并字符串
15.         cout << a.substr(0, p) + b + a.substr(p) << endl;
16.     }
17.     return 0;
18. }
```

1.37 数列的绝对值排序

描述

掌握基本排序算法以及函数调用之后，小鲁决定试试自定义规则的排序算法怎么写：

输入 n ($n \leq 100$) 个整数，将它们按照绝对值从大到小的顺序后输出。

输入

第一行输入正整数 n ，表示有 n 组测试实例， $n \leq 100$ 。

每组测试实例分两行输入。

第一行输入 m ，表示该实例包含 m 个整数 ($m \leq 100$)，第二行输入 m 个整数 x ($-100 \leq x \leq 100$)。

输出

输出n行，每行对应一个输入测试实例排序结果，整数之间用空格分开。

输入样例 1

```
2
3
6 -1 8
4
0 9 2 -8
```

输出样例 1

```
8 6 -1
9 -8 2 0
```

参考代码(C++)

```
1. //NQ037 数列的绝对值排序
2. #include <algorithm>
3. #include <cmath>
4. #include <iostream>
5. using namespace std;
6. //定义比较函数
7. bool less_abs(int a, int b) { return abs(a) > abs(b); }
8. int main() {
9.     int n;
10.    cin >> n;
11.    while (n-->0) {
12.        int m;
13.        cin >> m;
14.        vector<int> nums;
15.        for (int i = 0; i < m; i++) {
16.            int x;
17.            cin >> x;
18.            nums.push_back(x);
19.        }
20.        //排序
21.        sort(nums.begin(), nums.end(), less_abs);
22.        //输出
23.        for (int i = 0; i < m - 1; i++) cout << nums[i] << " ";
24.        cout << nums[m - 1] << endl;
25.    }
26.    return 0;
27. }
```

1.38 数字字符的统计^{NQ038}

描述

自从小鲁迷上编程，拜小华为师后，小华每次给的题目都是四则运算题。

小鲁决定不问小华，自己编个字符串处理的程序，左思右想，觉得自己得先从简单的统计开始，他给自己定了一个任务：

读入一个字符串，统计该字符串中数字字符出现的次数。

输入

第一行输入 n ，代表有 n 个测试实例。

接着输入 n 行，每行是一个包含字母和数字的字符串。

输出

输出 n 行，每行对应输入行中数字字符的个数。

输入样例

```
2
1312dfhfh
reythy78
```

输出样例

```
4
2
```

参考代码(C++)

```
1. // NQ038 数字字符统计
2. #include <cstring>
3. #include <iostream>
4. using namespace std;
5. int main() {
6.     int n;
7.     cin >> n;
8.
9.     while (n--) {
10.         string s;
11.         cin >> s;
12.         //循环统计数字个数
13.         int cnt = 0;
14.         for (auto c : s)
```

```

15.     if (isdigit(c)) cnt++;
16.     //输出字符个数
17.     cout << cnt << endl;
18. }
19. return 0;
20. }

```

1.39 合法标识符^{NQ039}

描述

编程课上Andy老师特别提到写代码要养成良好的命名习惯，标识符的命名要合法且有意义，代码才能一目了然。下课后，小鲁和小栋和一起去食堂的路上还在继续讨论有关字符串标识符的问题。小栋笑嘻嘻的逗小鲁说，“一说到“串”，我马上想到烤肉串，都流口水了，字符串不一定是合法，但是烤肉串肯定合法。晚上出去吃烤肉，怎样？”。小鲁当即同意，但是谁买单呢？

小华建议：“我们来写个程序自动判断字符串是否是合法的标识符，看谁先写出来，后写出来的请客”。

文字领域是小鲁的必争之地，然而小华实力摆在那里，你猜最后谁能赢呢？

注意：合法标识符由字母（A-Z, a-z）、数字（0-9）、下划线“_”组成，并且首字符不能是数字，但可以是字母或者下划线。

输入

第一行输入整数n，表示要输入n行字符串。

接着每行输入一个字符串，长度不超过50。

输出

对于每个输入的字符串，输出“yes”，表示是合法的标识符，否则“no”，表示非法的标识符。

输入样例

```

3
_tea345
39wei_
hap567

```

输出样例

```

yes
no
yes

```

参考代码(C++)

```

1. // NQ039 合法标识符

```

```

2. #include <cstring>
3. #include <iostream>
4. using namespace std;
5. int main() {
6.     int n;
7.     cin >> n;
8.
9.     while (n--) {
10.        string s;
11.        cin >> s;
12.        //判定首字符是否为数字
13.        if (isdigit(s[0]))
14.            cout << "no" << endl;
15.        else {
16.            //循环判定每个字符是否合法
17.            bool valid = true;
18.            for (auto c : s)
19.                if (!(isdigit(c) || (isalpha(c)) || c == '_')) {
20.                    valid = false;
21.                    break;
22.                }
23.
24.            if (valid)
25.                cout << "yes" << endl;
26.            else
27.                cout << "no" << endl;
28.        }
29.    }
30.    return 0;
31. }

```

1.40 每行诗最大元素^{NQ040}

描述

文艺青年小鲁写过不少英文诗，突然有个奇想，通过编程对每行诗字符串，查找其中的最大字母，在该字母后面插入字符串“(max)”。

输入

输入数据包括多个测试实例，每个实例由一行长度不超过100的字符串组成，字符串仅由大小写字母构成。

输出

对于每个测试实例输出一行字符串，输出的结果是插入字符串“(max)”后的结果，如果存在多个最大的字母，就在每一个最大字母后面都插入“(max)”。

输入样例

```
avwbgggjssd
ccccc
```

输出样例

```
avw(max)bgggjssd
c(max)c(max)c(max)c(max)c(max)c(max)
```

参考代码(C++)

```
1. // NQ040 每行诗最大元素
2. #include <algorithm>
3. #include <cstring>
4. #include <iostream>
5. using namespace std;
6. int main() {
7.     string s;
8.     while (cin >> s) {
9.         //找到最大字符
10.        string temp = s;
11.        sort(temp.begin(), temp.end()); //排序
12.        char maxchar = temp[temp.length() - 1]; //最后一个字符是最大字符
13.        //循环打印结果
14.        for (auto c : s) {
15.            cout << c;
16.            if (c == maxchar) cout << "(max)";
17.        }
18.        cout << endl;
19.    }
20.    return 0;
21. }
```

1.41 统计字符个数初步^{NQ041}

描述

作为新编程文化运动的先行者，小鲁立志改变人对文化人是编程盲的刻板印象，他立志解决各种文字处理领域的难题。为了将来能够海量的文本中统计出合乎文学特征的样本，小鲁开始尝试统计文本中的字符个数。伟大的变革，往往有个很卑微的开始。这是小鲁往文本识别领域迈出一小步：

给定n行字符串，请统计每行字符串中元音aeiou出现的次数，并且输出各元音的出现次数的统计表。

数据范围：

$0 < n < 1000$

$0 < \text{字符串长度} < 100000$

输入

第一行为一个整数n,表示要统计的字符串行数。

从第二行开始到文件结尾是n行字符串

输出

n个元音出现次数统计表

每个统计表之间用空行隔开

最后一张统计表的结束没有空行

输入样例

```
5
To know wisdom and instruction; to perceive the words of understanding;
To receive the instruction of wisdom, justice, and judgment, and equity;
To give subtilty to the simple, to the young man knowledge and discretion.
A wise man will hear, and will increase learning; and a man of understanding shall
attain unto wise counsels:
To understand a proverb, and the interpretation; the words of the wise, and their dark
sayings.
```

输出样例

```
a:2
e:5
i:5
o:7
u:2

a:2
e:7
i:6
o:4
u:4

a:2
e:7
i:5
```

```
o:6
u:2

a:13
e:8
i:8
o:3
u:3

a:7
e:9
i:5
o:5
u:1
```

参考代码(C++)

```
1. //NQ041 统计字符个数初步
2. #include <algorithm>
3. #include <cstring>
4. #include <iostream>
5. using namespace std;
6. int charCounts[6]; //存储元音 aeiou 的统计个数
7. inline int f(char c) {
8.     if (c == 'a') return 1;
9.     if (c == 'e') return 2;
10.    if (c == 'i') return 3;
11.    if (c == 'o') return 4;
12.    if (c == 'u') return 5;
13.    return 0;
14. }
15. string vowels = "aeiou";
16. int main() {
17.     int n;
18.     cin >> n;
19.     cin.get(); //去掉末尾换行符为 getline 预备读入
20.     for (int t = 0; t < n; t++) { //清空数组
21.         for (int i = 0; i < 6; i++) charCounts[i] = 0;
22.         string str;
23.         getline(cin, str); //读入一行
24.         //字符串全部变小写
25.         transform(str.begin(), str.end(), str.begin(), ::tolower);
26.         //统计 vowels 的出现次数
27.         for (int i = 0; i < str.size(); i++) charCounts[f(str[i])]++;
28.         //输出统计结果
```



```

29.     for (int i = 1; i < 6; i++)
30.         cout << vowels[i] << ":" << charCounts[f(vowels[i])] << endl;
31.     //结尾换行
32.     if (t < n - 1) cout << endl;
33. }
34. return 0;
35. }

```

1.42 不包含重复字符的最长子串长度^{NQ042}

描述

经过一段时间的学习，小鲁对字符串处理有了新的认识。他发现要用数学模型判别两篇文章是否有相似度，其中一个很有趣的指标是：一个字符串中不含有重复字符的最长子串的长度。

一串串的字符串统计何等麻烦，但是对学会编程的小鲁，这不是一个太困难的问题：

给定一个字符串，请你找出其中不含有重复字符的最长子串的长度。

输入

输入一个字符串s (长度不超过1000)。

输出

不含有重复字符的最长子串的长度。

输入样例 1

```
abrownfoxjumpsoveralazydoginxiamenuniversity
```

输出样例 1

```
12
```

输入样例 2

```
xxxxxxxxxx
```

输出样例 2

```
1
```

参考代码(C++)

```

1. // NQ042 不包含重复字符的最长子串长度
2. #include <cstring>
3. #include <iostream>
4. #include <unordered_map>
5. using namespace std;
6.

```

```

7. int lengthOfLongestSubstring(string s) {
8.     //暴力枚举的思路:
9.     //枚举所有以 i 结尾的字串, 对于每个以 i 结尾的字串, 计算最长的不重复字符串长度
10.    unordered_map<char, int> heap; //存储{char, 出现次数}的哈希表
11.    //使用双指针的方法计算
12.    int res = 0; //长度
13.    for (int i = 0, j = 0; i < s.size(); i++) {
14.        heap[s[i]]++; //将 s[i] 插入堆, 并且 s[i] 的映射项++
15.        //需要判断从 s[j]-->s[i] 的字符是否有重复, 采用 O(1) 的堆最快
16.        //若 [j,i] 区间代表最长的不重复字符串, 那么唯有新的字符 s[i] 会导致出现重复字符
17.        //因此若 s[i] 已经出现, 那么就移动 j, 从 s[j,i] 中间剔除与 s[i] 重复的字符
18.        while (heap[s[i]] > 1) heap[s[j++]]--; //这是此方法的精华所在, 大家仔细品味
19.        res = max(res, i - j + 1); //取最大值
20.    }
21.    return res;
22. }
23. int main() {
24.    string s;
25.    cin >> s;
26.    cout << lengthOfLongestSubstring(s) << endl;
27.    return 0;
28. }

```

1.43 统计班级成绩^{NQ043}

描述

为了帮助小鲁学以致用, 小华请小鲁编个程序统计班级成绩。

每班有 n ($n \leq 50$) 个学生, 每人考 m ($m \leq 5$) 门课。

请算出每个学生的平均成绩和每门课的平均成绩, 并输出各科成绩均大于等于平均成绩的学生数量。

输入

输入数据有多个测试实例, 每个测试实例的第一行包括两个整数 n 和 m , 分别表示学生数和课程数。然后是 n 行数据, 每行包括 m 个整数 (即: 考试分数)。

输出

对于每个测试实例, 输出 3 行数据, 第一行包含 n 个数据, 表示 n 个学生的平均成绩, 结果保留两位小数; 第二行包含 m 个数据, 表示 m 门课的平均成绩, 结果保留两位小数; 第三行是一个整数, 表示该班级中各科成绩均大于等于平均成绩的学生数量。每个测试实例后面跟一个空行。

输入样例 1

```
2 2
```

```
70 80
85 90
```

输出样例 1

```
75.00 87.50
77.50 85.00
1
```

参考代码(C++)

```
1. //NQ043 统计班级成绩
2. #include <stdio>
3. //二维数组存储 n 个学生 m 门课的成绩
4. int a[50][5];
5. //存储 m 门课的平均成绩
6. double b[5];
7. int main() {
8.     int n, m;
9.     double sum_stu, sum_cos;
10.    while (scanf("%d %d", &n, &m) != EOF) {
11.        //输入 n 个学生 m 门课的成绩
12.        for (int i = 0; i < n; i++) {
13.            for (int j = 0; j < m; j++) {
14.                scanf("%d", &a[i][j]);
15.            }
16.        }
17.        //输出 n 个学生的平均成绩
18.        for (int i = 0; i < n; i++) {
19.            sum_stu = 0;
20.            for (int j = 0; j < m; j++) {
21.                sum_stu += a[i][j];
22.            }
23.            if (i == n - 1)
24.                printf("%.2f\n", sum_stu / m);
25.            else
26.                printf("%.2f ", sum_stu / m);
27.        }
28.        //输出 m 门课的平均成绩
29.        for (int j = 0; j < m; j++) {
30.            sum_cos = 0;
31.            for (int i = 0; i < n; i++) {
32.                sum_cos += a[i][j];
33.            }
```

```

34.     b[j] = sum_cos / n;
35.     if (j == m - 1)
36.         printf("%.2f\n", b[j]);
37.     else
38.         printf("%.2f ", b[j]);
39. }
40. //输出每门课成绩都>=课程平均成绩的学生数
41. int cnt = 0;
42. for (int i = 0; i < n; i++) {
43.     bool is_greater = true;
44.     for (int j = 0; j < m; j++)
45.         if (a[i][j] < b[j]) is_greater = false;
46.     if (is_greater) cnt++;
47. }
48. printf("%d\n\n", cnt);
49. }
50. return 0;
51. }

```

1.44 密码安全问题^{NQ044}

描述

小鲁是个苹果产品的发烧友，手机、IPAD都是苹果的，为了学习方便，准备再买个笔记本电脑，可是又不好意思再向家里要钱，于是他想通过银行贷款来买，于是他通过QQ和计算机专业的好友小栋聊了通过贷款买笔记本的事。小栋提醒他注意校园网贷有陷阱。不曾想，第二天就有网络贷款公司主动给他打电话了。他想，难道是自己的QQ聊天信息被人窃取了，或者是密码已经泄露了？他赶紧把这事告诉小栋，问是不是密码泄露了？小栋说，很有可能啊！赶紧把密码改掉，重新设置一个安全的密码。那什么样的密码才安全呢？一般说比较安全的密码应该至少满足以下两个条件：

- (1) 密码长度大于或等于8，且不要超过16。
- (2) 密码中的字符应该包含下面四种字符中至少三种：
 - 大写字母：A, B, C...Z;
 - 小写字母：a, b, c...z;
 - 数字：0, 1, 2...9;
 - 特殊符号：~, !, @, #, \$, %, ^;

输入

第一行输入一个整数T，表示有T行测试数据

每行输入一个密码，密码仅仅包括以上所列的字符。

输出

对每个输入的密码，如果是安全密码，输出“YES”，否则输出“NO”

输入样例

```
3
Icango123
whitehouse1234
@canyou!123F
```

输出样例

```
YES
NO
YES
```

参考代码(C++)

```
1. //NQ044 密码安全问题
2. #include <iostream>
3. using namespace std;
4. int main() {
5.     int t;
6.     cin >> t;
7.
8.     while (t--) {
9.         string s;
10.        cin >> s;
11.        int len = s.size();
12.        //密码长度大于或等于 8, 且不要超过 16
13.        if (len < 8 || len > 16) {
14.            cout << "NO" << endl;
15.            break;
16.        }
17.        int j[4] = {0};
18.        for (int i = 0; i < len; i++) {
19.            //密码仅仅包括所列的 4 种字符
20.            bool a = (s[i] >= 'a' && s[i] <= 'z');
21.            bool b = (s[i] >= 'A' && s[i] <= 'Z');
22.            bool c = (s[i] >= '0' && s[i] <= '9');
23.            bool d = (s[i] == '~' || s[i] == '!' || s[i] == '@' || s[i] == '#' ||
24.                    s[i] == '$' || s[i] == '%' || s[i] == '^');
25.            if (!(a || b || c || d)) {
26.                cout << "NO" << endl;
27.                return 0;
28.            }
29.            //如果曾包含过某一种字符就赋值 1
30.            if (a) j[0] = 1;
31.            if (b) j[1] = 1;
```

```

32.     if (c) j[2] = 1;
33.     if (d) j[3] = 1;
34. }
35. //密码中的字符应该包含四种字符中至少三种
36. if (j[0] + j[1] + j[2] + j[3] > 2)
37.     cout << "YES" << endl;
38. else
39.     cout << "NO" << endl;
40. }
41. return 0;
42. }

```

1.45 奇偶校验^{NQ045}

描述

一个字符串中1的个数为奇数为奇校验，1的个数为偶数的为偶校验。0的个数不影响校验结果，全0的串为偶校验。

输入

输入一行或多行字符串，最后一行为“#”，每个字符串有1-31位，最后一位不出现，用校验位‘e’或‘o’替代。

输出

输出输入的每一行，并把最后一个校验位替换成正确的0或1。

输入样例

```

011o
110010e
1100e
01110100o
110100101o
#

```

输出样例

```

0111
1100101
11000
011101001
1101001010

```

参考代码(C++)

```

1. // NQ045 奇偶校验

```

```

2. #include <cstring>
3. #include <iostream>
4. using namespace std;
5. int main() {
6.     string s;
7.     while (cin >> s, s != "#") {
8.         int cnt = 0;
9.         int len = s.length(); //字符串长度
10.        for (int i = 0; i < len - 1; i++)
11.            if (s[i] == '1') cnt++;
12.        //前面得到的1的个数的奇偶性
13.        cnt %= 2;
14.        //判断最后一位是否要设置为1
15.        if ((s[len - 1] == 'e' && cnt == 1) || (s[len - 1] == 'o' && cnt == 0))
16.            s[len - 1] = '1';
17.        else
18.            s[len - 1] = '0';
19.        cout << s << endl;
20.    }
21.    return 0;
22. }

```

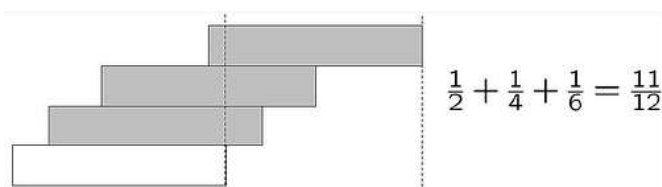
1.46 悬垂问题(Overhang) ^{NQ046}

描述

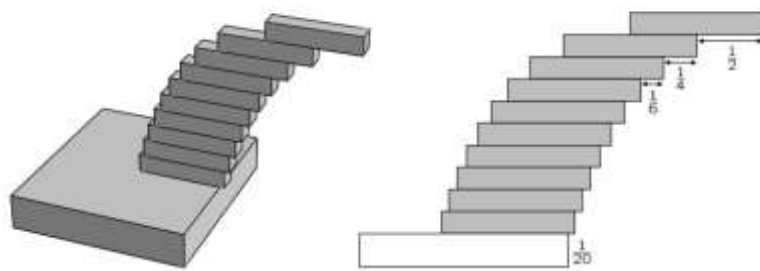
山区物资短缺，如何能够在没有水泥的前提下造出弧形的廊道？小栋决定采用悬垂的方式来设计这类型的廊道。

在开始动工前，他需要计算 n 个长度为 1 的砖块，叠起来能伸出底面多远？（只考虑方块各平面都与底面平行的情况）。

当 $n=3$ ，只有三块砖的情况下，可以这样叠放砖块，让长度为 $11/12$ 的部分伸出底部。



下图是 $n=10$ 的情况，伸出底面的长度是 $1/2 + 1/4 + 1/6 + \dots + 1/2i + \dots + 1/10 = \frac{1}{2}H_n = \frac{1}{2} \sum_{i=1}^n \frac{1}{i}$



这就是著名的悬垂问题 (Overhang)

假设需要把砖伸出底面的长度为 x ($0.01 < x < 5.00$), 请问用如上的方式堆叠需要几块砖?

输入

有若干行数字, 每行都是实数, 表示悬垂在底面之外的总长度。

读到0.00表示输入数据结束。 ($0.01 < \text{输入的数字} < 5.00$, 保留两位小数)

输出

每一行输出一个整数, 表示每组数据需要的砖块数 n 。

输入样例 1

```
1.25
3.18
4.13
0.78
0.5
```

输出样例 1

```
7 blocks
325 blocks
2171 blocks
3 blocks
1 block
```

参考代码(C++)

```
1. // NQ046 悬垂问题
2. #include <iostream>
3. using namespace std;
4. int main() {
5.     double top;
6.     while (cin >> top, top != 0) {
7.         double res = 0;
8.         //求比差数列的和
```



```

9.     int cnt;
10.    //按照题目模拟题意做判断
11.    for (cnt = 1; res < top; cnt++)
12.        res += 1.0 / (2.0 * cnt);
13.    //输出单复数有区别
14.    cout << cnt - 1 << (cnt - 1 == 1 ? " block" : " blocks") << endl;
15. }
16. return 0;
17. }

```

1.47 数组去重问题^{NQ047}

描述

小鲁想要在博览群书的前提下又学习编程，每天都在面临选择的张力，因为著书多没有穷尽，读书多身体疲倦！

他灵机一动，心里暗暗思想，每一类书其实只要读一本最好的即可，这样可以省下大量的重复阅读的时间，如果能事先知道要读多少类的书，还能提前规划自己的阅读时间，这个方法真妙。于是他花了1周的时间，把自己的藏书归类，形成一个长度为n的类别编号数组a。

请写一个函数SumofUnique计算a数组中前size个数中，独一无二值的个数：

```
int SumofUnique(int a[], int size); //返回数组前size个数中的不同数的个数。
```

数据范围

$1 \leq \text{size} \leq n \leq 1000$

输入

第一行包含两个整数n和size。

第二行包含n个整数，表示数组a。

输出

共一行，包含一个整数表示a数组中前size个数去掉重复数后，剩下的数组长度是多少。

输入样例

```

204 40
6 15 16 29 36 50 50 52 68 70 76 78 84 87 90 90 97 104 104 117 118 125 128 140 140 147
149 156 160 160 164 165 168 174 186 188 192 205 213 220 220 225 226 231 232 233 234
234 236 244 250 258 265 270 289 290 300 304 306 310 323 328 330 340 341 376 378 383
385 387 392 394 395 396 416 426 428 428 429 430 440 464 475 478 480 480 480 486 487
496 500 511 516 517 520 520 520 520 527 528 528 535 537 544 548 552 552 561 563 584
595 600 600 608 608 610 610 622 629 640 640 640 644 645 647 647 648 656 670 688 689
690 691 693 720 720 722 734 744 748 749 755 758 764 764 775 780 790 790 792 800 800
802 807 808 815 816 832 832 833 839 840 840 844 844 848 856 860 862 864 870 870 875

```

```
879 890 890 890 896 896 917 920 924 928 932 934 935 943 950 950 951 952 952 954 955
959 960 969 970 975 976 978 980 995 998
```

输出样例

```
199
```

参考代码(C++)

```
1. //NQ047 数组去重问题
2. #include <iostream>
3. using namespace std;
4. //统计前 size 中，去重后的数值个数
5. int SumofUnique(int a[], int size) {
6.     int cnt = 0; //个数
7.     for (int i = 0; i < size; i++) {
8.         bool is_exist = false;
9.         for (int j = 0; j < i; j++)
10.            if (a[j] == a[i]) {
11.                is_exist = true;
12.                break;
13.            }
14.         if (!is_exist) cnt++; //统计
15.     }
16.     return cnt; //独一无二值的个数
17. }
18. int main() {
19.     int a[1000];
20.     int n, size;
21.     cin >> n >> size;
22.     for (int i = 0; i < n; i++) cin >> a[i];
23.     //前 size 个数去重后，数值长度变为 n-size+SumofUnique
24.     cout << n - size + SumofUnique(a, size) << endl;
25.     return 0;
26. }
```

1.48 寻找多数元素^{NQ048}

描述

小鲁最近对统计很感兴趣，小华给了他一道经典题：

给定一个大小为 n 的数组，找到其中的多数元素。

多数元素是指在数组中出现次数大于 $\lfloor n/2 \rfloor$ 的元素。

你可以假设数组是非空的，并且给定的数组总是存在多数元素。

输入

输入一个数组，长度为 n 。 $(1 < n \leq 1000000)$

数组里面一定存在多数元素。

输出

输出多数元素。

输入样例

```
1 7 5 6 7 7 7 7 9
```

输出样例

```
7
```

参考代码(C++)

```
1. // NQ048 寻找多数元素
2. #include <iostream>
3.
4. using namespace std;
5. int main() {
6.     int x, r, cnt = 0;
7.     while (cin >> x) {
8.         if (!cnt)
9.             r = x, cnt = 1;
10.        else if (r == x)
11.            cnt++;
12.        else
13.            cnt--;
14.    }
15.    //返回 r 就是多数数
16.    cout << r << endl;
17.    return 0;
18. }
```

1.49 数字5的封杀^{NQ049}

描述

小鲁在某一场比赛中做出5道题后不幸打铁，于是一气之下与数字5杠上了。

当小鲁拿到一行数字，他会首先把这行数字中所有的5换成空格，以达到眼不见心不烦的效果。

进行完这项操作后，一串数字被拆成若干个新的数字（忽略整数开头的0；每个位都是0的数输出0）。

最后对分割后得到的数字从小到大的排序。

输入

输入包含多组测试用例，每组输入数据只有一行数字（数字之间没有空格），这行数字的长度不大于1000。

输入数据保证：分割得到的非负整数不会大于100000000；输入数据不可能全由`5`组成。

输出

对于每个测试用例，输出分割得到的整数排序的结果，相邻的两个整数之间用一个空格分开，每组输出占一行。

输入样例

```
0551223532050775
```

输出样例

```
0 77 320 1223
```

参考代码(C++)

```
1. // NQ049 数字 5 的封杀
2. #include <algorithm>
3. #include <iostream>
4. #include <string>
5. #include <vector>
6. using namespace std;
7.
8. int main() {
9.     vector<int> res; //结果数组
10.    string str; //输入字符串
11.    getline(cin, str); // 读入一整行字符串
12.    //把 5 当作分隔符来处理
13.    if (str.back() != '5') str += '5'; //在末尾加上 5 作为结束符
14.    string number; //数字字符串
15.    for (auto c : str) {
16.        if (c != '5')
17.            number += c;
18.        else if (!number.empty()) {
19.            while (number.size() > 1 && number[0] == '0')
20.                number = number.substr(1); //去掉前面的 0
21.            res.push_back(stoi(number)); //转换为整数
22.            number.clear(); //清空当前的数字字符串
23.        }
24.    }
25.    //排序
26.    sort(res.begin(), res.end());
```

```

27. //输出
28. for (auto s : res) cout << s << ' ';
29. cout << endl;
30. return 0;
31. }

```

1.50 排序考试^{NQ050}

描述

处于某种原因，一点都不懂编程的某某系小讯老师竟然成功跳槽到计算机系来教大一新生如何编程。看着这位在台上胡言乱语，水平比同学小华弱太多的老师，小鲁简直不忍直视。

那学期的期中考试，小讯老师出的题目竟然是：“请写一个排序算法给数组排序，结果按照升序输出。”

经过NQ48斩的小鲁分分钟就完成了代码。小讯老师一看，哎呀不得了，得提高期中考试难度。

他立刻把题目改为：“给定任意T组整数，每组整数都要按升序输出。”

小鲁笑了笑，原来这类题已经难不倒他了，原来他早就超过了大一上小讯老师的水平了！

小鲁水平进阶了，你做得到的吗？

输入

第一行是整数T，表示一共有T组数据。

接下来T行，每行有N+1个数，第一个整数表示该行有N个待排序的数字。

整数N ($1 \leq N \leq 1000000$)，T ($1 \leq T \leq 100$)。

输出

对于每组整数，按照升序输出排序结果，每个结果占一行。

输入样例

```

3
4 412 120 5560 3760
5 576 66 35 99 88
4 127 100 510 380

```

输出样例

```

120 412 3760 5560
35 66 88 99 576
100 127 380 510

```

参考代码(C++)

```

1. // NQ050 排序考试
2. #include <algorithm>

```

```
3. #include <iostream>
4. #include <vector>
5. using namespace std;
6. int main() {
7.     int T;
8.     cin >> T;
9.     while (T--) {
10.        int n, a, i;
11.        cin >> n;
12.        //读入每组数据
13.        vector<int> nums;
14.        for (i = 0; i < n; i++) {
15.            cin >> a;
16.            nums.push_back(a);
17.        }
18.        sort(nums.begin(), nums.end()); //排序
19.        //循环输出数组
20.        for (i = 0; i < n - 1; i++) cout << nums[i] << " ";
21.        //换行
22.        cout << nums[i] << endl;
23.    }
24.    return 0;
25. }
```

第2章 南强百题基础算法篇

2.1 数组元素交换^{NQ051}

描述

校长小嘉常常要做艰难的人事决定，要选取哪些人当领导，哪些人要放到那个岗位上工作，这些决定都非常需要智慧，需要深思熟虑，需要对人的现状和潜力有准确的评估。

假设数组 $a[n]$ 存储的是某个部门各个教职工的负评率，数组的值越高表示这个人的综合素质越低，服务的满意度越低。

数组的第一个元素 $a[0]$ 代表部门领导的负评率。小嘉设立的换帅规则是：找到数组中负评率最低的值，与数组首位交换。在新一轮的人事任命中，排首位的就当部门领导。

你可以写一个程序帮助小嘉这个事关重大的数组元素的交换吗？

输入

输入数据有若干行，每行第一个元素是 n ，代表数组元素的个数，接下来 n 个数代表数组 $a[0 \dots n-1]$ 的值。

当读到 $n=0$ 的时候输入表示数据结束。

数据范围： $1 < n < 1000$ ； $-100000 < \text{负评率} < 100000$

输出

按次序输出交换后的数组 $a[n]$ 的每个元素

每一行代表一组数据的处理结果

（注意：每一行最后一个数字后没有空格，直接换行）。

输入样例

```
13 -712 -629 488 423 471 428 -494 630 -497 620 -9 248 -118
12 383 614 -637 -655 683 586 -777 508 -691 -96 -333 377
0
```

输出样例

```
-712 -629 488 423 471 428 -494 630 -497 620 -9 248 -118
-777 614 -637 -655 683 586 383 508 -691 -96 -333 377
```

参考代码(C++)

```
1. //NQ051 数组元素交换
2. #include <iostream>
3. #include <algorithm>
```

```

4. using namespace std;
5. #define INT_MAX 2147483647
6. const int N = 10007;
7. int a[N], m, p; //数组,最小值,最小值下标
8. int main()
9. {
10.     int n;
11.     while (cin >> n, n > 0)
12.     {
13.         m = INT_MAX; //初始化为最大值
14.         p = 0; //数字下标初始化为第一项
15.         for (int i = 0; i < n; i++)
16.         {
17.             cin >> a[i]; //读入数据
18.             if (m > a[i]) m = a[i], p = i; //记录最小值和相应下标
19.         }
20.         swap(a[0], a[p]); //交换
21.         for (int i = 0; i < n; i++)
22.             if (i < n - 1) cout << a[i] << " "; else cout << a[i] << endl;
23.     }
24.     return 0;
25. }

```

2.2 大数排序^{NQ052}

描述

小鲁虽然编程能力很一般，但是嘴皮子上不服输。他很喜欢拿自己不懂的问题考小华。

刚学会冒泡排序的他，决定挑战一下小华的智商：

给定你一个长度为 n 的整数数列。

请你对这个数列按照从小到大进行排序。

并将排好序的数列按顺序输出。

小鲁刻意隐瞒了数据的规模，你觉得小华能够搞定吗？

偷偷告诉你： $1 \leq n \leq 100000$ ，所有整数均在 $1--10^9$ 范围内

后记：小华使用快速排序秒过，小鲁惨败，因为他还看不懂小华的代码.....

输入

输入共两行，第一行包含整数 n 。

第二行包含 n 个整数，表示整个数列。

输出

输出共一行，包含 n 个整数，表示排好序的数列。

输入样例

```
12
111584322 907287398 13562672 615771042 83035480 323016588 501254568 158361642
388135644 58329696 400904710 38908140
```

输出样例

```
13562672 38908140 58329696 83035480 111584322 158361642 323016588 388135644 400904710
501254568 615771042 907287398
```

参考代码(C++)

```
1. // NQ052 大数排序
2. #include <iostream>
3. #include <cmath>
4. #include <algorithm>
5. using namespace std;
6.
7. long long *numbers;
8.
9. void quicksort(long long nums[], int left, int right)
10. {
11.     if (left >= right)
12.         return;
13.     //选取分割数
14.     long long target = nums[left]; //选取第一个快排的分割数
15.     int i = left - 1, j = right + 1;
16.     while (i < j)
17.     {
18.         do
19.             i++;
20.         while (nums[i] < target); //i 指针定位
21.         do
22.             j--;
23.         while (nums[j] > target); //j 指针定位
24.         if (i < j)
25.             swap(nums[i], nums[j]); //交换
26.     }
27.     //分治
28.     quicksort(nums, left, j); //j 左边的都是不比 Target 大的
29.     quicksort(nums, j + 1, right); // j 右边的都比 Target 大
```

```

30. }
31. int main()
32. {
33.     //!cin\cout 提速
34.     ios::sync_with_stdio(false); //来打消 iostream 的输入输出缓存
35.     cin.tie(0);                  //cin.tie(0)来解除 cin 与 cout 的绑定,0 表示 NULL
36.     int n;
37.     //创建动态数组
38.     cin >> n;
39.     numbers = new long long[n];
40.     for (int i = 0; i < n; i++)
41.         cin >> numbers[i];
42.     quicksort(numbers, 0, n - 1); //注意边界
43.     for (int i = 0; i < n; i++)
44.         cout << numbers[i] << " ";
45.     delete numbers;
46.     return 0;
47. }

```

2.3 找到最长回文字串^{NQ053}

描述

回文诗乃小鲁的挚爱，他第一次接触的回文诗是宋代李禺的回文诗《两相思》

《思妻诗》

枯眼望遥山隔山，往来曾见几心知？壶空怕酌一杯酒，笔下难成和韵诗。
途路阻人离别久，讯音无雁寄回迟。孤灯夜守长寥寂，夫忆妻兮父忆儿。

《思夫诗》

儿忆父兮妻忆夫，寂寥长守夜灯孤。迟回寄雁无音讯，久别离人阻路途。
诗韵和成难下笔，酒杯一酌怕空壶。知心几见曾往来，山隔山遥望眼枯。

大文豪苏轼自然也有回文佳作：

第一首：酴颜玉碗捧纤纤，乱点余花吐碧衫。歌咽水云凝静院，梦惊松雪落空岩。

第二首：空花落尽酒倾缸，日上山融雪涨江。红培浅瓿新火活，龙团小碾斗晴窗。

字符串处理中，怎能少得了回文？小鲁从沉思回到自己眼前的编程学习中，他想知道一个最大长度不超过1000的字符串 *s* 中，是否有回文子串，请找到 *s* 中最长的回文子串。

输入

一个长度不超过1000的字符串*s*

输出

最长的回文字串（数据保证有唯一解）

输入样例

```
iknowiloveyouuoyevoliiiknlow
```

输出样例

```
iloveyouuoyevoli
```

参考代码(C++)

```
1. // NQ037 最长回文子串
2. #include <cstring>
3. #include <iostream>
4. using namespace std;
5. string longestPalindrome(string s) {
6.     string res; //结果字符串
7.     for (int i = 0; i < s.size(); i++) {
8.         //奇数个数回文的情况 Left=i-1; right=i+1,以 i 为中心的回文字串
9.         int left = i - 1, right = i + 1;
10.        while (left >= 0 && right < s.size() && s[left] == s[right])
11.            left--, right++;
12.        //回文字串的长度是 (r-1)-(l+1)+1=r-l-1
13.        if (res.size() < right - left - 1)
14.            res = s.substr(left + 1, right - left - 1); //保留最长的字符串
15.        //偶数个数回文的情况 Left=i; right=i+1,以 i, i+1 为中心的回文字串
16.        left = i;
17.        right = i + 1;
18.        while (left >= 0 && right < s.size() && s[left] == s[right])
19.            left--, right++;
20.        if (res.size() < right - left - 1)
21.            res = s.substr(left + 1, right - left - 1); //保留最长的字符串
22.    }
23.    return res;
24. }
25. int main() {
26.     string s;
27.     cin >> s;
28.     cout << longestPalindrome(s) << endl;
29.     return 0;
30. }
```

2.4 ACM纳新赛^{NQ054}

描述

每年信息学院的ACM队都会举办一个ACM纳新赛，在新生中选拔编程能力强的同学参加ACM集训队。今年的纳新赛在体育馆进行，所有同学都可以参加现场编程。体育馆排放着 m 行 n 列的电脑，足够让愿意参加的大一新生用。

信息学院的纳新赛有个特殊奖：为分数绝对值为最大的学生颁发特别奖。（绝对值为最大：若分数为正数最大，那就获得冠军；若分数为负数最大，则获得特别参与奖。）

小华和小鲁都参加纳新赛，小华是肯定能轻松斩获冠军，至于小鲁，你看行吗？

输入

第一行输入 m 和 n ，分别表示比赛的同学分为 m 行 n 列。

接着输入 m 行 n 列整数，代表每个学生的分数值，最后输入为0（即第 m 行第 n 列）。

输出

对每组输入，输出三个整数 x, y, s ，分别表示行号，列号和该学生分数。

输入样例

```
3 4
12 -10 35 78
-4 34 60 57
2 85 -13 60
```

输出样例

```
3 2 85
```

参考代码(C++)

```
1. // NQ054 ACM 纳新测试赛
2. #include <iostream>
3. using namespace std;
4. int main() {
5.     int m, n;
6.     cin >> m >> n;
7.     int a[m][n]; // 定义一个 m 行 n 列的数组
8.     for (int i = 0; i < m; i++) {
9.         for (int j = 0; j < n; j++) {
10.            cin >> a[i][j];
11.        }
12.    }
13.    int max = 0;
14.    int max_m, max_n;
```

```
15.  for (int i = 0; i < m; i++) // 找到数组中最大的数，并且记录下它的值和索引
16.  {
17.      for (int j = 0; j < n; j++) {
18.          if (a[i][j] > max) {
19.              max = a[i][j];
20.              max_m = i + 1;
21.              max_n = j + 1;
22.          }
23.      }
24.  }
25.  cout << max_m << " " << max_n << " "
26.      << a[max_m - 1][max_n - 1]; // 输出结果，注意数组下标
27.  return 0;
28. }
```

2.5 最小公倍数^{NQ055}

描述

最小公倍数是编程中常常考察的知识点，为了重建小鲁的数学根基，消除小鲁对数学的惧怕，小华给小鲁出了道简单题：

求n个数的最小公倍数问题。

输入

输入包含多个测试实例，每个测试实例的开始是一个正整数n，然后是n个正整数。

输出

为每组测试数据输出它们的最小公倍数，每个测试实例的输出占一行。你可以假设最后的输出是一个32位的整数。

输入样例

```
3 12 8 6
4 4 7 5 8
```

输出样例

```
24
280
```

参考代码(C++)

```
1. #include <iostream>
2. using namespace std;
3. int a[10000];
4. // 求两个数的最小公倍数的方法
5. int gcd(int a, int b) {
6.     int tmp;
7.     // 如果 a 大于 b 就交换 a 和 b 的值
8.     if (a > b) {
9.         tmp = a, a = b, b = tmp;
10.    }
11.    tmp = a;
12.    if (a > 1) {
13.        // 从两个数中较小的数开始做递减的循环
14.        while (a) {
15.            if (tmp % a == 0 && b % a == 0)
16.                return (b * tmp) / a;
17.            a--;
18.        }
```

```

19. }
20. return b;
21. }
22.
23. int main() {
24.     int n;
25.     while (scanf("%d", &n) != EOF) {
26.         for (int i = 0; i < n; i++) {
27.             cin >> a[i];
28.             if (i > 0) a[i] = gcd(a[i], a[i - 1]);
29.         }
30.         cout << a[n - 1] << endl;
31.     }
32.     return 0;
33. }

```

2.6 三数判断^{NQ056}

描述

小栋想要将宿舍楼的顶部设计成三角形，有一堆长度不等的，粗细相同的梁木，小鲁也想试着帮忙，但是没有学过排列组合。

小栋不忍打击小鲁的热情，他请小鲁帮忙计算，给定三根长度不同的梁木，请问这三根梁木是否可以构成一个三角形。

问题抽象为：输入三个数 a, b, c ，请问 a, b, c 是否可以构成一个合法的三角形。（ a, b, c 为正整数）

输入

输入数据第一行为 n ，表示有 n 组数据需要小鲁判断（ $0 < n < 100$ ）。

其余 n 行，每行有三个正整数 $a \ b \ c$ （ $0 < \text{输入数据} < 1000$ ）

输出

输出 n 行，如果可以构成三角形就输出：OK 否则输出 NO

输入样例1

```

3
4 5 6
7 8 9
5 5 10

```

输出样例1

```

OK
OK

```

NO

输入样例2

```
13
560 500 484
484 425 922
551 524 802
624 240 875
563 609 82
390 851 320
845 704 560
990 716 609
536 358 800
224 755 609
894 441 85
228 88 329
952 362 480
```

输出样例2

```
OK
NO
OK
NO
OK
NO
OK
OK
OK
OK
NO
NO
NO
```

参考代码(C++)

```
1. // NQ056 三数判断
2. #include <iostream>
3. #include <algorithm>
4. using namespace std;
5. const int N = 7; // 只需要判断 3 个数
6. int a[N];
7. int main()
8. {
9.     int n; cin >> n; // 读入 n
```



```

10. while (n--){//n 组数据
11.     cin>>a[0];//读入第一个数，这个数存储最大值
12.     for (int i = 1; i <= 2; i++)
13.     {
14.         cin >> a[i];//读入数据
15.         if (a[0]<a[i]) swap(a[0],a[i]);//a[0]存储最大值
16.     }
17.     cout<<(a[0]<a[1]+a[2]?"OK":"NO")<<endl;//输出可否构成三角形
18. }
19. return 0;
20. }

```

2.7 灯的开关^{NQ057}

描述

为了让小鲁意识到编程中数学和推理的重要，小华行小鲁过来，问他一个脑筋急转弯的题：

初始时有 n 个灯泡关闭。第 1 轮，你打开所有的灯泡。第 2 轮，每两个灯泡你关闭一次。第 3 轮，每三个灯泡切换一次开关（如果关闭则开启，如果开启则关闭）。第 i 轮，每 i 个灯泡切换一次开关。对于第 n 轮，你只切换最后一个灯泡的开关。

请统计出 n 轮后有多少个亮着的灯泡。

输入

输入若干行测试数据

每行是一个正整数 n ($1 < n < 10^9$)，表示有 n 个灯泡，经过 n 轮的开关实验。

输出

每个测试数据输出一行，表示最后一共有多少灯泡是亮着的。

输入样例

```

3
5
7
10

```

输出样例

```

1
2
2
3

```

参考代码(C++)

```
1. // NQ057 灯的开关
2. #include <iostream>
3. #include <cmath>
4. using namespace std;
5. int main()
6. {
7.     int n;
8.     while(cin>>n)
9.         cout<<(int)sqrt(n)<<endl;
10.    return 0;
11. }
```

2.8 两数之和^{NQ058}

描述

双指针是极其常用的算法，这是必须学会，也是不可不会的。

但是双指针看似简单，背后的思想并不容易掌握，小华深谙此理，为了帮助小鲁一步一步的掌握这编程利器，他为小鲁精心设计了三道题：

第一题：

给定一个目标值 `target`，请你在不包含重复元素的按升序排列的整数数组 `a` 中，找出和为目标值的那两个整数，并返回他们的数组下标。

你可以假设每种输入只会对应一个答案。

例如:给定 `a = [2, 7, 10, 15]`, `target = 17`, 因为 `a[1] + a[2] = 7 + 10 = 17`, 所以返回 `[1 2]`

输入

输入数据为2行，第一行有两个整数 `target`和`n`，其中`target`代表要搜索的目标和，`n`表示数组`a`的元素个数

第二行表示数组`a`的`n`个数，每个元素用空格隔开。

输出

输出和为`target`的两个元素的下标 `i j`，其中 $(i < j)$ 。

输入样例

```
17 7
1 3 5 7 10 11 19
```

输出样例

```
3 4
```

参考代码(C++)

```
1. // NQ058 两数之和
2. #include <iostream>
3. #include <vector>
4. using namespace std;
5. int main() {
6.     int target, n, x;
7.     vector<int> a;
8.     cin >> target >> n;
9.     for (int i = 0; i < n; i++) {
10.         cin >> x;
11.         a.push_back(x);
12.     }
13.     //双指针算法
14.     for (int i = 0, j = n - 1; i <= j; i++) {
15.         while (j > i && a[i] + a[j] > target) j--;
16.         if (a[i] + a[j] == target) {
17.             cout << i << " " << j << endl;
18.             break;
19.         }
20.     }
21.     return 0;
22. }
```

2.9 三数之和^{NQ059}

描述

看着小鲁AC了第一题，小华接着提出第二问：

给定一个目标值 $target$ ，请在整数数组 a 中，找出三个元素 (x, y, z) 使 $x+y+z=target$ 。

请找到所有满足条件的三元组，并且请按从小到大的顺序输出所有合法的三元组。

注意：三元组中不允许包含重复数字，且输出的三元组中要求 $x < y < z$ 。

例如：给定 $target = 17$ ， $n=7$ ，数组 $a = [0, 2, 7, 10, 15, 18, 25]$

结果返回两个三元组： $(0, 2, 15)$ ， $(2, 7, 10)$

输入

输入数据为2行，第一行有两个整数 $target$ 和 n ，其中 $target$ 代表要搜索的目标和， n 表示数组 a 的元素个数

第二行表示数组 a 的 n 个数，每个元素用空格隔开。

输出

输出所有满足和为target的三元组(x,y,z)，要求(x< y <z) 并且不允许有重复数字。

把三元组按照x的大小升序输出，x相同的按照y的大小升序输出。

输入样例

```
17 7
0 2 7 10 15 18 25
```

输出样例

```
0 2 15
0 7 10
```

参考代码(C++)

```
1. // NQ059 三数之和
2. #include <algorithm>
3. #include <iostream>
4. #include <vector>
5. using namespace std;
6. vector<vector<int>> threeSum(vector<int>& nums, int target) {
7.     vector<vector<int>> res;
8.     //双指针做法
9.     // 1.排序
10.    sort(nums.begin(), nums.end());
11.    // 2.双指针,3重循环 i,j,k 要求 i<j<k
12.    for (int i = 0; i < nums.size(); i++) {
13.        //跳过重复项
14.        if (i && nums[i] == nums[i - 1]) continue;
15.        //双指针循环
16.        for (int j = i + 1, k = nums.size() - 1; j < k; j++) {
17.            //跳过重复项
18.            if (j > i + 1 && nums[j] == nums[j - 1]) continue;
19.            //预判下一个数,如果下一个数 k 与 j 没有重叠,并且下一个数满足 nums[i]+num[j]+nums[k-1]>=0
20.            //移动 k 指针 k--
21.            while (j < k - 1 && nums[i] + nums[j] + nums[k - 1] >= target) k--;
22.            //判断是否找到和为 0 的三个数
23.            if (nums[i] + nums[j] + nums[k] == target)
24.                res.push_back({nums[i], nums[j], nums[k]});
25.        }
26.    }
27.    return res;
28. }
29. //三元组排序的比较函数
```

```

30. bool comp(vector<int>& a, vector<int>& b) {
31.     if (a[0] < b[0])
32.         return true;
33.     else if (a[0] == b[0] && a[1] < b[1])
34.         return true;
35.     else
36.         return false;
37. }
38. int main() {
39.     int target, n, x;
40.     vector<int> a;           //输入数组
41.     vector<vector<int>>> res; //存储三元组的二维数组
42.     cin >> target >> n;
43.     for (int i = 0; i < n; i++) {
44.         cin >> x;
45.         a.push_back(x);
46.     }
47.     //寻找三元组
48.     res = threeSum(a, target);
49.     //排序三元组
50.     sort(res.begin(), res.end(), comp);
51.     //输出三元组
52.     for (auto line : res)
53.         cout << line[0] << " " << line[1] << " " << line[2] << endl;
54.
55.     return 0;
56. }

```

2.10 四数之和^{NQ060}

描述

当小鲁AC了第二问，小华接着提出第三问：

给定一个目标值 `target`，请在整数数组 `A` 中，找出四个元素 `(a, b, c, d)` 使 `a+b+c+d==target`。

请找到所有满足条件的四元组，并且请按从小到大的顺序输出所有合法的四元组。

注意：四元组中不允许包含重复数字，且输出的四元组中要求 `a<b<c<d`

例如：给定 `target = 17`，`n=7`，数组 `a= [0, 2, 5, 10, 15, 18, 25]`

结果返回两个四元组： `(0, 2, 5, 10)`

输入

输入数据为2行，第一行有两个整数 `target` 和 `n`，其中 `target` 代表要搜索的目标和，`n` 表示数组 `A` 的元素个数

第二行表示数组A的n个数，每个元素用空格隔开。

输出

输出所有满足和为target的四元组 (a,b,c,d) 使 (a<b<c<d) 并且不允许有重复数字。

把四元组按照a的大小升序输出，a相同的按照b的大小升序输出，a,b相同的按照c的大小升序输出。

输入样例

```
17 7
0 2 5 10 15 18 25
```

输出样例

```
0 2 5 10
```

参考代码(C++)

```
1. // NQ060 四数之和
2. #include <algorithm>
3. #include <iostream>
4. #include <vector>
5. using namespace std;
6. vector<vector<int>> fourSum(vector<int>& nums, int target) {
7.     vector<vector<int>> res;           //答案数组
8.     sort(nums.begin(), nums.end()); //排序
9.
10.    //四重循环，后两重用双指针算法 i,j,k,l
11.    for (int i = 0; i < nums.size(); i++) {
12.        //去重
13.        if (i && nums[i] == nums[i - 1]) continue;
14.        for (int j = i + 1; j < nums.size(); j++) {
15.            //去重
16.            if (j > i + 1 && nums[j] == nums[j - 1]) continue;
17.            //双指针算法
18.            for (int k = j + 1, l = nums.size() - 1; k < l; k++) {
19.                //去重
20.                if (k > j + 1 && nums[k] == nums[k - 1]) continue;
21.                while (k < l - 1 && nums[i] + nums[j] + nums[k] + nums[l - 1] >= target)
22.                    l--; //找到最小的 l
23.                if (nums[i] + nums[j] + nums[k] + nums[l] == target) //找到一组解
24.                    res.push_back({nums[i], nums[j], nums[k], nums[l]});
25.            }
26.        }
27.    }
28.    return res;
29. }
```

```

30. //四元组的比较函数
31. bool comp(vector<int>& a, vector<int>& b) {
32.     if (a[0] < b[0])
33.         return true;
34.     else if (a[0] == b[0] && a[1] < b[1])
35.         return true;
36.     else if (a[0] == b[0] && a[1] == b[1] && a[2] == b[2])
37.         return true;
38.     else
39.         return false;
40. }
41.
42. int main() {
43.     int target, n, x;
44.     vector<int> a;
45.     vector<vector<int>> res;
46.     cin >> target >> n;
47.     for (int i = 0; i < n; i++) {
48.         cin >> x;
49.         a.push_back(x);
50.     }
51.     //寻找四元组
52.     res = fourSum(a, target);
53.     //输出四元组
54.     sort(res.begin(), res.end(), comp);
55.     for (auto line : res)
56.         cout << line[0] << " " << line[1] << " " << line[2] << " " << line[3]
57.             << endl;
58.
59.     return 0;
60. }

```

2.11 求矩形重叠面积^{NQ061}

描述

小栋看着新校区的教学楼设计图俯视正交平面设计图，看着大大小小的矩形，他需要知道其中有多少矩形与其他矩形重叠。

为了简化问题，我们把 n 个矩形两两分组，请求出每组数据中的两个矩形的重叠面积。

输入

输入数据有若干行。

每一行是8个正数 $x_1, y_1, x_2, y_2, x_3, y_3, x_4, y_4$ 。第一个矩形的对角线顶点是 (x_1, y_1) 、

(x2, y2) 第二个矩形的对角线顶点是 (x3, y3)、(x4, y4)。

输出

对于每组数据，输出重叠的面积值，结果保留2位小数。

输入样例

```
1.00 1.00 3.00 3.00 2.00 2.00 4.00 4.00
5.00 5.00 13.00 13.00 4.00 4.00 12.50 12.50
```

输出样例

```
1.00
56.25
```

参考代码(C++)

```
1. // NQ061 求矩形重叠面积
2. #include <iostream>
3. #include <algorithm>
4. using namespace std;
5. int main()
6. {
7.     double x1, y1, x2, y2, x3, y3, x4, y4;
8.     while(scanf("%lf %lf %lf %lf %lf %lf %lf %lf", &x1, &y1, &x2, &y2, &x3, &y3, &x4, &y4)
9.         != EOF)    {
10.         if(x1 > x2) swap(x1, x2);
11.         if(x3 > x4) swap(x3, x4);
12.         if(y1 > y2) swap(y1, y2);
13.         if(y3 > y4) swap(y3, y4);
14.         double minx = x1 > x3 ? x1 : x3;
15.         double maxx = x2 < x4 ? x2 : x4;
16.         double miny = y1 > y3 ? y1 : y3;
17.         double maxy = y2 < y4 ? y2 : y4;
18.         if(minx > maxx || miny > maxy)
19.             printf("0.00\n");
20.         else
21.             printf("%.2lf\n", (maxx - minx) * (maxy - miny));
22.     }
23.     return 0;
24. }
```


2.12 二进制中1的个数^{NQ062}

描述

经过一段时间的琢磨，小鲁意识到很多算法优化都需要计算二进制中1的个数，并且这个操作常被称为lowbit操作。

常用的问题如下：

给定一个长度为n的数列，请你求出数列中每个数的二进制表示中1的个数。

数据范围： $1 \leq n \leq 100000$, $0 \leq \text{数列中元素的值} \leq 10^9$

输入

第一行包含整数n。

第二行包含n个整数，表示整个数列。

输出

共一行，包含n个整数，其中第 i 个数表示数列中的第 i 个数的二进制表示中1的个数。

输入样例

```
7
28935360 1830078 15857560 25537776 425203316 219792210 511045995
```

输出样例

```
10 15 15 12 13 13 19
```

参考代码(C++)

```
1. // NQ062 二进制中 1 的个数
2. #include <iostream>
3. using namespace std;
4. inline int lowbit(int &x)
5. {
6.     return x & -x; // x 的二进制的最后一个 1 所构成的整数
7. }
8. int main()
9. {
10.    int n, x;
11.    cin >> n;
12.    while (n--)
13.    {
14.        // 读入一个数，并且计算 1 的个数，并且输出
15.        cin >> x;
16.        int res = 0; // 计数变量
17.        while (x)
```

```

18.         x -= lowbit(x), res++; //使用,表达式, 简化代码去括号
19.         cout << res << " ";      //输出运算结果
20.     }
21.     return 0;
22. }

```

2.13 求上台阶方案数^{NQ063}

描述

栋栋大礼堂是小栋设计的标志性建筑，虽然礼堂自身高度不高，但是因为依山而建，礼堂门口有百级台阶，人要拾级而上方能抵达。

假设礼堂前的台阶数是N，小鲁上台阶有两种方式，一种是一次上一级台阶，另一种是一次上两级台阶。

请问，要抵达大礼堂，小鲁一共有多少种不同的走法？

输入

第一行是整数q表示测试实例的个数。

然后是Q行数据，每行包含一个整数N（ $1 \leq N \leq 40$ ），表示台阶的级数。

输出

一共q行，每行表示不同的走法总数。

输入样例1

```

3
20
30
40

```

输出样例

```

10946
1346269
165580141

```

参考代码(C++)

```

32. // NQ063 求上台阶方案数
33. #include <iostream>
34. using namespace std;
35. int N;
36. int stairs(int n)
37. {

```

```

38.     if (n == 1)//只有 1 级台阶
39.         return 1;
40.     else if (n == 2)//有 2 级台阶
41.         return 2;
42.     else
43.         return stairs(n - 1) + stairs(n - 2);//递归
44. }
45. int main()
46. {
47.     int Q;
48.     while (scanf("%d", &Q) != EOF)//使用 scanf 也可以使用 cin
49.     {
50.         while (Q--)
51.         {
52.             scanf("%d", &N);
53.             printf("%d\n", stairs(N));//调用递归函数
54.         }
55.     }
56.     return 0;
57. }

```

2.14 小鲁与猫的羁绊^{NQ064}

描述

文艺青年小鲁喜欢猫，他老家的家里养了一只公主（母）猫。编码的日子挺苦，宿舍又不允许养猫。小鲁只能看着老家的视频云养猫。

相思成灾的小鲁想，如果我家的这只公主猫每年都会生一只小公主猫，小公主猫长到4周岁后可以生小猫，新生的小公主猫出生4年后每年也会生出一只小公主猫。

那么请问 n 年后，家里一共有多少只公主猫呢？

提示：这是一个递推问题，问第几年共有几只猫。

分析可知，当 $n \leq 4$ 时，第 i 年就有 i 只。 $n > 4$ 时，设 $a[n]$ 为第 n 年的公主猫数，分析可得： $a[n] = a[n-1] + a[n-3]$ 。

输入

第一行输入正整数 n ，表示有 n 个测试实例。

接着输入 $n+1$ 行，每行输入一个正整数 m ，表示第 m 年。第 $n+1$ 行输入0，表示输入结束。

输出

一共 q 行，每行表示不同的走法总数。

输入样例

```
3
4
6
7
```

输出样例

```
4
9
13
```

参考代码(C++)

```
1. // NQ064 小鲁与猫的羁绊
2. #include <iostream>
3. using namespace std;
4. int fn(int x) {
5.     if (x < 5)
6.         return x;
7.     else
8.         return fn(x - 1) + fn(x - 3);
9. }
10. int main() {
11.     int n, m;
12.     cin >> n;
13.
14.     while (n--) {
15.         cin >> m;
16.         cout << fn(m) << endl;
17.     }
18.     return 0;
19. }
```

2.15 计算N!NQ065

描述

小鲁掌握了如何用递归函数计算阶乘，他发现自己的函数只能计算到20！

如何能计算更大数的阶乘呢？他带着问题来问小华：

给定一个正整数 N ($0 \leq N \leq 10000$)，应该如何计算 $N!$ 呢？

小华笑着说，这种计算方法和高精度乘法相似，需要自己用数组来模拟某个整数，并且注意乘法过程中的进位处理。

言谈之间，小华就把代码写完了。小鲁看着小华的代码，敬仰之情如滔滔江水。
你呢？也来试试吧？

输入

每个N占一行,直到文件尾

输出

每个N,输出一行N!

输入样例

```
2
3
4
```

输出样例

```
2
6
24
```

参考代码(C++)

```
1. // NQ065 计算 N!  
2. #include <iostream>  
3. #include <vector>  
4. using namespace std;  
5. vector<int> res;  
6. int main() {  
7.     int n;  
8.     while (cin >> n) {  
9.         if (n == 1)  
10.            cout << '1' << endl; // n 为 1 时, 直接打印结果  
11.         else {  
12.             res.push_back(1); //先向向量中插入一个数字  
13.             for (int i = 2; i <= n; ++i) { //逐步操作到 n  
14.                 for(auto&x: res) x*=i;  
15.                 int t = 0; //变量 t 用于保存该位要向上进位的数字  
16.                 for (int j = res.size() - 1; j >= 0; --j) {  
17.                     //变量 temp 用于保存该位上加上进上来的数字后的结果  
18.                     int temp = t + res[j];  
19.                     t = temp / 10; //更新变量 t  
20.                     res[j] = temp % 10; //确定该位上的数字  
21.                 }  
22.                 //对最前面一位进上去的数字进行处理, 由于这个数字可能不止一位  
23.                 //所以要用一个 while 循环一步步插到最前面
```

```

24.     while (t) {
25.         res.insert(res.begin(), t % 10);
26.         t /= 10;
27.     }
28. }
29. for(auto x:res) cout<<x;
30.     cout << endl;
31. }
32. res.clear();
33. }
34. return 0;
35. }

```

2.16 求谦虚数 NQ066

描述

经过在线编程的洗礼，小鲁越来越看到自己与小华的差距，当他真实面对自己的软弱之后，他谦卑了下来，从以前与小华吹毛求疵，到现在向小华虚心求教。

小华很开心，让小鲁尽可能算出200000000之内的所有谦虚数，并且要求他提供谦虚数的查询服务。

这个小小的挑战旨在训练小鲁的递推能力，并且进一步帮助小鲁学习谦虚。

谦虚数定义如下：一个数，如果所有的因数都是2，3，5，7，那么这个数被称为谦虚数。

前20个谦虚数如下表：

序号	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	
谦虚数	1	2	3	4	5	6	7	8	9	10	12	14	15	16	18	20	21	24	25	27	

输入

输入q组询问

每组询问是一个数字n

(1<=n<=2000)

输出

每组询问输出一行，每行是一个数字代表第n号谦虚数是多少

输入样例

```

7
7
27
77
777
940
945

```

1777

输出样例

```
7
42
256
150528
302526
307328
4147200
```

参考代码(C++)

```
1. // NQ066 求谦虚数
2. #include <iostream>
3. #include <cstring>
4. using namespace std;
5. const int N = 6007; //最大是 5842
6. int a[N];
7. bool is_humnumber(int x)
8. {
9.     int res = x;
10.    while (res % 2 == 0) res /= 2;
11.    while (res % 3 == 0) res /= 3;
12.    while (res % 5 == 0) res /= 5;
13.    while (res % 7 == 0) res /= 7;
14.    return res == 1;
15. }
16. int main()
17. {
18.    memset(a, 1, sizeof(a)); //初始化
19.    int k = 1;
20.    //暴力枚举
21.    for (int i = 1; i < 20000000; i++)
22.    {
23.        //判断 i 是否是谦虚数
24.        if (is_humnumber(i))
25.            a[k++] = i;
26.    }
27.    int q;
28.    cin >> q; //读入 q 组询问
29.    for (int j = 1; j <= q; j++)
30.    {
31.        int idx;
32.        cin >> idx; //读入待查询的数
```

```

33.         cout << a[idx] << endl; //查表输出
34.     }
35.     return 0;
36. }

```

2.17 贴瓷砖^{NQ067}

描述

校区建设如火如荼，看着已经建好的墙，小栋需要算出一共有多少种贴瓷砖的方法，可以保证正方形的瓷砖被完全填充。

然后从其中选出纹路最美的贴法。

瓷砖是 2×1 的长方形：



墙壁是 $3 \times n$ 的大小



可以按照此法填充整个墙壁。

请问一共有几种方式可以用 2×1 的砖块平铺 $3 \times n$ 墙面？

输入

一次输入可能包含多行，每一行分别给出不同的 n 值（即 3 乘 n 墙面的列数）。

当输入 -1 的时候结束。 n 的值最大不超过 30 。

输出

针对每一行的 n 值，输出 3 乘 n 墙面的不同的完全覆盖的总方案数。

输入样例

```

2
8
12
-1

```


输出样例

```
3
153
2131
```

参考代码(C++)

```
1. // NQ067 贴瓷砖
2. #include <iostream>
3. #include <cstring>
4. using namespace std;
5. int n;
6. //wall[i],当 i 为奇数时没有完全平铺的方案
7. int wall[31]; //wall[i]表示 3 行 i 列的墙一共有多少完全平铺数, wall[1]==0,
8. int main()
9. {
10.     //清零
11.     memset(wall, 0, sizeof wall);
12.     //边界值
13.     wall[0] = 1; wall[2] = 3;
14.     //注意递归步长和范围
15.     for (int i = 4; i <= 30; i += 2) {
16.         wall[i] = 4 * wall[i - 2] - wall[i - 4];
17.     }
18.     //查表得答案
19.     while (cin >> n, n != -1) {
20.         cout << wall[n] << endl;
21.     }
22.     return 0;
23. }
```

2.18 求方程的根^{NQ068}

描述

看着小鲁的编程水平再次进阶，小华很欣慰，是时候教导他基本算法的时候了。

小华喊小鲁过来，对他说：今天要教你一招新招，二分法。这种分而治之的思想是算法中常见的思维方式，也是优化算法必须要掌握的利器。二分法的用途很广。我出道题，你想想要怎么用二分法做。

正说着，小华在纸上写下了一个方程：

$$f(x) = x^3 - 5x^2 + 10x - 80 = 0$$

请用二分法求方程的根，精确到小数点后9位。

输入

没有输入

输出

5.705085930

输入样例

无

输出样例

5.705085930

参考代码(C++)

```
1. // NQ068 求方程的根
2. #include <cmath>
3. #include <iostream>
4. using namespace std;
5. #define EPS 1e-9
6. // f(0)=-80; f(10)=1000-500+100-80=520
7. double f(double x) { return x * x * x - 5 * x * x + 10 * x - 80; }
8. double binarySearch(double left, double right) {
9.     double mid;
10.    while (right - left > EPS) {
11.        mid = left + (right - left) / 2;
12.        if (f(mid) > 0) //根在 mid 的左边
13.            right = mid;
14.        else
15.            left = mid + EPS;
16.    }
17.    return mid;
18. }
19. int main() {
20.    // left=0.0 right =10.0
21.    printf("%.9lf\n", binarySearch(0.0, 10.0));
22.    return 0;
23. }
```

2.19 寻找指定和的整数对^{NQ069}

描述

小华说：小鲁，过来！给你一道老题目，你想想怎么用二分法来解：

对长度为 n 的数组，请用二分算法查找是否其中是否有一对数的和等于给定的数。

小鲁读了两遍题目，认出这是他已经做过的两数之和问题，然而要怎样用二分来解决呢？

看着百思不得其解的小鲁，小华忍不住提示他：对于元素 a ，只要用二分查找 $target-a$ 的值就可以啦！

小鲁恍然大悟，赶紧开始奔去写代码了！

看着听懂提示后，奔去编码的小鲁同学，小华会心的笑了：树木需要十年，而树人需要百年，这话是真的，唯有恒久忍耐，持续付出的爱才能真正的树人。

输入

共三行：

第一行是整数 n ($0 < n \leq 100,000$)，表示有 n 个整数。

第二行是 n 个整数。整数的范围是在0到 10^8 之间。

第三行是一个整数 m ($0 \leq m \leq 2^{30}$)，表示需要得到的和。

输出

若存在和为 m 的数对，输出两个整数，小的在前，大的在后，中间用单个空格隔开。

若有多个数对满足条件，选择数对中较小的数更小的。若找不到符合要求的数对，输出一行No。

输入样例

```
4
2 5 1 4
6
```

输出样例

```
1 5
```

参考代码(C++)

```
1. // NQ069 寻找指定和的整数对
2. #include <bits/stdc++.h> //万能头文件
3. #define For(a, b, c) for (register int a = b; a <= c; ++a)
4. using namespace std;
5. int N, q[110000], M;
6. int main() {
7.     scanf("%d", &N);
8.     For(a, 1, N) scanf("%d", &q[a]);
9.     scanf("%d", &M);
10.    sort(q + 1, q + N + 1);
11.    For(a, 1, N) {
12.        int k = lower_bound(q + a + 1, q + N + 1, M - q[a]) - q; //直接调用 lower_bound
13.        if (k != N + 1 && q[k] == M - q[a]) return !printf("%d %d", q[a], M - q[a]);
14.    }
```

```
15. printf("No");
16. return 0;
17. }
```

2.20 寻找最接近的元素^{NQ070}

描述

看着掌握二分算法的小鲁，小华知道现在是帮助他更上一层楼的时候了。

他告诉小鲁，二分算法早就是C++的STL库函数的了，只不过函数名字不是binarySearch乃是lower_bound和upper_bound。

如今是熟练运用二分函数来解决难题的时候了，请听题：

在一个非降序列中，查找与给定值最接近的元素。

输入

第一行包含一个整数n，为非降序列长度。 $1 \leq n \leq 100000$ 。

第二行包含n个整数，为非降序列各元素。所有元素的大小均在0-1,000,000之间。

第三行包含一个整数m，为要询问的给定值个数。 $1 \leq m \leq 10000$ 。

接下来m行，每行一个整数，为要询问最接近元素的给定值。所有给定值的大小均在0-1,000,000之间。

(注意：最接近但是不能相等。输入保证有解)

输出

m行，每行一个整数，为最接近相应给定值的元素值，保持输入顺序。

若有多个值满足条件，输出最小的一个。

(注意：最接近但是不能相等。输入保证有解)

输入样例

```
3
2 5 8
2
10
5
```

输出样例

```
8
2
```

参考代码1(C++)

```
1. //NQ070 寻找最接近的数(例程 1)
2. #include <iostream>
```

```

3. #include <cmath>
4. #include <algorithm>
5. using namespace std;
6. long long *numbers;
7. //搜索区间[0,n),左闭右开,升序
8. int lowerBound(long long nums[], int n, long long target) {
9.     int left = 0, right = n;
10.    int result=-1; //表示没找到
11.    while(left<right) { //如果 left==right 例如 [10,10)循环退出
12.        int mid = left + (right - left) / 2;
13.        if (nums[mid] >= target) { //mid 右边的所有数都大于 target
14.            right = mid; //右边是开区间, 查找范围为[left,mid)
15.        } else
16.        { // nums[mid]< target
17.            result = mid; //记录这此的结果, 缩小查找区间
18.            left=mid+1;
19.        }
20.    }
21.    if(result==-1&&nums[0]<target) result=0; //判断临界点的值
22.    return result;
23. }
24. //搜索区间[0,n),左闭右开,升序,寻找 target 的右边界
25. int upperBound(long long nums[], int n, long long target) {
26.    int left = 0, right = n;
27.    int result=-1; //表示没找到
28.    while(left<right) { //如果 left==right 例如 [10,10)循环退出
29.        int mid = left + (right - left) / 2; //防止溢出
30.        if (nums[mid] <= target) { //mid 左边的所有数都小于 target
31.            left = mid+1; //在 mid 右侧搜索, 去掉 mid, 查找范围为[left,mid)
32.        } else
33.        { // nums[mid]> target
34.            result=mid;
35.            right=mid;
36.        }
37.    }
38.    if(result==-1&&nums[n-1]>target) result=n-1; //判断右边界临界值
39.    return result;
40. }
41. int main()
42. {
43.    int n,m,left,right;
44.    long long target,foundNumber;
45.    //创建动态数组
46.    cin>>n;

```

```

47.     numbers = new long long[n] ;
48.     for (int i=0;i<n;i++)
49.         cin>>numbers[i];
50.     //寻找 LowerBound
51.     cin>>m; //读入 m 组查询的数
52.     while(m--){
53.         cin>>target;
54.         //输入数据在范围之外，不需要二分查找
55.         if (target<numbers[0]) {
56.             cout<<numbers[0]<<endl;
57.             continue;
58.         }
59.         if (target>numbers[n-1]){
60.             cout<<numbers[n-1]<<endl;
61.             continue;
62.         }
63.         //二分查找
64.         left=lowerBound(numbers,n,target);//寻找 target 的左边界
65.         right=upperBound(numbers,n,target);//寻找 target 的右边界
66.
67.         if (left == -1) {//没有找到左边界
68.             cout<<numbers[right]<<endl; //最近值在 right 的位置
69.             continue;
70.         }
71.         if (right == -1){
72.             cout<<numbers[left]<<endl;//最近值在 left 的位置
73.             continue;
74.         }
75.         foundNumber = target-numbers[left]>numbers[right]-
            target ? numbers[right]: numbers[left];
76.         cout<<foundNumber<<endl;
77.     }
78.     delete numbers;
79.     return 0;
80. }

```

参考代码2(C++)

```

1. //NQ070 寻找最接近的数(例程 2)
2. #include <bits/stdc++.h>
3. using namespace std;
4. #define For(a, b, c) for (register int a = b; a <= c; ++a)
5. const int NUM = 10;
6. int N, g[110000], M, t, l, r;
7. int main() {

```

```

8.  scanf("%d", &N);
9.  g[0] = -1E9;
10. g[N + 1] = 1E9;
11. For(a, 1, N) scanf("%d", &g[a]);
12. scanf("%d", &M);
13. while (M--) {
14.     scanf("%d", &t);
15.     l = lower_bound(g + 1, g + N + 1, t) - g;
16.     r = upper_bound(g + 1, g + N + 1, t) - g;
17.     if (l <= N) l--;
18.
19.     if (l == N + 1)
20.         printf("%d\n", g[N]);
21.     else if (r == 1)
22.         printf("%d\n", g[1]);
23.     else
24.         printf("%d\n", t - g[l] > g[r] - t ? g[r] : g[l]);
25. }
26. return 0;
27. }

```

2.21 手撕农夫与牛的问题^{NQ071}

描述

完成三道二分题之后，小华借着对小鲁说：调用stl的函数解题固然爽，但是真正的高手在很多时候需要更加灵活的运用二分算法的思想来解决实际问题，而这些非通用的问题没法靠直接调用stl解决。

你来试试一道著名的OJ题：农夫与牛。

农夫John建造了一座很长的畜栏，它包括 N ($2 \leq N \leq 100,000$) 个隔间，这些小隔间的位置为 $x_0, \dots, x_{N-1}$ ($0 \leq x_i \leq 1,000,000,000$, 均为整数, 各不相同)。

John的 C ($2 \leq C \leq N$) 头牛每头分到一个隔间。牛都希望互相离得远点省得互相打扰。

怎样才能使任意两头牛之间的最小距离尽可能的大，这个最大的最小距离是多少呢？

这道题的可以用二分来解决，然而判定函数需要根据题意自定义函数。

看着似懂非懂的小鲁，小华继续鼓励他说，你试试先把题写完就知道我的意思了！

输入

第1行：两个由空格分隔的整数 N, C

第2.. $N + 1$ 行：第 $i + 1$ 行包含整数隔间的位置 x_i

输出

第1行：一个整数：最大最小距离

输入样例

```
5 3
1
2
8
4
9
```

输出样例

```
3
```

参考代码1(C++)

```
1. //NQ071 手撕农夫与牛问题(例程 1)
2. #include <iostream>
3. #include <algorithm>
4. using namespace std;
5. int N,Stalls[100005],C;
6. bool isContainAllcows(int distance)
7. {
8.     int count =1,tempStall=Stalls[0];
9.     for (int i=1;i<N;i++)
10.    { //判断当前隔间是否可以放牛
11.        if(Stalls[i]-tempStall>distance)
12.        { //把 1 头牛逐个放入当前畜栏的隔间
13.            count++;
14.            tempStall=Stalls[i];
15.            //判断是否 C 头牛都放完
16.            if (count>=C)
17.                return true;
18.        }
19.    }
20.    //循环结束, count 技术还不到 C,表明有剩下牛
21.    return false;
22. }
23. //搜索区间[0,n),左闭右闭,升序
24. int binarySearch(int nums[], int n) {
25.     int left = nums[0], right = nums[n-1]-nums[0]; //查找空间
26.     while(left<right) { //如果 left==right 例如 [10,10)循环退出
27.         int mid = left + (right - left) / 2;
28.         //mid 为要测试的 Distance 值
29.         if (isContainAllcows(mid))
30.             left=mid+1; //在右边区间找更大的距离值
31.         else
```



```

32.         right=mid;    //在左边区间找可行的距离值
33.     }
34.     return left;
35. }
36. int main()
37. {
38.     scanf("%d%d",&N,&C);
39.     for(int i=0;i<N;i++) scanf("%d",&Stalls[i]);
40.     sort(Stalls,Stalls+N);
41.     int D= binarySearch(Stalls,N);
42.     printf("%d\n",D);
43.     return 0;
44. }

```

参考代码2(C++)

```

1. //NQ071 手撕农夫与牛问题(例程 2)
2. #include <bits/stdc++.h>
3. #define For(a, b, c) for (register int a = b; a <= c; ++a)
4. using namespace std;
5. int N, M, tot = 0, g[110000];
6. bool judge(int t) {
7.     int k = M, left = 0;
8.     For(a, 1, N) {
9.         left += g[a];
10.        if (t <= left) {
11.            k--;
12.            if (k == 0) return 1;
13.            left = 0;
14.        }
15.    }
16.    return 0;
17. }

18. int main() {
19.     cin >> N >> M;
20.     For(a, 1, N) cin >> g[a];
21.     sort(g + 1, g + 1 + N);
22.     int l = 1, r = g[N];
23.     M--;
24.     N--;
25.     For(a, 1, N) g[a] = g[a + 1] - g[a];
26.     while (l + 1 < r) {
27.         int mid = (l + r) / 2;
28.         if (judge(mid))

```

```

29.     l = mid;
30.     else
31.         r = mid;
32.     }
33.     printf("%d", l);
34.     return 0;
35. }

```

2.22 N皇后编程问题^{NQ072}

描述

随着编程水平的进阶，小华知道是全面扩充小鲁知识面，帮助他快速升级的时候到了，他再给小鲁布置了道经典题：

输入一个正整数N，请下一个程序，输出N皇后问题的全部摆法。

输入

输入皇后的个数n ($n \leq 13$)

输出

输出长度为n的正整数。

输出结果里的每一行都代表一种摆法。

行里的第i个数字如果是n，就代表第i行的皇后应该放在第n列。

皇后的行、列编号都是从1开始算。

输入样例

4

输出样例

2413
3142

参考代码(C++)

```

1. #include <iostream>
2. #include <vector>
3. using namespace std;
4. int N; //读入的皇后个数
5. // res 存储皇后的一组解,res[0]存储皇后第 0 行的位置
6. // res[i-1]存储第 i 皇后的位置
7. vector<int> res;
8.
9. void dfs(int n) {

```

```

10. //递归出口, n 从 0 开始到 N-1 全部遍历完毕
11. if (n == N) {
12.     for (auto x : res) cout << x;
13.     cout << endl;
14.     return;
15. }
16. //假定 n-1 个皇后已经全部摆好, 现在准备摆第 n 个皇后
17. //意味着 res[0]--res[n-1]已经有值了
18. for (int i = 1; i <= N; i++) //尝试 0-N 列位置, 摆放第 n 个皇后
19. {
20.     int k;
21.     for (k = 0; k < n; k++)
22.         if ((res[k] == i) || //发现同列, 跳出
23.             abs(res[k] - i) == abs(n - k)) //对角线, 跳出
24.             break;
25.     if (k == n) {
26.         res[n] = i; //把当前第 n 个皇后放在第 i 个位置上
27.         dfs(n + 1);
28.     }
29. }
30. }
31. int main() {
32.     //全局变量 N 皇后
33.     cin >> N;
34.     res.resize(N);
35.     dfs(0);
36.     return 0;
37. }

```

2.23 递归求波兰表达式^{NQ073}

描述

站在巨人的肩膀上, 编程之路才能越走越宽。小华继续向小鲁讲授新的知识点: 波兰表达式。

逆波兰表达式, 英文为 Reverse Polish notation, 跟波兰表达式 (Polish notation) 相对应。

之所以叫波兰表达式和逆波兰表达式, 是为了纪念波兰的数理科学家 Jan Łukasiewicz 的创意。

平时我们习惯将表达式写成 $(1 + 2) * (3 + 4)$, 加减乘除等运算符写在中间, 因此称为中缀表达式。

而波兰表达式的写法为 $(* (+ 1 2) (+ 3 4))$, 将运算符写在前面, 因而也称为前缀表达式。

逆波兰表达式的写法为 $((1 2 +) (3 4 +) *)$, 将运算符写在后面, 因而也称为后缀表达式。

波兰表达式和逆波兰表达式有个好处, 就算将圆括号去掉也没有歧义。

上述的波兰表达式去掉圆括号, 变为 $* + 1 2 + 3 4$ 。逆波兰表达式去掉圆括号, 变成 $1 2 + 3 4$

+ *也是无歧义并可以计算的。事实上我们通常说的波兰表达式和逆波兰表达式就是去掉圆括号的。
而中缀表达式，假如去掉圆括号，将 $(1 + 2)(3 + 4)$ 写成 $1 + 23 + 4$ ，就改变原来意思了。
 $(2 + 3) * 4$ 的波兰表示法为 $* + 2 3 4$
请写程序求解波兰表达式的值。

注意：本题输入的运算符只包括如下4个运算符：+ - * /

提示：可使用 `atof(str)` 把字符串转换为一个 `double` 类型的浮点数，方便求解。

输入

输入为一行波兰表达式，其中运算符和运算数之间都用空格分隔，运算数是浮点数。

输出

输出为一行，表达式的值。

可直接用 `printf("%f\n", v)` 输出表达式的值 `v`。

输入样例

```
* + 11.0 12.0 + 24.0 35.0
```

输出样例

```
1357.000000
```

参考代码(C++)

```
1. // NQ073 递归求波兰表达式
2. #include <iostream>
3. #include <cmath>
4. using namespace std;
5. double PolishNotation()
6. {
7.     char str[32];
8.     cin>>str;
9.     switch (str[0])
10.    {
11.        case '+': return PolishNotation() + PolishNotation();
12.        break;
13.        case '-': return PolishNotation() - PolishNotation();
14.        break;
15.        case '*': return PolishNotation() * PolishNotation();
16.        break;
17.        case '/': return PolishNotation() / PolishNotation();
18.        default:
19.            return atof(str);
20.        break;
21.    }
```

```

22. }
23. int main() {
24.     printf("%lf", PolishNotation());
25.     return 0;
26. }

```

2.24 求八皇后的第n种解^{NQ074}

描述

掌握基本的递归函数也明白如何生成N皇后问题后，在小华的指导下，小鲁决定加大难度，下一步挑战的就是n皇后问题的升级版：

国际象棋中的皇后可以在横、竖、斜线上不限步数地吃掉其他棋子。

八个皇后问题是如何将八个皇后放在棋盘上（有8 * 8个方格），使它们谁也不能被其他皇后吃掉！

已经知道八皇后问题一共有92组解，每组解可以用一个字符串表示：

对于某个满足要求的八皇后的摆放方法，定义一个皇后串a与之对应，即a=b1b2...b8，其中bi为相应摆法中第i行皇后所处的列数。

题目是：

输入一个数n，要求输出八皇后问题的第n个解，也就是第n个皇后字符串。

串的比较是这样的：皇后串x置于皇后串y之前，当且仅当将x视为整数时比y小。

请你写一个程序，能够根据输入的数n（1≤n≤92），输出第n个合法的八皇后串。

输入

第1行是测试数据的组数T，后面跟着T行输入。

每组测试数据占1行，包含一个正整数n（1 ≤ n ≤ 92）

输出

输出有T行，每行输出对应一个输入。输出应是一个正整数，是第n个八皇后串。

输入样例

```

2
1
92

```

输出样例

```

15863724
84136275

```

参考代码(C++)

```

1. // NQ074 求八皇后的第 n 种解
2. #include <cstring>

```

```

3. #include <iostream>
4. using namespace std;
5. //八皇后的解空间数组，每一行是一种皇后的摆法
6. int queenSpace[92][8];
7. // 存储皇后的一组解,res[0]存储皇后第 0 行的位置，res[i]存储皇后第 i 行的位置
8. int res[8];
9. int cnt = 0; //可行解的计数器
10. void dfs(int n) {
11.     //递归出口，n 从 0 开始到 7 全部遍历完毕
12.     if (n > 7) {
13.         for (int k = 0; k < 8; k++) queenSpace[cnt][k] = res[k];
14.         cnt++;
15.         return;
16.     }
17.     //假定 n-1 个皇后已经全部摆好，现在准备摆第 n 个皇后
18.     //意味着 solution[0]--solution[n-1]已经有值了
19.     for (int i = 1; i <= 8; i++) //尝试 0-7 列位置，摆放第 n 个皇后
20.     {
21.         int k;
22.         for (k = 0; k < n; k++)
23.             if ((res[k] == i) || //发现同列，跳出
24.                 abs(res[k] - i) == abs(n - k)) //对角线，跳出
25.                 break;
26.         if (k == n) {
27.             res[n] = i; //把当前第 n 个皇后放在第 i 个位置上
28.             dfs(n + 1);
29.         }
30.     }
31. }
32. int main() {
33.     int T, n;
34.     memset(res, 0, sizeof(res));
35.     //打表法，计算 92 组解并排序
36.     cnt = 0;
37.     dfs(0); //深度优先搜索
38.     // T 组测试数据
39.     cin >> T;
40.     while (T--) {
41.         cin >> n;
42.         for (int i = 0; i < 8; i++) cout << queenSpace[n - 1][i];
43.         cout << endl;
44.     }
45.     return 0;

```

2.25 P进制^{NQ075}

描述

小华知道现在计算机入门普遍教的是二进制，八进制和十六进制。对数学家来说，七进制，十一进制，从逻辑角度来说是一样的。

为了帮助小鲁明白什么是抽象，他让小鲁写个p进制的程序，把十进制转换为p进制。

十进制与P进制的字符的映射如图：

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35
0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z

用来表示数字的字符是数字字符0-9以及大写的英文字符A-Z，因此p进制最多支持 $10+26=36$ 个不同的字符来表示字符，最高支持36进制。

请写一个算法把十进制数n转换为p进制数输出 ($2 \leq p \leq 36$)。

输入

输入数据包含若干行，每行是两个整数n, p, 其中 $2 \leq p \leq 36$, 表示要把十进制n转换为p进制。

输出

输出数据包含若干行，每行是一个n转换为的p进制数。

输入样例

```
127 2
127 16
127 10
127 11
127 20
127 32
127 36
127 27
127 4
127 8
-127 16
-127 8
-127 32
-127 35
```

输出样例

```
1111111
7F
127
```

```
106
67
3V
3J
4J
1333
177
-7F
-177
-3V
-3M
```

参考代码(C++)

```
1. // NQ075 P 进制
2. #include <iostream>
3. #include <algorithm>
4. using namespace std;
5. const int N = 1007;           //P 进制位最多 N 位
6. char a[N], P[36];           //P 进制转换结果,P 进制字符映射表, 最高 36 进制
7. inline char f(int n) { return P[n % 36]; } //把数字转换为字符
8. int main()
9. {
10.     //打表法
11.     for (int i = 0; i < 36; i++)
12.         P[i] = (i < 10) ? char(i + '0') : char(i - 10 + 'A');
13.     int n, p;
14.     //完成 p 进制的转换
15.     while (cin >> n >> p)
16.     {
17.         if (p > 36 || p <= 1) continue; //越界忽略
18.         int minus = 0;
19.         if (n < 0)
20.             minus = -1, n = -1 * n;
21.         int k = 0;
22.         while (n) {
23.             a[k] = f(n % p); //转换位相应位数
24.             k++;
25.             n /= p;
26.         }
27.         if (minus < 0) cout << "-";
28.         while (--k >= 0) cout << a[k];
29.         cout << endl;
30.     }
31.     return 0;
```


2.26 二进制取幂法^{NQ076}

描述

经过不懈的努力，小鲁终于掌握了各种编程的语法，原本对编程缺乏信心，如今他信心满满准备迎接更妙的算法挑战。

小华看到零基础的文化人小鲁，能够有如此进步，非常欣慰，于是给他出了个新的问题，求整数 32736600 的 25026484 次方 mod 61007538 的值。

小鲁一看数据的大小，就知道不能直接暴力计算，他淡定的向小华作揖，谦卑的说，这道题，一定有妙招，烦请指点迷津。

小华挥一挥手中的白羽扇，微笑着说：

你可以试试二进制取幂 (Binary Exponentiation)，俗称快速幂！

问题：输入三个数 a k p ，请计算 $a^k \bmod p$ 的值。

数据范围

$1 \leq a, k, p \leq 1000000000$

输入

第一行是整数 q ，表示接下来有 q 组数据

接下来 q 行，每行包含三个整数 a k p 。

输出

对于每组数据，请计算 $a^k \bmod p$ 的值，每组数据输出一行。

输入样例

```
5
5986 3509 6209
9321 8755 80
4314 9373 1100
4016 3748 8906
4860 3379 1172
```

输出样例

```
3585
41
844
4600
580
```

参考代码(C++)

```
1. // NQ076 二进制取幂法
2. #include <iostream>
3. #include <algorithm>
4. using namespace std;
5. typedef long long LL;
6. int qmi(int a, int k, int p)
7. {
8.     int res = 1; //初始值为1
9.     while (k)
10.    {
11.        if (k & 1) //b的最末尾为1
12.            res = (LL)res * a % p; //必须用 long long 防止溢出
13.        k >>= 1; //去掉 b 最低位
14.        a = (LL)a * a % p;
15.    }
16.    return res;
17. }
18. int main()
19. {
20.     int q;
21.     scanf("%d", &q); //数据很大, 需要用 LL 读入
22.     while (q--)
23.     {
24.         int a, k, p;
25.         scanf("%d%d%d", &a, &k, &p); //读入 a b p
26.         printf("%d\n", qmi(a, k, p)); //计算快速幂
27.     }
28.     return 0;
29. }
```

2.27 求第K大的正整数^{NQ077}

描述

为了让自己在排列组合方面更加娴熟, 小鲁自己给自己出了一道简单题, 尝试将自己所学的排序, 寻找指定元素的技巧结合起来练练。

题目如下:

给定N个整数, $x_1, x_2 \dots x_n$, 任取两个不同的整数计算其差的绝对值 $|x_i - x_j|$, 其中 $(0 < i, j \leq N)$ 并且 i 与 j 不相等)。

现在请你计算第K大的绝对值是多少 (第K大是指有K-1个不同的绝对值比它大, 它排行老K)

输入

输入数据首先包含一个正整数 q ，表示包含 q 组测试用例。

每组测试数据的第一行包含两个整数 N ， K 。

接下去一行包含 N 个整数。

数据范围：

```
1 < q ≤ 100,  
1 < N ≤ 1000,  
0 < K ≤ N,  
1 < 输入整数 < 100000
```

输出

对于每组测试数据，请输出第 K 大的组合数，每个输出实例占一行。

输入样例

```
3  
7 4  
89 11 33 98 12 62 65  
5 2  
1 2 3 4 5  
2 1  
2 7
```

输出样例

```
77  
3  
5
```

参考代码(C++)

```
1. // NQ077 求第 K 大的正整数  
2. #include <iostream>  
3. #include <algorithm>  
4. using namespace std;  
5. typedef long long LL;  
6. int qmi(int a, int k, int p)  
7. {  
8.     int res = 1; //初始值为 1  
9.     while (k)  
10.    {  
11.        if (k & 1) //b 的最末尾为 1  
12.            res = (LL)res * a % p; //必须用 long long 防止溢出  
13.        k >>= 1; //去掉 b 最低位
```

```

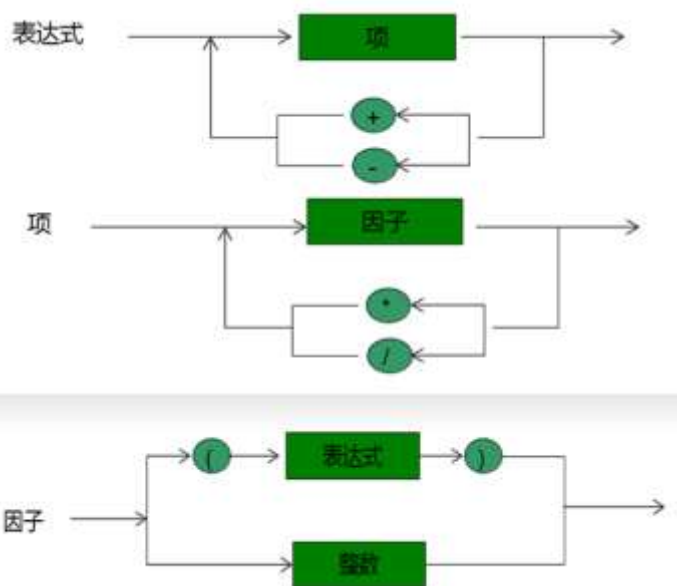
14.         a = (LL)a * a % p;
15.     }
16.     return res;
17. }
18. int main()
19. {
20.     int q;
21.     scanf("%d", &q); //数据很大，需要用 LL 读入
22.     while (q-->0)
23.     {
24.         int a, k, p;
25.         scanf("%d%d%d", &a, &k, &p); //读入 a b p
26.         printf("%d\n", qmi(a, k, p)); //计算快速幂
27.     }
28.     return 0;
29. }

```

2.28 四则运算表达式求值^{NQ078}

描述

在掌握递归函数和初步接触深度优先搜索的技术之后，小华知道又到了帮助小鲁再进一步的时候了。使用状态机来分析函数的递归，简单直观，这是成为编程高手的必备技能。题目看起来很简单：求一个可以带括号的小学算术四则运算表达式的值。表达式、项、因子、与输入的+ - * / 以及数字的关系可用状态机来表示：



请根据如上的状态机，写一个可以计算简单输入表达式的程序。

例如：

- 输入表达式3.4 输出3.40
- 输入表达式:7+8.3输出:15.30

输入

一行，一个四则运算表达式。 '*' 表示乘法， '/' 表示除法

输出

一行，该表达式的值，保留小数点后面两位

输入样例

```
3+4.5*(7+2)*(3)*((3+4)*(2+3.5)/(4+5))-34*(7-(2+3))
```

输出样例

```
454.75
```

参考代码(C++)

```
1. // NQ078 表达式求值
2. #include <cstdlib>
3. #include <cstring>
4. #include <iostream>
5. using namespace std;
6. int factor_value();      //递归函数读入并处理因子
7. int term_value();       //递归函数读入并处理项
8. int expression_value(); //递归函数读入并处理表达式
9. int main() {
10.     cout << expression_value() << endl;
11.     return 0;
12. }
13. int expression_value() //求一个表达式的值
14. {
15.     int result = term_value(); //求第一项的值
16.     bool more = true;
17.     while (more) {
18.         char op = cin.peek(); //看一个字符,不取走
19.         if (op == '+' || op == '-') {
20.             cin.get(); //从输入中取走一个字符
21.             int value = term_value();
22.             if (op == '+')
23.                 result += value;
24.             else
25.                 result -= value;
26.         } else
27.             more = false;
```

```

28. }
29. return result;
30. }
31. int term_value() //求一个项的值
32. {
33.     int result = factor_value(); //求第一个因子的值
34.     while (true) {
35.         char op = cin.peek();
36.         if (op == '*' || op == '/') {
37.             cin.get();
38.             int value = factor_value();
39.             if (op == '*')
40.                 result *= value;
41.             else
42.                 result /= value;
43.         } else
44.             break;
45.     }
46.     return result;
47. }
48. int factor_value() //求一个因子的值
49. {
50.     int result = 0;
51.     char c = cin.peek();
52.     if (c == '(') {
53.         cin.get(); //读入左括号
54.         result = expression_value();
55.         cin.get(); //读入右括号，不做任何处理
56.     } else {
57.         while (isdigit(c)) { //按字符读入十进制数
58.             result = 10 * result + c - '0';
59.             cin.get();
60.             c = cin.peek();
61.         }
62.     }
63.     return result;
64. }

```

2.29 算24^{NQ079}

描述

熟悉了递归和深度优先搜索，小华知道现在是让小鲁综合运用所学知识的时候了，他让小鲁调整一道经典题：算24。

给出4个小于10个正整数，你可以使用加减乘除4种运算以及括号把这4个数连接起来得到一个表达式。现在的问题是，是否存在一种方式使得得到的表达式的结果等于24。

这里加减乘除以及括号的运算结果和运算的优先级跟我们平常的定义一致（这里的除法定义是实数除法）。

比如，对于5，5，5，1，我们知道 $5 * (5 - 1 / 5) = 24$ ，因此可以得到24。又比如，对于1，1，4，2，我们怎么都不能得到24。

注意：输入数字的次序可以改变。

输入

输入数据包括多行，每行给出一组测试数据，包括4个小于10个正整数。最后一组测试数据中包括4个0，表示输入的结束，这组数据不用处理。

输出

对于每一组测试数据，输出一行，如果可以得到24，输出“YES”；否则，输出“NO”。

输入样例

```
5 5 5 1
1 1 4 2
0 0 0 0
```

输出样例

```
YES
NO
```

参考代码(C++)

```
1. // NQ079 算24
2. #include <cmath>
3. #include <iostream>
4. using namespace std;
5. #define spacesize 4
6. double inputnumber[spacesize + 1];
7. #define EPS 1e-6
8. bool isZero(double x) { return fabs(x) <= EPS; }
9. bool count24(double a[], int n) //用数组里a的n个数计算24
10. {
11.     //数组大小为1，这时候只需要判断a[0]是否是24即可，若不是表示这个计算序列得不到24.
12.     if (n == 1) {
13.         if (isZero(a[0] - 24))
14.             return true;
15.         else
16.             return false;
17.     }
```

```

18. // n>1 把问题规模缩小为 n-1
19. //取出两个数, 执行+、-、*、/, 生成新的实数, 原来的数中剩下 n-2 个。
20. //因为+与*满足交换律, 所以共有 6 种情况需要计算
21. //每种情况都导致一个新数的诞生, 把这个新数加回到 n-2 个数中
22. //问题转换为计算 n-1 个数的算 24 点问题
23. //枚举取出两个数
24. //输入数组 a[0--n-1]
25. // a[i], a[j] 为当前取的两个数
26. for (int i = 0; i < n - 1; i++) //从 0 取到 n-2 个数
27.     for (int j = i + 1; j < n; j++) //从 i+1 开始取到 n-1
28.     {
29.         //创建 temp 数组, 存储 n-1 个要处理的数
30.         double temp[n - 1] = {0};
31.         int iTemp = 0;
32.         for (int k = 0; k < n; k++)
33.             if ((k != i) && (k != j)) temp[iTemp++] = a[k];
34.         // iTemp++ 执行了 n-2 次, 所以 temp[iTemp] 就是当前最后一个元素, 指向第 n-1 个数
35.         //分六种情况求 temp[iTemp] 的值
36.         // 1. +
37.         temp[iTemp] = a[i] + a[j];
38.         if (count24(temp, n - 1)) return true;
39.         // 2. *
40.         temp[iTemp] = a[i] * a[j];
41.         if (count24(temp, n - 1)) return true;
42.         // 3. -
43.         temp[iTemp] = a[i] - a[j];
44.         if (count24(temp, n - 1)) return true;
45.         // 4. -
46.         temp[iTemp] = a[j] - a[i];
47.         if (count24(temp, n - 1)) return true;
48.         // 5. /
49.         if (!isZero(a[j])) {
50.             temp[iTemp] = a[i] / a[j];
51.             if (count24(temp, n - 1)) return true;
52.         }
53.         // 6. /
54.         if (!isZero(a[i])) {
55.             temp[iTemp] = a[j] / a[i];
56.             if (count24(temp, n - 1)) return true;
57.         }
58.     } // for 枚举两个数
59. return false;
60. }
61. int main() {

```



```

62. while (true) {
63.     //判断是否输入是连续 n 个 0
64.     bool isEndInput = true;
65.     for (int i = 0; i < spacesize; i++) {
66.         cin >> inputnumber[i];
67.         if (!isZero(inputnumber[i])) isEndInput = false;
68.     }
69.     if (isEndInput) break;
70.     //调用 count24 计算结果
71.     if (count24(inputnumber, spacesize))
72.         cout << "YES" << endl;
73.     else
74.         cout << "NO" << endl;
75. }
76. }

```

2.30 质数环^{NQ080}

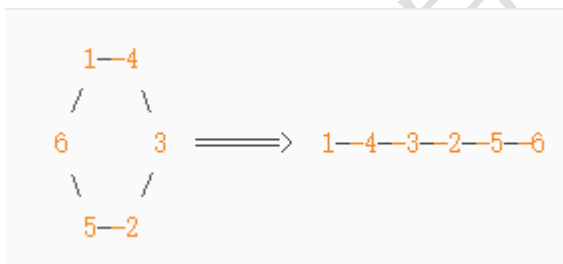
描述

看着小鲁迈进80道题大关，还差20题就可以百题斩了，小华特别高兴，他琢磨着如何帮助小鲁把所学融会贯通。

有哪些经典题包含数学知识+排列组合+深度优先搜索呢？小华灵机一动，想到了经典的质数环问题：

给一个自然数 N ，在 $1, 2, 3, 4, 5, 6 \dots N$ 个数组成的全排列中，求满足相邻两个数之和是质数的排列。

为了简化问题，排列的第一项固定是1，排列的最后一项与第一项之和必须是质数，由此成环。



输入

输入若干行，每行是一个自然数 n ($0 < n \leq 17$)。

输出

每组数据输出若干行：

每一行代表环中从1开始的满足题意的排列，按字典序输出每组排列。

每组数据的答案之间用空行隔开。

输入样例 1

```
8
6
4
```

输出样例 1

```
Case 1:
1 2 3 8 5 6 7 4
1 2 5 8 3 4 7 6
1 4 7 6 5 8 3 2
1 6 7 4 3 8 5 2

Case 2:
1 4 3 2 5 6
1 6 5 2 3 4

Case 3:
1 2 3 4
1 4 3 2
```

参考代码(C++)

```
1. //NQ080 质数环
2. #include <algorithm>
3. #include <cstring>
4. #include <iostream>
5. #define For(i, a, b) for (int i = a; i <= b; i++)
6. using namespace std;
7. bool p[100], used[100]; //默认初始化为 false
8. int n, a[100], now;
9. void dfs(int deep) {
10.    //结束条件抵达搜索个数 n, 并且头尾元素为质数
11.    if (deep == n + 1 && p[a[1] + a[n]]) {
12.        For(i, 1, n) {
13.            printf("%d", a[i]);
14.            if (i != n) printf(" "); //打印空格
15.        }
16.        puts(""); //换行
17.        return; //退出
18.    }
19.    For(i, 2, n) { //深度优先搜索
20.        if (p[i + a[deep - 1]] && !used[i]) {
21.            used[i] = true;
22.            a[deep] = i;
```

```

23.     dfs(deep + 1);
24.     used[i] = false;
25. }
26. }
27. }
28. int main() {
29.     //打表判断是否是质数, N<=20 因此只需要打表到 31 即可。
30.     p[2] = p[3] = p[5] = p[7] = p[11] = p[13] =
31.         p[17] = p[19] = p[23] = p[29] = p[31] =
32.         true; //每日打表心情好
33.     bool isf = false;
34.     while (~scanf("%d", &n)) {
35.         a[1] = 1; //质数环首项为 1
36.         if (isf) puts("");
37.         isf = true;
38.         printf("Case %d:\n", ++now);
39.         memset(used, false, sizeof(used));
40.         dfs(2); //从第 2 项开始深搜
41.     }
42.     return 0;
43. }

```

2.31 试除法求约数^{NQ081}

描述

看着越来越有悟性的小鲁，小华心里美滋滋。

他没想到一个愤世嫉俗的文人可以变成如此谦卑好学的编程爱好者。

他忍不住，又给小鲁出了一道数论题：

给定 n 个正整数 a ，对于每个整数 a ，请你按照从小到大的顺序输出它的所有约数。

数据范围

$$1 \leq n \leq 1000$$

$$2 \leq a \leq 2 \cdot 10^9$$

输入

第一行包含整数 n 。

接下来 n 行，每行包含一个整数 a 。

输出

输出共 n 行，其中第 i 行输出第 i 个整数 a 的所有约数。

输入样例

```
77439
8650
7790
8734
3660
5375
8167
```

输出样例

```
1 43 173 7439
1 2 5 10 25 50 173 346 865 1730 4325 8650
1 2 5 10 19 38 41 82 95 190 205 410 779 1558 3895 7790
1 2 11 22 397 794 4367 8734
1 2 3 4 5 6 10 12 15 20 30 60 61 122 183 244 305 366 610 732 915 1220 1830 3660
1 5 25 43 125 215 1075 5375
1 8167
```

参考代码(C++)

```
1. // NQ081 试除法求约数
2. #include <iostream>
3. #include <vector>
4. #include <algorithm>
5. using namespace std;
6.
7. //求约数，返回值存在 vector 之中
8. vector<int> get_divisors(int n)
9. {
10.     vector<int> res;
11.     for (int i = 1; i <= n / i; i++)
12.         if (n % i == 0) //是 n 的约数
13.             {
14.                 res.push_back(i); //i 如果是约数
15.                 if (i != n / i)
16.                     res.push_back(n / i); // n/i 也是约数
17.             }
18.     //排序
19.     sort(res.begin(), res.end()); //使用算法 sort 排序
20.     return res;
21. }
22. int main()
23. {
24.     int n; cin >> n; //读入 n
```

```

25.     while (n--)
26.     {
27.         int a; cin >> a;           //读入 a
28.         auto res = get_divisors(a); //使用 auto 自动判定属性
29.         for (auto i : res) cout << i << ' ';
30.         cout << endl;
31.     }
32.     return 0;
33. }

```

2.32 试除法分解质因数^{NQ082}

描述

看着小鲁渐入佳境，小华心里乐呵呵。

他灵机一动，再给小鲁出了一道数论题，帮助他巩固试除法：

给定 n 个正整数 a ，将每个数分解质因数，并按照质因数从小到大的顺序输出每个质因数的底数和指数。

数据范围：

$1 \leq n \leq 1000$

$1 \leq a \leq 2 \times 10^9$

输入

第一行包含整数 n 。

接下来 n 行，每行包含一个正整数 a 。

输出

对于每个正整数 a ，按照从小到大的顺序输出其分解质因数后，每个质因数的底数和指数，每个底数和指数占一行。

每个正整数的质因数全部输出完毕后，输出一个空行。

输入样例

```

4
3073
5620
2060
4611

```

输出样例

```

7 1
439 1

```

```

2 2
5 1
281 1

2 2
5 1
103 1

3 1
29 1
53 1

```

参考代码(C++)

```

1. // NQ082 试除法分解质因数
2. #include <algorithm>
3. #include <iostream>
4. #include <vector>
5. using namespace std;
6. void divide(int n) {
7.     for (int i = 2; i <= n / i; i++) //注意不要用 i*i<=n 避免溢出
8.         if (n % i == 0) {
9.             int nums = 0;
10.            while (n % i == 0) {
11.                n /= i;
12.                nums++;
13.            }
14.            printf("%d %d\n", i, nums); // i 出现 nums 次
15.        }
16.    // n 中只包含一个大于 sqrt(n)的质因子
17.    if (n > 1) printf("%d %d\n", n, 1); // n 是除掉所有因子后剩下的数
18.    puts(""); //换行
19.}
20. int main() {
21.     int n;
22.     cin >> n; //读入 n
23.     while (n--) {
24.         int a;
25.         cin >> a; //读入 a
26.         divide(a); //使用 auto 自动判定属性
27.     }
28.     return 0;
29.}

```

2.33 求最长公共子列长度^{NQ083}

描述

终于回到自己热爱的文字领域了，小鲁很开心的在网上找寻文字处理的难题。

对于好学的小鲁，经典题是一定要过的：

给出两个字符串，求出这样的一个**最长的公共子序列**的长度：

子序列中的每个字符都能在两个原串中找到，而且每个字符的先后顺序和原串中的先后顺序一致。

数据范围

$1 \leq N \leq 1000$

输入

输入若干行数据，每行有两个字符串，用空格隔开。

输出

对逐行输出每行数据的最大公共子序列的长度。

输入样例

```
abcfbc      abfcab
programming contest
abcd        mnp
```

输出样例

```
4
2
0
```

参考代码(C++)

```
1. // NQ083 最长公共子序列的长度
2. #include <iostream>
3. #include <cstring>
4. #include <algorithm>
5. using namespace std;
6. #define N 1007
7. //最长公共子列的状态矩阵
8. //maxLength[i][j]表示 序列 s1 的前 i 项与序列 s2 的前 j 项的最长公共子列长度
9. int maxLength[N][N];
10. string s1, s2;
11. int main()
12. {
13.     while (cin >> s1 >> s2) //直到输入为空
14.     {
```

```

15.      //初始化 maxSlength
16.      int len1 = s1.length(), len2 = s2.length();
17.      //把长度为 0 的边界设置为 0;
18.      for (int i = 0; i <= len1; i++) //maxSlength[i][0]==0
19.          maxSlength[i][0] = 0;
20.      for (int i = 0; i <= len2; i++) //maxSlength[0][i]==0
21.          maxSlength[0][i] = 0;
22.      //递推计算
23.      for (int i = 1; i <= len1; i++)
24.          for (int j = 1; j <= len2; j++)
25.              if (s1[i - 1] == s2[j - 1]) //字符串从 0 开始计数, 比较第 i, j 个字符的比较用
                  s1[i-1],s2[j-1]
26.                  maxSlength[i][j] = maxSlength[i - 1][j - 1] + 1;
27.              else
28.                  maxSlength[i][j] = max(maxSlength[i - 1][j], maxSlength[i][j - 1]);
29.      //输出最长公共子串
30.      cout << maxSlength[len1][len2] << endl; //[1, n] n 代表字符串长度, 最长是 n
31.  }
32. }

```

2.34 同构字符串^{NQ084}

描述

给定两个字符串 **s** 和 **t**, 判断它们是否是同构的。

如果 **s** 中的字符可以被替换得到 **t**, 那么这两个字符串是同构的。

所有出现的字符都必须用另一个字符替换, 同时保留字符的顺序。两个字符不能映射到同一个字符上, 但字符可以映射自己本身。

例如:

输入: **s** = "egg", **t** = "add"

输出: true 因为 e→a, g→d, 这两个单射, 使两个字符串同构

输入: **s** = "paper", **t** = "title"

输出: true 因为 p→t, a→i, e→l, r→e, 这样两个字符串同构

输入

第一行输入一个整数 **n**, 代表接下来有 **n** 行数据。

从第二行开始, 每行有 2 个长度相等的字符串 **s** 和 **t**。

输出

输出 **n** 行, 如果 **s** 与 **t** 同构, 输出 true, 否则输出 false。

输入样例

```
2
egg add
paper title
```

输出样例

```
true
true
```

参考代码(C++)

```
1. //NQ084 同构字符串
2. #include <cstring>
3. #include <iostream>
4. #include <unordered_map>
5. using namespace std;
6. bool isIsomorphic(string s, string t) {
7.     unordered_map<char, char> st, ts;
8.     for (int i = 0; i < s.size(); i++) {
9.         int a = s[i], b = t[i];
10.        if (st.count(a) && st[a] != b) return false;
11.        st[a] = b;
12.        if (ts.count(b) && ts[b] != a) return false;
13.        ts[b] = a;
14.    }
15.    return true;
16. }

17. int main() {
18.     int n;
19.     cin >> n;
20.     while (n--) {
21.         string s, t;
22.         cin >> s >> t;
23.         if (isIsomorphic(s, t))
24.             cout << "true" << endl;
25.         else
26.             cout << "false" << endl;
27.     }
28.     return 0;
29. }
```

2.35 数字三角形^{NQ085}

描述

数字三角形是动态规划的入门题，小鲁如果要入门动态规划，这道题是不可不做的：

```
7
3  8
8  1  0
2  7  4  4
4  5  2  6  5
```

(图1)

图1给出了一个数字三角形。从三角形的顶部到底部有很多条不同的路径。

对于每条路径，把路径上面的数加起来可以得到一个和，你的任务就是找到最大的和。

注意：路径上的每一步只能从一个数走到正下方和右下方的位置。

输入

输入的是一行是一个整数 N ($1 < N \leq 1000$)，给出三角形的行数。

下面的 N 行给出数字三角形。数字三角形上的数的范围都在0和100之间。

输出

输出最大的和。

输入样例

```
5
7
3 8
8 1 0
2 7 4 4
4 5 2 6 5
```

输出样例

```
30
```

参考代码(C++)

```
1. // NQ085 数字三角形
2. #include <bits/stdc++.h>
3. using namespace std;
4. const int maxN=1007;
5. int s[maxN+1][maxN+1];
6. int d[maxN+1][maxN+1]; //状态数组
7. int n; //本题的 n (<maxN)
8. int solve1(int i, int j)
```

```

9. {
10.     if (i==n) return s[i][j]; //抵达底部
11.     //当前顶点值加上最大的分支值
12.     return s[i][j]+ max(solve(i+1,j),solve(i+1,j+1));
13. }
14. int solve2(int i, int j)
15. {
16.     //该节点已经计算过
17.     if (d[i][j]>=0) return d[i][j];
18.     //当前顶点值加上最大的分支值
19.     return d[i][j]= s[i][j]+ (i==n?0:max(solve2byLiu(i+1,j),solve2byLiu(i+1,j+1)));
20. }
21. int solve3(int n)
22. { //滚动数组
23.     int* dPointer = s[n]; //指向 d 的最后一行
24.     int i,j;
25.     for (i=n-1; i>=1; i--) //使用 s[n] 行, 所以前 n-1 行遍历
26.         for(j=1; j<=i; j++)
27.             dPointer[j]=s[i][j]+max(dPointer[j],dPointer[j+1]); //从左到右计算
28.     return dPointer[1];
29. }
30. int main()
31. {
32.     //fopen("1.in","r",stdin);
33.     // memset(d,-1,sizeof(d)); //初始化状态数组 d 为-1
34.     cin>>n; //读入三角形个数
35.     for (int i=1;i<=n;i++) //读入三角形源数据
36.         for (int j=1;j<=i;j++)
37.             cin>>s[i][j];
38.     cout<<solve3(n)<<endl;
39. }

```

算法进阶

第3章 南强百题自强进阶篇

3.1 林克的01背包^{NQ086}

描述

一眨眼一学期快结束了，早就自学完大四4年计算机专业的小华拿到了任天堂公司的offer提前毕业走上他喜爱的设计之路。

放心不下还在学校苦苦自学的小鲁，小华送给小鲁一套VR装置和一个新名字——林克。

他对小鲁说：从此以后，我写的VR编程环境来继续引导你自学编程，但是在这新的虚拟世界中，你要化身为林克，接受各种编程试炼的挑战，你愿意吗？

小鲁知道随着小华的离开，自己自主学习的新时代来了，他挥泪痛别小华，欣然接过小华的赠与，带上林克头套，进入编程大陆，开始了奇妙的VR编程之旅。

他还来不及为着小华的离开伤感，第一个挑战就来临了。

“林克”面临一个艰难的选择，他的背包容量有限，而面前摆放着价值不同的物品（各种套装），你可以帮帮他吗？



有 N 件物品和一个容量是 V 的背包。每件物品只能使用一次。

第 i 件物品的体积是 v_i ，价值是 w_i 。

求解将哪些物品装入背包，可使这些物品的总体积不超过背包容量，且总价值最大。
输出最大价值。

输入

第一行两个整数， N ， V ，用空格隔开，分别表示物品数量和背包容积。

接下来有 N 行，每行两个整数 v_i, w_i ，用空格隔开，分别表示第 i 件物品的体积和价值。

$0 < N, V \leq 1000$
 $0 < v_i, w_i \leq 1000$

输出

输出一个整数，表示最大价值。

输入样例 1

```
4 5
1 2
2 4
3 4
4 5
```

输出样例 1

```
8
```

输入样例 2

```
16 24
30 34
78 40
96 97
2 38
59 57
10 48
28 65
40 17
95 84
17 5
80 83
74 32
37 28
44 36
25 14
95 29
```

输出样例 2

```
86
```

参考代码(C++)

1. // NQ086 林克的 01 背包
2. /*
3. `d[i][j]` 表示只看前 `i` 个物品，总体积是 `j` 的情况下，总价值最大的是多少

```

4. 答案:
5. result = max (d[n][0~V]) n 件物品, 体积是 0~V 的最大值
6. 递推公式:
7. d[i][j]= max (d[i-1](第 i 个物品不放)[j] , d[i-i][j-Vi] + W[i])
8. */
9. #include <algorithm>
10. #include <iostream>
11. using namespace std;
12. #define For(a, b, c) for (register int a = b; a <= c; ++a)
13. #define RFor(a, end, begin) for (register int a = end; a >= begin; a--)
14.
15. #define maxN 1007
16. int d[maxN][maxN];
17. int volumn[maxN], weight[maxN];
18. int main() {
19.     int n, m;
20.     cin >> n >> m;
21.     int* dP = d[0]; //只用一维数组
22.     For(i, 1, n) cin >> volumn[i] >> weight[i];
23.     For(i, 1, n) //枚举所有物品
24.         RFor(j, m, volumn[i]) //枚举体积
25.             {
26.                 dP[j] = max(dP[j], dP[j - volumn[i]] + weight[i]);
27.             }
28.     cout << dP[m] << endl; ///答案就是 dP[m]
29.     return 0;
30. }

```

题解 (微信扫码观看)



3.2 踩盾滑行^{NQ087}

描述



踩盾滑行是林克的最爱，作为滑行爱好者，只有高度差的斜坡，无论是草地、雪地、沙地还是空气，林克都可以踩盾滑行。

给定一个有高度差的区域(用二维数组表示)数组的每个数字代表点的高度(如下图)，如何求出最长的滑行路径呢？

```
1  2  3  4  5
16 17 18 19 6
15 24 25 20 7
14 23 22 21 8
13 12 11 10 9
```

提示：林克可以从某个点滑向上下左右相邻四个点之一，当且仅当高度减小。

在上面的例子中，最长的滑行路径为25->24->23->22->21->20.....->5->4->3->2->1

(你看出来了吗?)

输入

输入的第一行表示区域的行数R和列数C ($1 \leq R, C \leq 100$)。

下面是R行，每行有C个整数，代表高度h， $0 \leq h \leq 10000$ 。

输出

输出最长区域的长度。

输入样例

```
5 5
1  2  3  4  5
16 17 18 19 6
15 24 25 20 7
14 23 22 21 8
```

```
13 12 11 10 9
```

输出样例

```
25
```

参考代码1(C++)

```
1. //NQ087 踩盾滑行
2. #include <iostream>
3. #include <cstring>
4. #include <algorithm>
5. using namespace std;
6. #define N 1007
7. int d[N][N];
8. int h[N][N]; //高度矩阵
9. int R,C;
10. int dfs(int i, int j)
11. {
12.     if (d[i][j]!=-1) return d[i][j]; //已经访问过这个点，直接返回
13.     //边界
14.     if ((h[i][j]<=h[i][j-1])&& //左
15.         (h[i][j]<=h[i-1][j])&& //上
16.         (h[i][j]<=h[i][j+1])&& //右
17.         (h[i][j]<=h[i+1][j])) //下
18.         return d[i][j]=1; //最低点，返回

19.     //递推计算
20.     int maxLen=0;
21.     //左
22.     if (h[i][j]>h[i][j-1]) {
23.         int left = dfs(i,j-1); //左边可以下滑
24.         if (maxLen < left+1) maxLen = left+1; //存储最大值
25.     }
26.     //上
27.     if (h[i][j]>h[i-1][j]){
28.         int up = dfs(i-1,j);
29.         if (maxLen < up+1) maxLen = up+1; //存储最大值
30.     }
31.     //右
32.     if (h[i][j]>h[i][j+1]){
33.         int right = dfs(i,j+1);
34.         if (maxLen < right+1) maxLen = right+1; //存储最大值
35.     }
36.     //下
37.     if (h[i][j]>h[i+1][j]){
```



```

38.     int down = dfs(i+1,j);
39.     if (maxLen < down+1) maxLen = down+1; //存储最大值
40. }
41. return d[i][j]=maxLen;
42. }
43. int main()
44. {
45.     cin>>R>>C;
46.     memset(h,0x3f,sizeof(h)); //把区域初始化为无穷大
47.     memset(d,-1,sizeof(d)); //把状态初始化为-1
48.     //读入高度值
49.     for(int a=1;a<=R;a++)
50.         for(int b=1;b<=C;b++)
51.             cin>>h[a][b];
52.     int ans=-1;
53.     for(int a=1;a<=R;a++)
54.         for(int b=1;b<=C;b++)
55.         {
56.             int res = dfs(a,b);
57.             if (ans<res) ans=res;
58.         }
59.     cout<<ans;
60.     return 0;
61. }

```

参考代码2(C++)

```

1. // Code by witcano
2. #include<bits/stdc++.h>
3. using namespace std;
4. int R,C,m[102][102],f[102][102],ans;
5. void CMax(int &k,int a) {if(k<a) k=a;}
6. int dfs(int r,int c)
7. {
8.     if(f[r][c]) return f[r][c];
9.     if(m[r+1][c]<m[r][c]) CMax(f[r][c],1+dfs(r+1,c));
10.    if(m[r-1][c]<m[r][c]) CMax(f[r][c],1+dfs(r-1,c));
11.    if(m[r][c+1]<m[r][c]) CMax(f[r][c],1+dfs(r,c+1));
12.    if(m[r][c-1]<m[r][c]) CMax(f[r][c],1+dfs(r,c-1));
13.    CMax(f[r][c],1);
14.    return f[r][c];
15. }
16. int main()
17. {
18.     cin>>R>>C;

```

```

19.     memset(m,0x3f,sizeof(m));
20.     for(int a=1;a<=R;a++) for(int b=1;b<=C;b++) cin>>m[a][b];
21.     for(int a=1;a<=R;a++) for(int b=1;b<=C;b++) CMax(ans,dfs(a,b));
22.     cout<<ans;
23.     return 0;
24. }

```

3.3 拦截雷电箭(1) NQ088

描述



莱尼尔是海鲁拉大地之中最强的怪兽之一，无论近战还是远程攻击，莱尼尔都近乎完美。

莱尼尔最令人闻风丧胆的远程攻击是箭雨，这是一种能够动态跟踪对手所在位置的空对地攻击。无论距离多远，只要箭雨飞出，打击必达。

为了能够完美抵御莱尼尔的箭雨，约珥开发了一种箭雨拦截系统，可以发射出若干海鲁拉盾，在空中拦截莱尼而的箭雨。于此同时，整个拦截系统可以透过ambo的方式传送给林克。

但是每套箭雨拦截系统有一个缺陷：第一面地对空的发射的海鲁拉盾可以到达任意的高度，但是以后每一面海鲁拉盾都不能高于前一面的高度。

在实际战斗中，已知莱尼尔射出的箭雨阵中每一支雷电箭依次飞来的高度（高度数据是 ≤ 50000 的正整数）。

请计算每套箭雨拦截系统最多能拦截多少支雷电箭？

输入

输入有两行，

第一行，莱尼尔射出的箭雨的数量 k ($k \leq 10000$)，

第二行， k 个正整数，表示 k 支雷电箭的高度，按发射顺序给出，以空格分隔。（高度Height ≤ 50000 ）

输出

输出只有一行，包含一个整数，表示最多能拦截多少支雷电箭。

输入样例

```
8
300 207 155 300 299 170 158 65
```

输出样例

```
6
```

参考代码(C++)

```
1. // NQ088 拦截雷电箭
2. #include <algorithm>
3. #include <iostream>
4. #include <vector>
5. using namespace std;
6. vector<int> s, d; //用 vector 来改写代码
7. int main() {
8.     int n, h;
9.     cin >> n;
10.    //读入雷电箭,存储到 vector 中
11.    for (int i = 0; i < n; i++) { //下标从 1 开始到 n
12.        cin >> h;
13.        s.push_back(h); // s[i]为终点的最长子序列
14.        d.push_back(1); // d[i]初始值为 1
15.    }
16.    // d[0]=1;
17.    //每次求以第 i 个数为终点的最长不上升子序列的长度
18.    for (int i = 1; i < n; i++)
19.        for (int j = 0; j < i; j++)
20.            //察看以第 j 个数为终点的最长不上升子序列
21.            if (s[i] <= s[j]) //比较是否不大于
22.                d[i] = max(d[i], d[j] + 1);
23.    cout << *max_element(d.begin(), d.end()); //取最大值
24.    return 0;
25. }
```

3.4 拦截雷电箭(2) ^{NQ089}

描述



莱尼尔是海鲁拉大地之中最强的怪兽之一，无论近战还是远程攻击，莱尼尔都近乎完美。

莱尼尔最令人闻风丧胆的远程攻击是箭雨，这是一种能够动态跟踪对手所在位置的空对地攻击。无论距离多远，只要箭雨飞出，打击必达。

为了能够完美抵御莱尼尔的箭雨，约珥开发了一种箭雨拦截系统，可以发射出若干海鲁拉盾，在空中拦截莱尼而的箭雨。于此同时，整个拦截系统可以透过ambo的方式传送给林克。

但是每套箭雨拦截系统有一个缺陷：第一面地对空的发射的海鲁拉盾可以到达任意的高度，但是以后每一面海鲁拉盾都不能高于前一面的高度。

在实际战斗中，已知莱尼尔射出的箭雨阵中每一支雷电箭依次飞来的高度（高度数据是 ≤ 50000 的正整数）。

请计算每套箭雨拦截系统最多能拦截多少支雷电箭？

如果要拦截所有雷电箭，林克最少要配备多少套这种雷电箭拦截系统？

输入

1行，若干个整数（个数 ≤ 1000000 ）

注意：共有20组测试数据，前10组 N^2 算法可AC，后10组需要 $n(\log n)$ 算法。

输出

2行，每行一个整数。

第一个数字表示这套箭雨系统最多能拦截多少雷电箭。

第二个数字表示如果要拦截所有雷电箭最少要配备多少套这种箭雨拦截系统。

输入样例

```
388 209 157 307 299 170 158 65
```

输出样例

```
6
2
```

参考代码(C++)

```
1. // NQ089 拦截雷电箭 2
2. //求最长的不上升子列,以及最长上升子列
3. #include <algorithm>
4. #include <iostream>
5. #include <vector>
6. using namespace std;
7. int main() {
8.     // d[i]的含义是: 最大不上升子序列长度为 i 时, 最优的结尾元素。
9.     // d[len]存储最优的结尾元素。len 为不上升子列的长度值。
10.    vector<int> s, d, d2;
11.    int n, h;
12.    n = 1;
13.    while (cin >> h) {
14.        n++;
15.        s.push_back(h); // s[i]为终点的最长子序列
16.    };
17.    n--; //输入
18.    d.push_back(*s.begin()); //第一个元素压入 d
19.    d2.push_back(*s.begin()); //第一个元素压入 d2
20.    //查看当前元素 i 是否需要压入 d 中
21.    for (int i = 1; i < n; i++) {
22.        if (s[i] <= d.back()) //比较是否不大于
23.            d.push_back(s[i]); //压入 d
24.        else //踢掉第一个比 s[i]大的元素
25.            *upper_bound(d.begin(), d.end(), s[i], greater<int>()) = s[i];
26.        if (s[i] > d2.back()) //比较是否大于
27.            d2.push_back(s[i]); //压入 d2
28.        else //踢掉第一个比 s[i]小的元素
29.            *lower_bound(d2.begin(), d2.end(), s[i]) = s[i];
30.    }
31.    cout << d.size() << endl; //取最大值
32.    cout << d2.size() << endl; //取最大值
33.    return 0;
34. }
```

3.5 击杀黄金蛋糕人马^{NQ090}

描述

在海拉鲁大陆冒险, 没有绝佳的剑法+想象力是不可能存活下来的。

这不, 林克遇到了一个特别巨大的敌人——黄金蛋糕人马 (莱尼尔的变种)

这黄金蛋糕人马长相非常特别，没有脚没有手没有嘴巴没有头，整个身材就是一个大矩形（喂喂，这不就是黄金莱尼尔吗？）



它的长和宽分别是整数 w 、 h 。

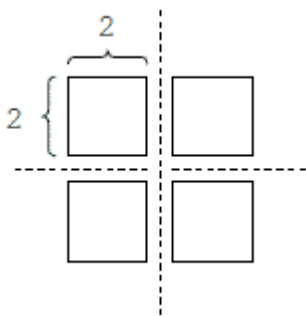
林克举起大师之剑，挥向黄金蛋糕人马，要将其切成 m 块矩形小块打包走，分给自己的朋友（每块都必须是矩形、且长和宽均为整数）。

大师之剑无比锐利，每一斩带出的剑气能将黄金蛋糕人马劈成两半（形成两个小矩形蛋糕）。

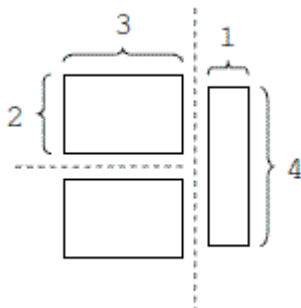
经过 $m-1$ 斩，黄金蛋糕人马居然被劈成 m 块小蛋糕（喂喂，你的想象力也太丰富了，明明切不开好吗？）

请计算：最后得到的 m 块小蛋糕中，最大的那块蛋糕的面积下限。

假设 $w=4, h=4, m=4$ ，则下面的斩击可使得其中最大蛋糕块的面积最小。（十字斩）



假设 $w=4, h=4, m=3$ ，则下面的斩击可使得其中最大蛋糕块的面积最小：（二连斩）



输入

共有多行，每行表示一个测试案例。

每行是三个用空格分开的整数 w, h, m ，其中 $1 \leq w, h, m \leq 20$ ， $m \leq wh$ 。

当 $w = h = m = 0$ 时不需要处理，表示输入结束。

输出

每个测试案例的结果占一行，输出一个整数，表示最大蛋糕块的面积下限。

输入样例

```
4 4 4
4 4 3
0 0 0
```

输出样例

```
4
6
```

参考代码(C++)

```
1. //NQ090 击杀黄金蛋糕人马
2. #include <iostream>
3. #include <cstring>
4. #include <cmath>
5. #include <algorithm>
6. using namespace std;
7. #define For(a,b,c) for (int a=b;a<c;++a)
8. #define Clear(a,b) memset(a,b,sizeof(a))
9. const int Inf=0x3f3f3f3f,Inf2=0x7fffffff;
10. int W,H,M;//宽、高、切成M块(只需要M-1斩)
11. int minMax[27][27][407]; //W, H 最高是 20, 所以 M 最多斩成 400 块
12. int dfs(int w,int h,int cnt)
13. {
14.     if( w * h < cnt+1)//不够斩
15.         return Inf;
16.     if( cnt == 0)//斩完毕
17.         return w * h;
18.     if( minMax[w][h][cnt] != -1)// Visited?
19.         return minMax[w][h][cnt];
20.     int minMArea = 0;
21.     minMArea = Inf;
22.     For(i,1,w) //第一斩是竖斩，产生的各种状态
23.         For(k,0,cnt) { //枚举左右两半的各种切法,若左边切为 k 块，右边就是 cnt-1-k;
24.             int m1 = dfs(i,h,k);//搜索左侧的切法
```

```

25.         int m2 = dfs(w-i,h,cnt-1-k); //搜索右侧的切法
26.         minMArea = min(minMArea,max(m1,m2)); //取最小值
27.     }
28.     For(j,1,h) //第一斩是横斩，产生的各种状态
29.         For(k,0,cnt) {
30.             int r1 = dfs(w,j,k); //枚举上下两半的各种切法
31.             int r2 = dfs(w,h-j,cnt-1-k);
32.             minMArea= min(minMArea,max(r1,r2));
33.         }
34.     return minMax[w][h][cnt] = minMArea;
35. }
36. int main()
37. {
38.     while(true) {
39.         cin >> W >> H >> M;
40.         if( W == 0 && H == 0)
41.             break;
42.         Clear(minMax,-1);
43.         cout << dfs(W,H,M-1) << endl;
44.     }
45.     return 0;
46. }

```

3.6 骑士林克的怜悯(1) NQ091

描述

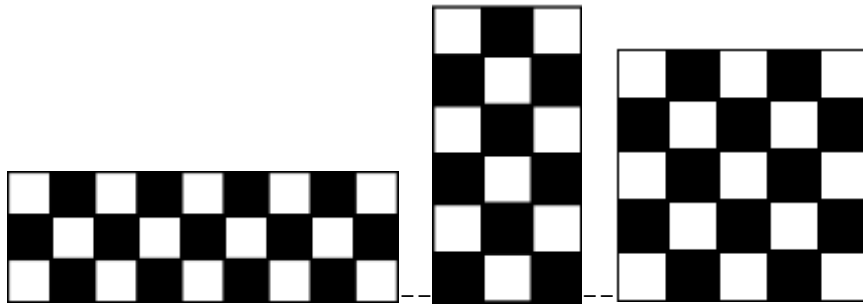


林克驰骋在海拉鲁大陆的平原上无比自由，他想起二维空间中的国际象棋同伴，回想起自己也活在2D世代的局限，心生怜悯。

那些骑士，永远被局限在 8×8 的棋盘之内厮杀，他们的世界永不改变。

林克找到去到阿卡来研究所，得到新的道具—变形棋盘。这个变形棋盘可以根据输入的两个参数的 (p, q) 创造全新的棋盘空间。

如下图分别是 (p, q) 为 $(3, 9)$, $(6, 3)$, 以及 $(5, 5)$ 的棋盘空间。



假设2D世界的骑士，移动的方式按字母次序有如下8种：



请问对于每一种棋盘 (p, q) , 请问2D骑士是否有一种一次遍历所有棋盘方格的路线？

如果有，请输出这条路线（若有多条路线，请输出字典序最小的路线）。

如果没有，请输出无。

输入

输入数据第一行为正整数 n ，代表有多少组输入样例

接下来 n 行是两个整数代表行 p 和列 q ，代表变形棋盘的行列参数，

其中 $(1 \leq p * q \leq 26)$ 。

输出

每个样例的输出2行，格式如下：

"#i:" 其中 i 代表第 i 种棋盘

骑士跳过的每个格子（每个访问的格子用大写字母加数字表示）

一条可行的路径输出如 $(A1B3C1A2B4C2A3B1C3A4B2C4)$,

如果没有可行方案，则第二行输出：none

输入样例

```
5
5 1
5 2
5 3
```

```
5 4
5 5
```

输出样例

```
#1:
none
#2:
none
#3:
none
#4:
A1B3A5C6D4B5D6C4D2B1A3C2B4A2C1D3B2D1C3D5B6A4C5A6
```

参考代码(C++)

```
1. //NQ095 骑士林克的怜悯
2. #include <cstring>
3. #include <iostream>
4. #include <vector>
5. using namespace std;
6. struct Point {
7.     int r, c;
8. };
9. Point dir[8] = {{-2, -1}, {-2, 1}, {-1, -2}, {-1, 2},
10.                {1, -2}, {1, 2}, {2, -1}, {2, 1}};
11. Point path[30];
12. int p, q;
13. int f[30][30];
14. //从 (r,c)出发, 此时已经走了 step 步, 看能否成功
15. bool dfs(int r, int c, int step) {
16.     //?所有棋盘格子都走到
17.     if (step == p * q) return true;
18.     //?是否超过边界
19.     if (r < 0 || r >= q || c < 0 || c >= p) return false;
20.     //?r,c 是否来过
21.     if (f[r][c]) return false;
22.     f[r][c] = 1;
23.     path[step].r = r;
24.     path[step].c = c;
25.     // todo 枚举 8 个方向, 深搜
26.     for (int i = 0; i < 8; ++i) {
27.         if (dfs(r + dir[i].r, c + dir[i].c, step + 1)) return true;
28.     }
29.     f[r][c] = 0; //回溯, 取消这一步的走法, 使得走其他步的时候, 能绕回到这里
30.     return false;
```

```

31. }
32. int main() {
33.     int t;
34.     cin >> t;
35.     for (int tt = 1; tt <= t; ++tt) {
36.         cout << "#" << tt << ":" << endl;
37.         cin >> p >> q; // p 数字, q 字母
38.         memset(f, 0, sizeof(f));
39.         /*复用变量 i 作为退出标记
40.         int i;
41.         for (i = 0; i < q; ++i)
42.             for (int j = 0; j < p; ++j) {
43.                 if (dfs(i, j, 0)) {
44.                     i = q + 77; /*加上你喜欢的数
45.                     for (int k = 0; k < p * q; ++k)
46.                         cout << char(path[k].r + 'A') << (path[k].c + 1);
47.                     break;
48.                 }
49.             }
50.         if (i == q) /*判断 i 的值是否被手动更改
51.             cout << "none";
52.         cout << endl;
53.     }
54. }

```

3.7 骑士林克的怜悯(2) NQ092

描述



帮助自己的2D骑士同伴进入到变形棋盘世界不久，林克意识到既然棋盘可变，但是骑士们的战斗水平却没有长进。虽然可以在见到更多的风景，探索更多不同的空间，但是如果自身的实力没有提升，那么骑士们感受不到那真正的自由以及成长的快乐。

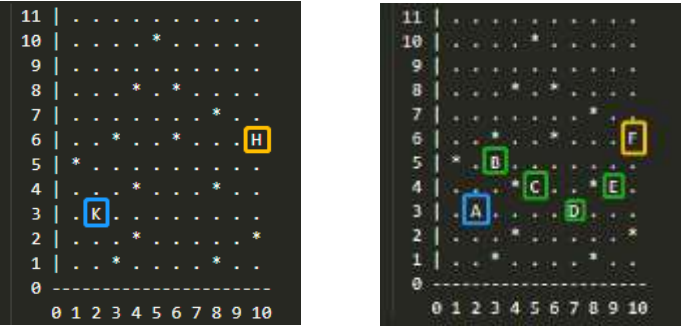
林克找到去到阿卡来研究所，得到新的道具——试炼棋盘。

这个新道具可以在骑士的2D空间中创造试炼场以及2D守护者，让骑士们可以开始实战演练，提升攻击力。

如下图是一个10列11行的棋盘（11×10）：

K代表骑士的位置，H代表守护者的位置。

. 代表可移动的位置，*代表障碍物。



骑士可以按照下图中的A,B,C,D...这条路径用5次跳跃，抵达守护者的位置偷袭它。

（有可能其它路线的长度也是5）.请问，2D骑士要偷袭守护者，至少要跳多少次？

输入

第一行： 两个数，表示棋盘的列数Column(<=150)和行数Row(<=150)

第二行到结尾： Row行Column列的棋盘。

输出

一个数，表示跳跃的最小次数。

输入样例

```
10 11
.....
....*....
.....
...*.
.....*
..*.*.*.H
*.....
...*.*.
.K.....
...*.*.
..*.*.*.
```

输出样例

5

参考代码(C++)

```
1. // NQ096 骑士林克的怜悯(2)
2. #include <bits/stdc++.h>
3. using namespace std;
4. #define FOR(i, a, b) for (int i = a; i <= b; i++)
5. // 搜索 8 个马跳跃的方向
6. const int dx[8] = {1, 2, 2, 1, -1, -2, -2, -1};
7. const int dy[8] = {2, 1, -1, -2, -2, -1, 1, 2};
8.
9. char g[160][160];
10. int n, m, d[160][160];
11. #define pii pair<int, int>
12. pii start, target;
13. queue<pii> q;
14.
15. bool check(int x, int y) { //在范围内;且不是障碍物;第一次被访问
16.     return x>=1 && x<=n && y>=1 && y<=m && g[x][y]!='*' && d[x][y]==-1;
17. }
18. void setST(void) //找起点和终点
19. {
20.     FOR(i, 1, n) FOR(j, 1, m) {
21.         if (g[i][j] == 'K') {
22.             start.first = i;
23.             start.second = j;
24.         }
25.         if (g[i][j] == 'H') {
26.             target.first = i;
27.             target.second = j;
28.         }
29.     }
30. }
31. int bfs(void) {
32.     memset(d, -1, sizeof(d));
33.     q.push(make_pair(start.first, start.second)); //压入起点
34.     d[start.first][start.second] = 0;
35.     while (q.size()) {
36.         pii pHead = q.front();
37.         q.pop();
38.         FOR(i, 0, 7) {
39.             int x = pHead.first + dx[i], y = pHead.second + dy[i]; //拓展
40.             if (check(x, y)) //满足条件
41.             {
42.                 d[x][y] = d[pHead.first][pHead.second] + 1;
```

```

43.     q.push(make_pair(x, y));
44.     if (x == target.first && y == target.second) //到达终点了
45.         return d[x][y];
46.     }
47. }
48. }
49. return -1; //然而并没有无解情况
50. }
51. int main() {
52.     scanf("%d%d\n", &m, &n); //谨慎读入!
53.     FOR(i, 1, n)
54.         scanf("%s", g[i] + 1); //读入从字符 1 开始
55.     setST(); //锁定起点和终点
56.     cout << bfs();
57.     return 0;
58. }

```

3.8 突袭路线^{NQ093}

描述



为了解救公主，林克必须深入敌后。在备战前，他拿出“关系分析仪”扫描敌营中每个士兵之间的关系。

关系分析仪的功能说明如下：如果A的活动范围在B的眼皮底下，那么分析仪就会从B出发连一条射线指向A（ $B \rightarrow A$ ）。

经过扫描，林克得到全营敌兵的相互关系。有些敌人被多个同伴看顾，有些敌人背后一个替他守望的都没有。

林克决定从背后没有人的敌人开始，潜伏到其背后，突袭之，并且避免被其他人发现。

军营一共有 n 个敌人，彼此之间的关系有 m 条射线，请找到一条可以逐个击破敌人的路线图。

如果找不到这样一条突袭路线，请则输出-1。

提示：

问题转化为：给定一个 n 个点 m 条边的有向图，点的编号是1到 n ，图中可能存在重边和自环。

请输出任意一个该有向图的拓扑序列，如果拓扑序列不存在，则输出-1。

若一个由图中所有点构成的序列 A 满足：对于图中的每条边 (x, y) ， x 在 A 中都出现在 y 之前，则称 A 是该图的一个拓扑

序列。

数据范围： $1 \leq n, m \leq 10^5$

输入

第一行包含两个整数 n 和 m

接下来 m 行，每行包含两个整数 x 和 y ，表示存在一条从点 x 到点 y 的有向边 (x, y) 。

输出

共一行，如果存在拓扑序列，则输出拓扑序列。

否则输出 -1 。

输入样例

```
3 3
1 2
2 3
1 3
```

输出样例

```
1 2 3
```

参考代码(C++)

```
1. // NQ093 突袭路线
2. #include <algorithm>
3. #include <cstring>
4. #include <iostream>
5. #include <queue>
6. using namespace std;
7. #define For(a, begin, end) for (int a = (begin); a < (end); a++)
8. const int N = 10000007; //题目 n 的范围
9. int n, m;
10. int head[N], edge[N], nextVertex[N], idx; //邻接表
11. int q[N], d[N]; //队列，入度
12. void add(int a, int b) {
13.     edge[idx] = b;
14.     nextVertex[idx] = head[a];
15.     head[a] = idx++;
16. }
17. bool topsort() {
18.     int qHeadIdx = 0, qTailIdx = -1; //队头，队尾
19.     //从前往后遍历入度为 0 的点，插入队列
20.     for (int i = 1; i <= n; i++)
```

```

21.     if (!d[i]) q[++qTailIdx] = i; //数组模拟队列，入队是尾指针+1
22. while (qHeadIdx <= qTailIdx) //如果头指针小于尾指针
23. {
24.     int t = q[qHeadIdx++]; //取队列头元素，，出队只是把头指针往后移动一位
25.     //遍历 t 的临边，空指针初始化为-1
26.     for (int i = head[t]; i != -1; i = nextVertex[i]) {
27.         int j = edge[i]; //取到出边
28.         d[j]--; //入度减一
29.         if (d[j] == 0) q[++qTailIdx] = j; //如若入度为 0，压入队列
30.     }
31. }
32. //判断是否所有顶点都入队
33. //也就是头指针 qTailIdx 是否等于 n-1,即所有点都进入队列了
34. return qTailIdx == n - 1;
35. }
36. int main() {
37.     cin >> n >> m; //读入 n,m
38.     memset(head, -1, sizeof(head)); //初始化 Head 数组指向-1 顶点
39.     For(i, 0, m) {
40.         int a, b;
41.         cin >> a >> b;
42.         add(a, b); //插入边
43.         d[b]++; //更新入度
44.     }
45.     if (topsort()) { //数组队列里面的元素就是拓扑序
46.         for (int i = 0; i < n; i++) printf("%d ", q[i]);
47.     } else
48.         puts("-1");
49.     return 0;
50. }

```

算法基础

3.9 全面反击^{NQ094}

描述



加农复活后，他邪恶的势力掌控了海拉鲁大陆几乎所有的神庙。

为了避免路上战斗的时间，决定绕开路上的敌人，全力攻取神庙。

他打开地图，研究进攻路线。

若神庙A到神庙B有一条可以绕开敌人的路，林克就把路线标在地图上。

经过N个小时的努力，林克完成这宏伟的绘图工作。

他得到一张N个点M条边的有向无环图，每个点代表一个神庙的位置。

请分别统计从每个点出发能够到达的点的数量。

数据范围

$$1 \leq N, M \leq 30000$$

输入

第一行两个整数N, M，接下来M行每行两个整数x, y，表示从x到y的一条有向边。

输出

输出共N行，表示每个点能够到达的点的数量。

输入样例 1

```
10 10
3 8
2 3
2 5
5 9
5 9
2 3
3 9
4 8
```

```
2 10
4 9
```

输出样例 1

```
1
6
3
3
2
1
1
1
1
1
1
```

参考代码(C++)

```
1. //NQ094 全面反击
2. #include <algorithm>
3. #include <bitset>
4. #include <cstdio>
5. #include <cstring>
6. #include <iostream>
7. #include <queue>
8. using namespace std;
9. #define MAXM 30007
10. int n, m, degree[MAXM], a[MAXM]; //存储入度、拓扑序的顶点
11. int edge[MAXM], Next[MAXM], head[MAXM], tot, cnt;
12. bitset<MAXM> f[MAXM]; //使用 bit
13. void add(int x, int y) { // 在邻接表中添加一条有向边
14.     edge[++tot] = y, Next[tot] = head[x], head[x] = tot;
15.     degree[y]++; //入度++
16. }
17. // 拓扑排序
18. void topsort() {
19.     queue<int> q; //队列
20.     for (int i = 1; i <= n; i++)
21.         if (degree[i] == 0) //所有入度为 0 的顶点入栈
22.             q.push(i);
23.     //栈不空的时候拓扑排序
24.     while (q.size()) {
25.         int x = q.front();
26.         q.pop();
27.         a[++cnt] = x; //保存拓扑过程中的顶点
28.         // 拓展顶点 x
```

```

29.     for (int i = head[x]; i; i = Next[i]) {
30.         int y = edge[i];
31.         --degree[y]; //入度减 1
32.         if (degree[y] == 0) q.push(y);
33.     }
34. } // end of while
35. } // end of topsort
36. void calReachableNodes() {
37.     //逆序统计
38.     for (int i = cnt; i; i--) {
39.         int x = a[i]; //取拓扑排序过程中的顶点
40.         f[x][x] = 1; //当前顶点设置为访问过
41.         for (int i = head[x]; i; i = Next[i]) {
42.             int y = edge[i];
43.             f[x] |= f[y]; //求并集合
44.         }
45.     }
46. }

47. int main() {
48.     cin >> n >> m; // 点数、边数
49.     for (int i = 1; i <= m; i++) {
50.         int x, y;
51.         cin >> x >> y;
52.         add(x, y); //建图
53.     }
54.     topsort(); //拓扑排序，顶点存储在全局数组 a 中
55.     calReachableNodes(); //逆序计算可达顶点数
56.     for (int i = 1; i <= n; i++) cout << f[i].count() << endl;
57. }

```

3.10 真假记忆碎片^{NQ095}

描述



在千辛万苦得到一个记忆碎片之后，林克需要辨别真假，若是所得到的并不是发生历史中的碎片，乃

是后人捏着的，那么林克如何寻回完整的自己呢？

已知林克找到的记忆碎片 9×9 矩阵是初始矩阵A的解法，也就是记忆碎片A。空白的部分在初始矩阵中用0表示。

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

(初始矩阵A)

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

(记忆碎片A)

请写一个算法，判定找到的记忆碎片是否是真的？

输入

输入的记忆碎片A是一个9行9列的数独矩阵。

每行包含9个数字（均不超过数字为1-9）。

初始矩阵A：

```
530070000
600195000
098000060
800060003
400803001
700020006
060000280
000419005
000080079
```

输出

如果输入数据真的是初始矩阵A的解，输出Yes，否则输出No

输入样例 1

```
534678912
672195348
198342567
859761423
426853791
713924856
961537284
287419635
345286179
```

输出样例 1

Yes

输入样例 2

```
123456789
123456789
123456789
123456789
123456789
123456789
123456789
123456789
123456789
123456789
```

输出样例 2

No

参考代码(C++)

```
1. //NQ091 真假记忆碎片
2. #include <iostream>
3. #include <cstring>
4. using namespace std;
5. #define For(a, begin, end) for (register int a = begin; a < end; ++a)
6. using namespace std;
7. //a 为 9×9 数组, 用 int 类型
8. int a[9][9]=
9.     { {5,3,0,0,7,0,0,0,0},
10.       {6,0,0,1,9,5,0,0,0},
11.       {0,9,8,0,0,0,0,6,0},
12.       {8,0,0,0,6,0,0,0,3},
13.       {4,0,0,8,0,3,0,0,1},
14.       {7,0,0,0,2,0,0,0,6},
15.       {0,6,0,0,0,0,2,8,0},
16.       {0,0,0,4,1,9,0,0,5},
17.       {0,0,0,0,8,0,0,7,9}};
18. int b[9][9]; //把输入也转换为 int 类型
19. string aline;
20. bool Check() {
21.     For(i, 0, 9) {
22.         cin >> aline;
23.         if (aline.size() != 9) {
24.             return false;
```

```

25.     }
26.     For(j, 0, 9) {
27.         b[i][j] = aline[j] - '0'; //转为整数
28.         if (b[i][j] < 1 || b[i][j] > 9) {
29.             return false;
30.         }
31.         if (a[i][j] != 0 && a[i][j] != b[i][j]) {
32.             return false;
33.         }
34.     }
35. }

36. bool isUsed[10];
37. For(i, 0, 9) {
38.     For(j, 0, 10) isUsed[j] = false;
39.     For(j, 0, 9) {
40.         if (isUsed[b[i][j]] == true) {
41.             return false;
42.         } else
43.             isUsed[b[i][j]] = true;
44.     }
45. }
46. For(j, 0, 9) {
47.     For(i, 0, 10) isUsed[i] = false;
48.     For(i, 0, 9) {
49.         if (isUsed[b[i][j]] == 1) {
50.             return false;
51.         } else
52.             isUsed[b[i][j]] = true;
53.     }
54. }
55. for (int i = 0; i < 9; i += 3) {
56.     for (int j = 0; j < 9; j += 3) {
57.         For(k, 0, 10) isUsed[k] = false;
58.         For(i1, 0, 3) For(j1, 0, 3) if (isUsed[b[i + i1][j + j1]] == 1) {
59.             return false;
60.         }
61.         else isUsed[b[i + i1][j + j1]] = true;
62.     }
63. }
64. return true;
65. }
66. int main() {
67.     if (Check())

```

```

68.     cout << "Yes" << endl;
69.     else
70.         cout << "No" << endl;
71.     return 0;
72. }

```

3.11 寻找林克的回忆(1) ^{NQ096}

描述



为了找到百年沉睡的原因，寻回百年前与公主一起的记忆碎片，明白自己是谁，林克必须破解数独谜题。

林克需要在限定时间内，把 9×9 的数独补充完整，使得图中每行、每列、每个 3×3 的九宫格内数字1~9均恰好出现一次。

林克需要寻回失去的记忆碎片，你，作为林克的朋友，需要帮忙林克寻回 9×9 棋盘上失去的数字。

或许有一天，林克也能帮助你，寻回关于你是谁，你从哪里来的记忆碎片。

这是数独试炼I（解密成功可以解锁林克前25%的记忆碎片）。

1		3			5		9
		2	1	9	4		
			7	4			
3			5	2			6
	6					5	
7			8	3			4
			4	1			
		9	2	5	8		
8		4			1		7

输入

输入为 9×9 的数据。一共9行，每行有9个数字。

数字为0表示对应的数字盘为空。

输出

对于每个测试用例，程序应以与输入数据相同的格式打印解决方案(9×9)。

空单元格必须根据规则进行填充。

如果解决方案不是唯一的，则程序可以打印其中任何一种。

输入样例

```

103000509
002109400
000704000
300502006

```

```
060000050
700803004
000401000
009205800
804000107
```

输出样例

```
143628579
572139468
986754231
391542786
468917352
725863914
237481695
619275843
854396127
```

参考代码(C++)

```
1. // NQ092 寻找林克的回忆 1
2. #include <cmath>
3. #include <cstring>
4. #include <iostream>
5. using namespace std;
6. #define For(a, begin, end) for (register int a = begin; a < end; ++a)
7. #define blankchar '0' //另外一题为'.'
8. //输入输出数据和全局变量
9. #define INF 0x7fffffff
10. const int N = 9;
11. const int maxArray = 1 << N; // *1<<9 为 512
12. int row[N], col[N], cell[3][3]; // ?9×9 的矩阵, 用 行 row 列 col 3×3 的 Cell
13. int ones[maxArray], lowbit2pos[maxArray];
14. bool found = false;
15. string str; //存储输入的每一行字符串
16. inline int lowbit(int x) { return x & -x; }
17. inline int clearbit(int &x, int n) { return x -= 1 << n; }
18. inline int resetbit(int &x, int n) { return x += 1 << n; }
19. //取交集的运算
20. inline int get(int x, int y) //返回 row[x], col[x] 与 cell[x][y] 的可用集合
21. {
22.     return row[x] & col[y] & cell[x / 3][y / 3];
23. }
24.
```



```

25. void init() {
26.     for (int i = 0; i < N; i++)
27.         row[i] = col[i] = maxArray - 1; //每一个位都设置为 1: 511=1 11111111
28.     for (int i = 0; i < 3; i++)
29.         for (int j = 0; j < 3; j++)
30.             cell[i][j] = maxArray - 1; //每一个位都设置为 1: 511=1 11111111
31. }
32. void printStr2line(string s) { cout << s << endl; }
33. void printStr2Grid(string s) {
34.     For(i, 0, 9) cout << s.substr(i * 9, 9) << endl;
35. }
36. string readGrid2str(int n) {
37.     string res;
38.     For(i, 0, n) {
39.         string s;
40.         cin >> s;
41.         res += s;
42.     }
43.     return res;
44. }

45. bool dfs(int cnt) {
46.     if (cnt == 0) return true;
47.     found = false;
48.     //找出可选方案数最小的格子
49.     int minv = INF;
50.     int x, y;
51.     for (int i = 0; i < N; i++)
52.         for (int j = 0; j < N; j++)
53.             if (str[i * 9 + j] == blankchar) {
54.                 int t = ones[get(i, j)]; //打表得到有几个可选方案 t
55.                 if (t < minv) {
56.                     minv = t, x = i, y = j;
57.                 }
58.             } //循环遍历, 得到方案最小的 x,y, 以及最小方案数 minv
59.     for (int sk = get(x, y); sk; sk -= lowbit(sk)) {
60.         // sk 为某数独串(使用 lowbit 循环取出每一个要填的空, 状态为 1 为要填的空)
61.         //得到需要填的数, 从小到大, 最小是 0, 最大是 8
62.         int t = lowbit2pos[lowbit(sk)];
63.         //修改状态 row,col,cell
64.         clearbit(row[x], t), clearbit(col[y], t);
65.         clearbit(cell[x / 3][y / 3], t);
66.         //修改 str 对应位置填上数(0-8 映射到 1-9)

```

```

67.     str[x * 9 + y] = '1' + t;
68.     //继续深搜
69.     found = dfs(cnt - 1);
70.     if (found) return true; //找到一种方案
71.     //恢复状态
72.     str[x * 9 + y] = blankchar;
73.     resetbit(row[x], t), resetbit(col[y], t);
74.     resetbit(cell[x / 3][y / 3], t);
75. }
76. return false;
77. }

78. int main() {
79.     // 初始化 lowbi2pos, 只有离散的几个数字有值
80.     for (int i = 0; i < N; i++)
81.         lowbit2pos[1 << i] = i; // *1<<8=256;...1<<1=2, 1<<0=1
82.     for (int i = 0; i < maxArray; i++) {
83.         int s = 0;
84.         for (int j = i; j; j -= lowbit(j)) s++;
85.         ones[i] = s; // i 的二进制表示中有 s 个 1
86.     }
87.     str = readGrid2str(9); //读入 9 行
88.     init();
89.     // i,j 是九宫格的行列坐标, k 是字符串中的线性坐标
90.     int cnt = 0;
91.     for (int i = 0, k = 0; i < N; i++)
92.         for (int j = 0; j < N; j++, k++) // *k 的用法
93.             if (str[k] != blankchar) {
94.                 // *把题目的 1-9 映射成 0-8
95.                 int t = str[k] - '1';
96.                 //第 t 位设置为 0
97.                 clearbit(row[i], t), clearbit(col[j], t);
98.                 clearbit(cell[i / 3][j / 3], t);
99.             } else
100.                 cnt++;
101.     dfs(cnt);
102.     printStr2Grid(str);
103.     return 0;
104. }

```

3.12 寻找林克的回忆(2) ^{NQ097}

描述



为了找到百年沉睡的原因，寻回百年前与公主一起的记忆碎片，明白自己是谁，林克必须破解数独谜题：

林克需要在限定时间内，把 9×9 的数独补充完整，使得图中每行、每列、每个 3×3 的九宫格内数字1~9均恰好出现一次。

林克需要寻回失去的记忆碎片，你，作为林克的朋友，需要帮忙林克寻回 9×9 棋盘失去的数字。

或许有一天，林克也能帮助你，寻回关于你是谁，你从哪里来的记忆碎片。

这是数独试炼II（解密成功可以解锁林克25%的记忆碎片）

请编写一个程序填写数独。

.	2	7	.	3	8	.	.	1
.	1	.	.	.	6	7	3	5
.	.	.	.	.	.	.	2	9
3	.	5	6	9	2	.	8	.
.	.	.	.	.	.	.	.	.
.	6	.	1	7	3	5	.	3
6	4	.	.	.	.	.	.	.
9	5	1	8	.	.	.	7	.
.	8	.	.	6	5	3	4	.

输入

输入包含多组测试用例。

每个测试用例占一行，包含81个字符，代表数独的81个格内数据（顺序总体由上到下，同行由左到右）。

每个字符都是一个数字（1-9）或一个“.”（表示尚未填充）。

您可以假设输入中的每个谜题都只有一个解决方案。

文件结尾处为包含单词“end”的单行，表示输入结束。

输出

每个测试用例，输出一行数据，代表填充完全后的数独。

输入样例

```
.2738..1..1...6735.....293.5692.8.....6.1745.364.....9518...7..8..6534.
.....52..8.4.....3...9...5.1...6..2..7.....3.....6...1.....7.4.....3.
end
```

输出样例

```
527389416819426735436751829375692184194538267268174593643217958951843672782965341
416837529982465371735129468571298643293746185864351297647913852359682714128
```

参考代码(C++)

```
1. // XNQ093 寻找林克的回忆(2)
2. #include <cmath>
3. #include <cstring>
4. #include <iostream>
5. using namespace std;
6. #define For(a, begin, end) for (register int a = begin; a < end; ++a)
7. /* 输入输出数据和全局变量
8. #define INF 0x7fffffff
9. #define BLANKCHAR '.'
10. const int N = 9;
11. const int MaxN = 1 << N; // *1<<9 为 512
12. string str; // 存储输入的每一行字符串
13. int row[N], col[N], cell[3][3]; // ?9x9 的矩阵, 用 行 row 列 col 3x3 的 Cell
14. int ones[MaxN];
15. int LOG2[MaxN];
16. void BuildLOG2() { For(i, 0, N) LOG2[1 << i] = i; }
17. // !取 x 二进制序列最后末一个 1000 模式的字串
18. inline int lowbit(int x) { return x & -x; }
19. inline int NumberOf1(int n) {
20.     int res = 0;
21.     while (n) {
22.         n -= lowbit(n);
23.         res += 1;
24.     }
25.     return res;
26. }
27. void Buildones() { For(i, 0, MaxN) ones[i] = NumberOf1(i); }
28. void init() {
29.     // 二进制为 1 表示该位数字为空
30.     For(i, 0, N) row[i] = col[i] = MaxN - 1; // !初始化为 11111111
31.     For(i, 0, 3) For(j, 0, 3) cell[i][j] = MaxN - 1; // !初始化为 11111111
32. }
33. // 打印一行 s
```

```

34. void printStr2line(string s) { cout << s << endl; }
35. //打印 9*9 矩阵
36. void printStr2Grid(string s) {
37.     For(i, 0, 9) cout << s.substr(i * 9, 9) << endl;
38.     cout << endl;
39. }

40. //把 x,y 位置二进制数的第 n 位取反
41. inline void flipbits(int x, int y, int n) {
42.     row[x] ^= 1 << n;
43.     col[y] ^= 1 << n;
44.     cell[x / 3][y / 3] ^= 1 << n;
45. }
46. //取交集的运算
47. inline int get(int x, int y) //返回 row[x],col[x]与 cell[x][y]的可用集合
48. {
49.     return row[x] & col[y] & cell[x / 3][y / 3]; //同时为 1 的二进制位
50. }
51. bool dfs(int cnt) {
52.     if (cnt == 0) {
53.         /*输出可行解
54.         printStr2line(str);
55.         return true;
56.     }
57.     //找出可选方案数最小的格子
58.     int minv = INF; //可以设置为 10 即可
59.     int x, y;
60.     For(i, 0, N) For(j, 0, N) if (str[i * 9 + j] == BLANKCHAR) {
61.         int t = ones[get(i, j)]; /*查表
62.         if (t < minv) {
63.             minv = t, x = i, y = j; /*从空格最少的行列开始处理，记录 x,y
64.         }
65.     }
66.     //从该方案[x,y,minV]开始枚举，指导找到 cellstosolve-1 方案为止
67.     // get(x,y) 的返回值代表行，列，Cell 中可选的二进制值
68.     for (int sk = get(x, y); sk; sk -= lowbit(sk)) {
69.         int t = LOG2[lowbit(sk)]; //得到 1 的最低位位置
70.         //修改状态 row,col,cell
71.         flipbits(x, y, t);
72.         str[x * 9 + y] = '1' + t; //修改 str 对应位置填上数(0-8 映射到 1-9)
73.         //继续深搜
74.         dfs(cnt - 1);
75.         //恢复状态
76.         str[x * 9 + y] = BLANKCHAR;

```

```

77.     flipbits(x, y, t);
78. }
79. return false;
80. }

81. int main() {
82.     /*初始化 LOG2
83.     BuildLOG2();
84.     // i 的二进制中有几个 1
85.     Builddones();
86.     while (cin >> str, str[0] != 'e') {
87.         init();
88.         // * i,j 是九宫格的行列坐标, k 是字符串中的线性坐标
89.         int cnt = 0;
90.         for (int i = 0, k = 0; i < N; i++)
91.             for (int j = 0; j < N; j++, k++)
92.                 if (str[k] != BLANKCHAR) {
93.                     int t = str[k] - '1'; /*把题目的 1-9 映射成 0-8
94.                     flipbits(i, j, t); /*初始化是 1, 这里取反设置为 0, 表示该位已占用
95.                 } else
96.                     cnt++;
97.         dfs(cnt);
98.     }
99.     return 0;
100. }

```

3.13 寻找林克的回忆(3) ^{NQ098}

描述



为了寻回百年前与公主一起的记忆碎片，林克历尽千辛万苦总算破解了数独试炼I和II的谜题，寻回50%的记忆碎片。

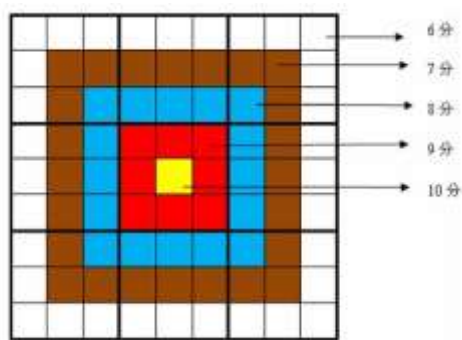
如今，摆在他面前是数独试炼III——传说中的靶形数独（通过后可以获得剩下的30%的记忆碎片）。

靶形数独的方格同普通数独一样，在9×9的大九宫格中有9个3×3的小九宫格（用粗黑色线隔开的）。

在这个大九宫格中，有一些数字是已知的，根据这些数字，利用逻辑推理，在其他的空格上填入1到9的数字。

每个数字在每个小九宫格内不能重复出现，每个数字在每行、每列也不能重复出现。

但靶形数独有一点和普通数独不同，即每一个方格都有一个分值，而且如同一个靶子一样，离中心越近则分值越高（如下图所示）。



上图具体的分值分布是：

最里面一格（黄色区域）为10分

黄色区域外面的一圈（红色区域）每个格子为9分

再外面一圈（蓝色区域）每个格子为8分

蓝色区域外面一圈（棕色区域）每个格子为7分

最外面一圈（白色区域）每个格子为6分

每个人必须完成一个给定的数独（每个给定数独可能有不同的填法），而且要争取更高的总分数。

而这个总分数即每个方格上的分值和完成这个数独时填在相应格上的数字的乘积的总和。

如图，在以下的这个已经填完数字的靶形数独游戏中，总分数为2829。

游戏规定，将以总分数的决出胜负。

7	5	4	9	3	8	2	6	1
1	2	8	6	4	5	9	3	7
6	3	9	2	1	7	4	8	5
8	6	5	4	2	9	1	7	3
9	7	2	3	5	1	6	4	8
4	1	3	8	7	6	5	2	9
5	4	7	1	8	2	3	9	6
2	9	1	7	6	3	8	5	4
3	8	6	5	9	4	7	1	2

求对于给定的靶形数独，能够得到的最高分数。

输入

输入一共包含9行。

每行 9 个整数（每个数都在 0-9 的范围内），表示一个尚未填满的数独方格，未填的空格用“0”表示。

每两个数字之间用一个空格隔开。

输出

输出可以得到的靶形数独的最高分数。

如果这个数独无解，则输出整数-1。

输入样例

```
7 0 0 9 0 0 0 0 1
1 0 0 0 0 5 9 0 0
0 0 0 2 0 0 0 8 0
0 0 5 0 2 0 0 0 3
0 0 0 0 0 0 6 4 8
4 1 3 0 0 0 0 0 0
0 0 7 0 0 2 0 9 0
2 0 1 0 6 0 8 0 4
0 8 0 5 0 4 0 1 2
```

输出样例

```
2829
```

参考代码(C++)

```
1. // NQ094 寻找林克的回忆(3)
2. #include <cmath>
3. #include <cstring>
4. #include <iostream>
5. using namespace std;
6. #define For(a, begin, end) for (register int a = begin; a < end; ++a)
7. /* 输入输出数据和全局变量
8. #define INF 0x7fffffff
9. #define BLANKCHAR '.'
10. const int N = 9;
11. const int MaxN = 1 << N;  /*1<<9 为 512
12. int ones[MaxN], LOG2[MaxN];
13. int row[N], col[N], cell[3][3];
14. int Sudoku[N][N];
15. int maxScore = -1;  /*记录最大值
16. inline int lowbit(int x) { return x & -x; }
17. void init() {
18.     For(i, 0, N) LOG2[1 << i] = i;  /*建表 LOG2
19.     For(i, 0, MaxN)  /*建表 ones
20.         for (int j = i; j; j -= lowbit(j)) ones[i]++;
```



```

21.  /**初始化为 11111111*/
22.  For(i, 0, 9) row[i] = col[i] = cell[i / 3][i % 3] = MaxN - 1;
23. }
24. /**返回 x,y 的权值 并且乘 t*/
25. inline int get_score(int x, int y, int t) {
26.     return (min(min(x, 8 - x), min(y, 8 - y)) + 6) * t;
27. }
28. /**返回 x,y 的可用位置, 用整数的二进制表示*/
29. inline int get(int x, int y) { return row[x] & col[y] & cell[x / 3][y / 3]; }
30. /**把 x,y 位置二进制数的第 n 位取反*/
31. inline void flipbits(int x, int y, int n) {
32.     row[x] ^= 1 << n;
33.     col[y] ^= 1 << n;
34.     cell[x / 3][y / 3] ^= 1 << n;
35. }
36. /**设置 g[x][y]的值为 t*/
37. inline void set(int x, int y, int t) { Sudoku[x][y] = t; }
38. /**score 参数记录当前 dfs 的累计权重*/
39. void dfs(int cellsLeft, int score) {
40.     if (!cellsLeft) {
41.         maxScore = max(maxScore, score); /**记录最大值*/
42.         return;
43.     }
44.     int minv = INF; /**寻找最少空格的位置*/
45.     int x, y;
46.     For(i, 0, N) For(j, 0, N) if (!Sudoku[i][j]) {
47.         int t = ones[get(i, j)];
48.         if (t < minv) {
49.             minv = ones[get(i, j)];
50.             x = i, y = j;
51.         }
52.     }
53.     /**从最少空格的点 x,y 开始深搜*/
54.     for (int i = get(x, y); i; i -= lowbit(i)) {
55.         int t = LOG2[lowbit(i)];
56.         set(x, y, t + 1); /**0-8 映射为 1-9*/
57.         flipbits(x, y, t); /**修改 row,col,cell 的状态位*/
58.         dfs(cellsLeft - 1, score + get_score(x, y, t + 1));
59.         flipbits(x, y, t); /**修改 row,col,cell 的状态位*/
60.         set(x, y, 0); /**把当前点 Sudoku[x][y]清 0*/
61.     }
62. }
63. int main() {
64.     /**建表, 初始化状态数组*/

```

```

65.  init();
66.  int cellsLeft = 0, score = 0;  ///初始数独的 score
67.  For(i, 0, N) For(j, 0, N) {
68.      int t;
69.      cin >> t;  //输入数据为空格分隔的数字
70.      if (t) {
71.          set(i, j, t);  //记录 Sudoku[x][y]为 t
72.          if (t > 0) flipbits(i, j, t - 1);
73.          score += get_score(i, j, t);  /*计算初始 score
74.      } else
75.          cellsLeft++;
76.  }
77.  dfs(cellsLeft, score);  ///不是从 score=0 开始!
78.  cout << maxScore << endl;
79.  return 0;
80. }

```

3.14 最省赛程^{NQ099}

描述



为了让自己能够驾驭大师摩托，打开了大师摩托的隐藏任务：“赛车试炼”。

然而这个特殊的赛车试炼，竟然比的不是车速，比的是如何“省”油钱。

林克需要驾驶不同邮箱容量各异的赛车，从起点城市S开到终点城市E。

有N个城市（编号0、1...N-1）和M条赛道（构成一张无向图）。

在每个城市里边都有一个加油站，不同的加油站的单位油价不一样（有些城市油价贵，有些城市油价便宜些）。

请计算，如果林克驾驶的是一辆油箱容量为C的赛车，那么他从起点城市S开到终点城市E至少要花多少油钱？

注意：车子初始时油箱是空的，需要在起点城市加油方可起行。

输入

第一行包含两个整数N和M。

第二行包含 N 个整数，代表 N 个城市的单位油价，第 i 个数即为第 i 个城市的油价 p_i 。

接下来 M 行，每行包括三个整数 u, v, d ，表示城市 u 与城市 v 之间存在道路，且赛车从 u 到 v 需要消耗的油量为 d 。

接下来一行包含一个整数 q ，代表问题数量 ($q < 100$)

接下来 q 行，每行包含三个整数 C, S, E ，分别表示赛车油箱容量、起点城市 S 、终点城市 E 。

数据范围：

$$\begin{aligned} 1 &\leq N \leq 1000, \\ 1 &\leq M \leq 10000, \\ 1 &\leq p_i \leq 100, \\ 1 &\leq d \leq 100, \\ 1 &\leq C \leq 100 \end{aligned}$$

输出

对于每个问题，输出一个整数，表示所需的最少油钱。

如果无法从起点城市开到终点城市，则输出“impossible”。

每个结果占一行。

输入样例

```
5 5
10 10 20 12 13
0 1 9
0 2 8
1 2 1
1 3 11
2 3 7
2
10 0 3
20 1 4
```

输出样例

```
170
impossible
```

参考代码(C++)

```
1. //NQ099 最省赛程
2. #include <cstring>
3. #include <iostream>
4. #include <algorithm>
5. #include <queue>
6. using namespace std;
```

```

7. //使用数组实现邻接表, 无向图
8. const int N = 1010;           //顶点数
9. const int M = 10100;          //边的数目
10. const int maxCapacity = 107; //油箱容量
11. int oilPrice[N], expenses[N][maxCapacity];
12. int head[N], Next[2 * M], ver[2 * M], dist[2 * M], tot = -1; //邻接表
13. bool visited[N][maxCapacity];
14. void add(int x, int y, int z)
15. {
16.     ver[++tot] = y;           //这条边到达的点
17.     Next[tot] = head[x]; //链接
18.     head[x] = tot;           //标记 x 起点
19.     //具体数据
20.     dist[tot] = z;           //权值
21. }
22. //! 访问从 u 出发的所有边, 遍历的代码如下, 当 next[i]为 0 的时候遍历结束
23. //todo:   for(int i=head[u]; i; i=Next[i])//遍历相邻顶点 v
24. //todo:   {
25. //todo:       int v = ver[i], d = dist[i];
26. //todo:   }
27. struct node
28. {
29.     int city, fuel, money; //城市, 赛车油量, 累计花费
30.     node(int x, int y, int z) : city(x), fuel(y), money(z) {}
31.     friend bool operator<(node a, node b)
32.     {
33.         return a.money > b.money; //权值从小到大排序, 记住是小根堆
34.     }
35. };
36. priority_queue<node> q;
37. bool buyOneOil(int city, int fuel, int c)
38. {
39.     //如果油还在油箱限制内, 且这买油一定更好的话
40.     if (fuel + 1 <= c && !visited[city][fuel + 1] &&
41.         (expenses[city][fuel + 1] > expenses[city][fuel] + oilPrice[city]))
42.         return true;
43.     else
44.         return false;
45. }
46. bool isNextRoad(int fuel, int cityid, int d, int money)
47. {
48.     //剩余油大于路径花费油、下一个状态未被访问过、并且花费更便宜
49.     if (fuel >= d && !visited[cityid][fuel - d] &&
50.         expenses[cityid][fuel - d] > money)

```

```

51.         return true;
52.     else
53.         return false;
54. }
55. //全局遍历列表
56. int n, m;
57. //Dijkstra 算法
58. int BFS(int currentCapacity, int start, int target)
59. {
60.     while (!q.empty())
61.         q.pop();
62.     memset(visited, false, sizeof(visited));
63.     memset(expenses, 0x3f, sizeof(expenses));
64.     //从起点开始搜索
65.     expenses[start][0] = 0;
66.     q.push(node(start, 0, 0)); //s 为起点,0 为剩余油量,0 为权值
67.     while (!q.empty())
68.     {
69.         node qHead = q.top();
70.         q.pop();
71.         int city = qHead.city;
72.         int fuel = qHead.fuel;
73.         int money = qHead.money;
74.         visited[city][fuel] = true; //访问过了
75.         if (city == target) //到达终点了
76.             return money;
77.         if (buyOneOil(city, fuel, currentCapacity)) //判断
78.         {
79.             //? 加上一升油的钱
80.             expenses[city][fuel + 1] = expenses[city][fuel] + oilPrice[city];
81.             //? 购买一升油的状态
82.             q.push(node(city, fuel + 1, expenses[city][fuel + 1]));
83.         }
84.
85.         for (int i = head[city]; i; i = Next[i]) //遍历相邻城市
86.         {
87.             int nextCity = ver[i], d = dist[i]; //此路径消耗油量 d
88.             if (isNextRoad(fuel, nextCity, d, money)) //判断
89.             {
90.                 expenses[nextCity][fuel - d] = money; //耗费油
91.                 q.push(node(nextCity, fuel - d, money));
92.             }
93.         } // end of for
94.     } // end of While

```

```

95.     return -1;
96. } // end of BFS
97. int main()
98. {
99.     ios::sync_with_stdio(false);
100.    cin >> n >> m;
101.    //代表 N 个城市的单位油价，第 i 个数即为第 i 个城市的油价 Pi
102.    for (int i = 0; i < n; i++)
103.        cin >> oilPrice[i];
104.    //每行包括三个整数 u,v,d，表示城市 u 与城市 v 之间存在道路，且赛车从 u 到 v 需要消耗的油量为 d。
105.    for (int i = 1; i <= m; i++)
106.    {
107.        int u, v, d;
108.        cin >> u >> v >> d;
109.        add(u, v, d); //无向图(u,v)
110.        add(v, u, d); //无向图(v,u)
111.    }
112.    //整数 q，代表问题数量 (q<100)
113.    int questions;
114.    cin >> questions;
115.    while (questions--)
116.    {
117.        // 每行包含三个整数 C、S、E，分别表示赛车油箱容量、起点城市 S、终点城市 E
118.        int c, s, e;
119.        cin >> c >> s >> e;
120.        int expenses = BFS(c, s, e);
121.        if (expenses == -1)
122.            printf("impossible\n"); //无解
123.        else
124.            printf("%d\n", expenses);
125.    }
126.    return 0;
127. }

```

3.15 滚石柱^{NQ100}

描述

驾驭着林克的新身份，在编程之息大陆里闯荡，不知不觉小鲁的水平进入到全新的阶段了。

突然，小华的全新投影显现在小鲁面前，他满意地看着自己的得意门生林克小鲁，喜上眉梢。

“祝贺你，林克（小鲁），如今你来到了最后一个试炼，这个试炼你需要综合运用你过去所掌握的知识，方能解决”。

小鲁摘下林克头盔，深深的向小华鞠了一个躬：

“滴水之恩当涌泉想报，如今这最后一题，我愿意以真实的身份来挑战，林克毕竟只是VR的数字人，并非真人。这最后一战，学生小鲁愿意挑战。”

小华欣慰的点点头，介绍最后一题，3D的迷宫类问题—滚石柱。



这个游戏的任务是滚动一个 $1 \times 1 \times 2$ 的长方体石柱，把它滚动到目的地。

石柱在地面上有3种放置形式，“立”在地面上（ 1×1 的面接触地面）横“躺”或者竖“趟”在地面上（ 1×2 的面接触地面）



站立



横躺



竖躺



悬空的格子是禁区



硬地



易碎地

迷宫是一个 N 行 M 列的矩阵，每个位置可能是硬地（用“.”表示）、易碎地面（用“E”表示）、禁地（用“#”表示）、起点（用“X”表示）或终点（用“O”表示）。

在每一步操作中，可以按上下左右四个键之一。

按下按键之后，石柱向对应的方向沿着棱滚动90度。

任意时刻，长方体不能有任何部位接触禁地，并且不能立在易碎地面上。

字符“X”标识长方体的起始位置，地图上可能有一个“X”或者两个相邻的“X”。

地图上唯一的一个字符“O”标识目标位置。

求把石柱移动到目标位置（即立在“O”上）所需要的最少步数。

在移动过程中，“X”和“O”标识的位置都可以看作是硬地被利用。

输入

输入包含多组测试用例。

对于每个测试用例，第一行包括两个整数N和M。

接下来N行用来描述地图，每行包括M个字符，每个字符表示一块地面的具体状态。

当输入用例N=0，M=0时，表示输入终止，且该用例无需考虑。

数据范围

$$3 \leq N, M \leq 500$$

输出

每个用例输出一个整数表示所需的最少步数，如果无解则输出“Impossible”。

每个结果占一行。

输入样例

```
7 7
#####
#..X###
#..##O#
#....E#
#....E#
#.....#
#####
0 0
```

输出样例

```
10
```

参考代码(C++)

```
1. // NQ100 滚石柱
2. #include <iostream>
3. #include <cstdio>
4. #include <cstring>
5. #include <algorithm>
6. #include <queue>
7. using namespace std;
8. #define For(i, a, b) for (int i = a; i < b; i++)
9. #define N 507 //最大是 500
10. struct Stone
11. {
12.     int x, y, status; //x,y 为石柱的基块坐标; status:0 为站立 1 为横躺 2 为竖躺
13.     Stone(int a, int b, int c) { x = a, y = b, status = c; }
```



```

14.     Stone() : x(0), y(0),status(0){};
15. };
16. void displayStone(Stone &s)
17. {
18.     cout << "(" << s.x << " " << s.y << " " << s.status << ")" ";
19. }

20. //四种方向 UP=0, DOWN=1, LEFT=2, RIGHT=3, stone 从状态 0,1,2 变化
21. //每种状态的坐标以方块的左上顶点的格子为基准点
22. int direction[4][3][3] = {
23.     {{-2, 0, 2}, {-1, 0, 1}, {-1, 0, 0}},
24.     {{1, 0, 2}, {1, 0, 1}, {2, 0, 0}},
25.     {{0, -2, 1}, {0, -1, 0}, {0, -1, 2}},
26.     {{0, 1, 1}, {0, 2, 0}, {0, 1, 2}}};
27. Stone moveStone(Stone &p, int udlf)
28. {
29.     //返回下一个状态
30.     int dx = direction[udlf][p.status][0];
31.     int dy = direction[udlf][p.status][1];
32.     int newstatus = direction[udlf][p.status][2];
33.     return Stone(p.x + dx, p.y + dy, newstatus);
34. }
35. //滚动区域
36. int n, m;
37. char area[N][N];
38. int dist[N][N][3];
39. queue<Stone> q;           //? BFS 使用的队列
40. Stone start, target, qHead; //?起点, 终点, 队列中的头坐标
41. bool isInside(int x, int y)
42. {
43.     return x >= 0 && x < n && y >= 0 && y < m;
44. }
45. bool isValid(Stone node) //判断:在范围内 Stone 的左上块为基准点判定
46. {
47.     if (!isInside(node.x, node.y) || area[node.x][node.y] == '#')
48.         return false; //不在范围内
49.     if (node.status == 2 && (!isInside(node.x + 1, node.y) || area[node.x + 1][node.y] == '#'))
50.         return false; //竖躺碰到禁区
51.     if (node.status == 1 && (!isInside(node.x, node.y + 1) || area[node.x][node.y + 1] == '#'))
52.         return false;
53.     if (node.status == 0 && area[node.x][node.y] == 'E')
54.         return false;
55.     return true;
56. }

```

```

57. bool isVisited(Stone node)
58. {
59.     return dist[node.x][node.y][node.status] != -1; //该状态没有被访问过
60. }
61. char readChar()
62. {
63.     char c = getchar();
64.     while (c != '#' && c != '.' && c != 'X' && c != 'O' && c != 'E')
65.         c = getchar();
66.     return c;
67. }
68. void BuildMap(int n, int m)
69. {
70.     memset(area, '#', sizeof(area)); //初始化为禁区
71.     memset(dist, -1, sizeof(dist)); //初始化
72.     //读入是 char, 内容为 01
73.     //Todo: 根据题意建立地图
74.     For(i, 0, n) For(j, 0, m) area[i][j] = readChar();
75.     //Todo: 寻找 start 和 target
76.     For(i, 0, n) For(j, 0, m)
77.     {
78.         char c = area[i][j];
79.         if (c == 'X')
80.         { //初始化 1 的起点距离为 0
81.             //搜索相邻格是否有 'X'
82.             start.x = i, start.y = j, start.status = 0, area[i][j] = '.'; //找到起点, 站立
83.             if (isInside(i,j+1) && area[i][j + 1] == 'X')
84.                 start.status = 1, area[i][j + 1] = '.'; //横躺
85.             if (isInside(i+1,j) && area[i + 1][j] == 'X')
86.                 start.status = 2, area[i + 1][j] = '.'; //竖躺
87.         }
88.         if (c == 'O')
89.             target.x = i, target.y = j, target.status = 0; //必须站立才能过关
90.         // cout<<area[i][j];
91.         // if(j==m-1) cout<<endl;
92.     }
93. } // end of BuildMap
94. int bfs(Stone &s)
95. {
96.     while (q.size())
97.         q.pop(); //清空队列
98.     // 压入顶点
99.     q.push(s);

```

```

100.     dist[s.x][s.y][s.status] = 0; //起始点步数为0
101.     while (q.size())
102.     {
103.         qHead = q.front(); //栈顶
104.         q.pop();
105.         //向4个方向 Up Down Left Right 尝试
106.         For(i, 0, 4)
107.         {
108.             //扩展 qHead 得到下一个坐标格
109.             Stone next = moveStone(qHead, i);
110.             if (!isValid(next)) continue; //不合法就 continue
111.             //更新 next 结点距离并且
112.             if (!isVisited(next))
113.             {
114.                 dist[next.x][next.y][next.status] = dist[qHead.x][qHead.y][qHead.status] + 1;
115.                 q.push(next);
116.                 //如果 next 是终点
117.                 if (next.x == target.x && next.y == target.y && next.status == target.status)
118.                     return dist[next.x][next.y][next.status];
119.             }
120.         }
121.     } // end of while
122.     return -1;
123. } // end of bfs

124. int main()
125. {
126.     while (cin >> n >> m && n)
127.     {
128.         BuildMap(n, m);
129.         int res = bfs(start);
130.         if (res == -1)
131.             cout << "Impossible" << endl;
132.         else
133.             cout << res << endl;
134.     }
135. }

```

后记

1. 本文的所有代码由Planetb提供格式转换: <http://www.planetb.ca/syntax-highlight-word>
2. 本文的所有代码在VSCode下编辑, 使用Devcpp编译通过, 并且在xmuoj.com上全部AC。
3. 本文所有的题目都可以在<http://xmuoj.com>上在线练习。
4. 本书的第三章的部分题解已经上传B站, 有兴趣的可以关注B站的Up主: 厦大的Andy。



5. 爱算法的同学, 笔者推荐ACWing (<https://www.acwing.com>)。要提前预备社招和面试的同学推荐, 笔者推荐力扣 (<https://leetcode-cn.com>)。
6. 本书为5DG在线编程团队撰写的第一本教材讲义, 虽然经过团队成员的多轮校稿, 但是难免有错误和不足之处, 恳请见谅, 请发现本书错误的同学邮件联系笔者, [反馈错误 37199625@qq.com](mailto:37199625@qq.com)。
7. 本书的参考代码虽然能够AC题目, 但是并非每一个都是最优的代码, 我们把AC代码全部打印出来, 目的是起到抛砖引玉作用, 希望同学们AC题目后能够改进优化现有代码, 写出优雅效率更好的代码。如果事情如此成就, 那么笔者的班门弄斧也算是一桩美事。因此, 我们诚邀各位同学贡献自己写的更好的代码给我们, 让本书的下一版AC代码能够更加有质量, 使后来人得着益处。